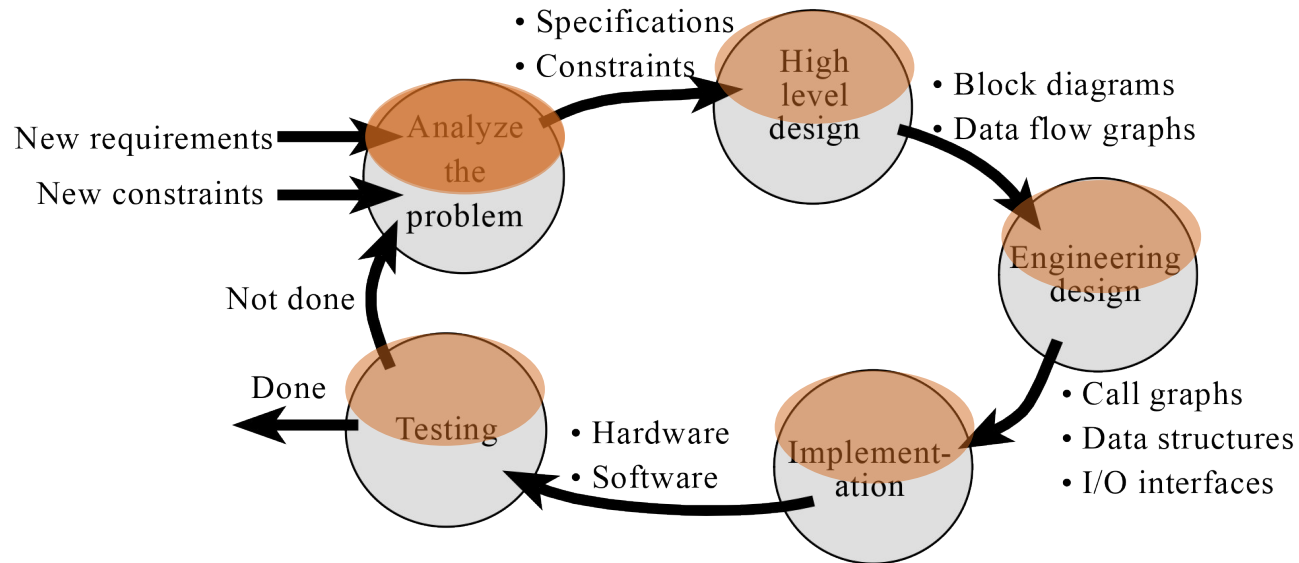


System/SW Design

- **System Design**
- SW/Program Design
- Finite State Machine (FSM)
- Software Architecture
- Interfacing
and I/O Synchronization
- Debugging Techniques



Product Development Cycle



- Analysis (What?)
 - Requirements -> Specifications

- Design (How?)
 - High-Level: Block Diagrams
 - Engineering: Algorithms, Data Structures, Interfacing

- Implementation(Real)
 - Hardware, Software
- Testing (Works?)
 - Validation: Correctness
 - Performance: Efficiency
- Maintenance (Improve)

Product Development Cycle

High-Level Design

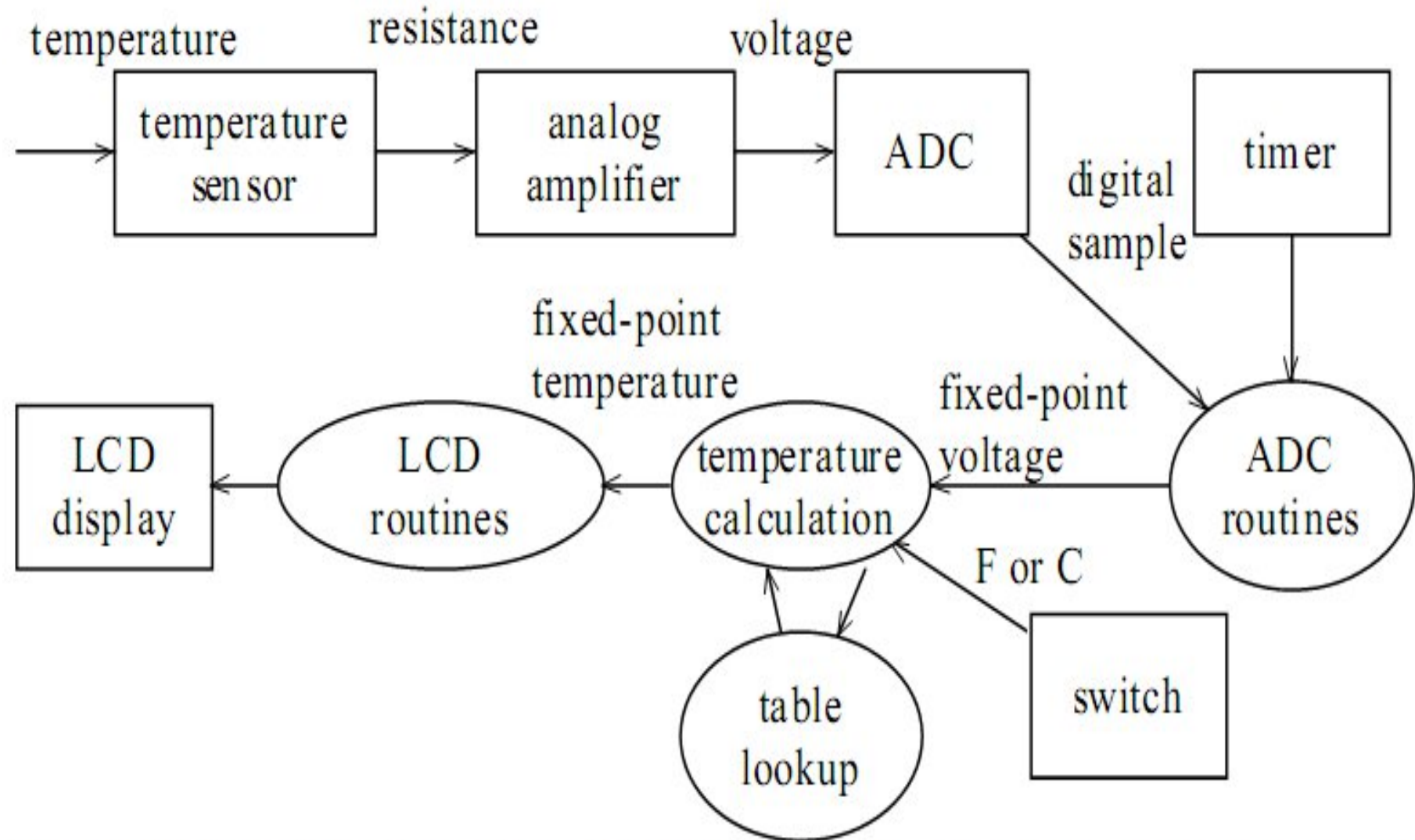
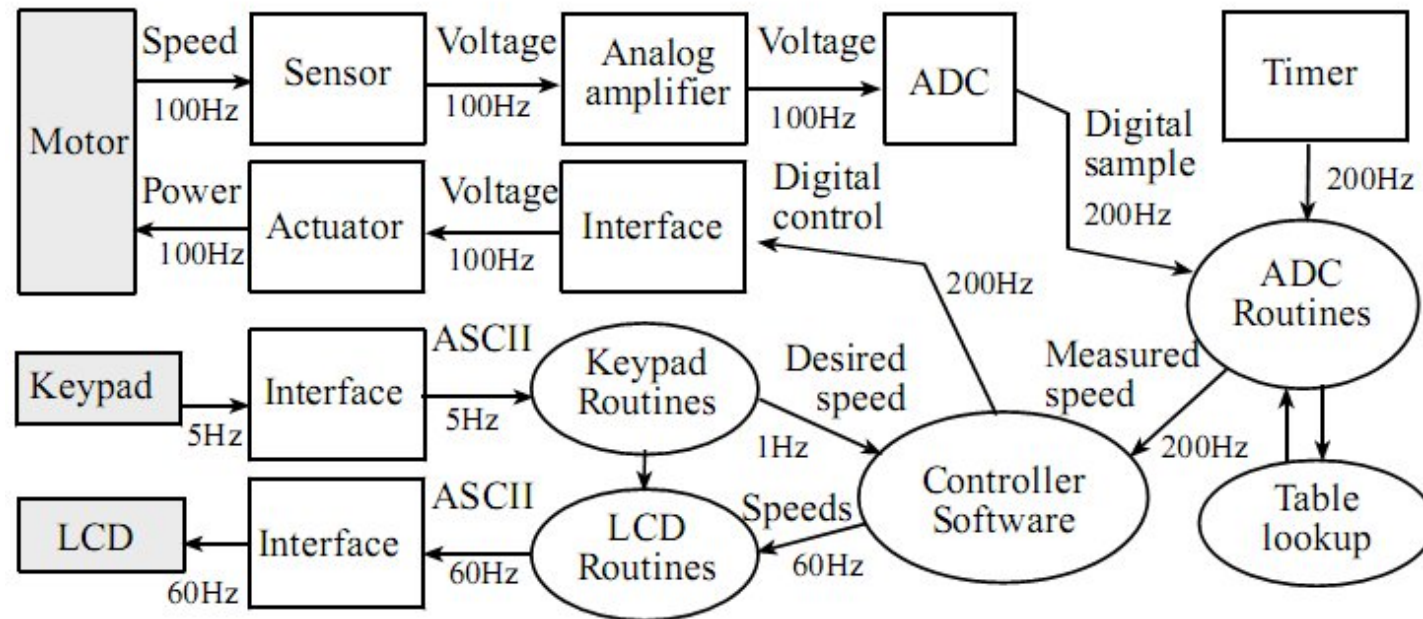


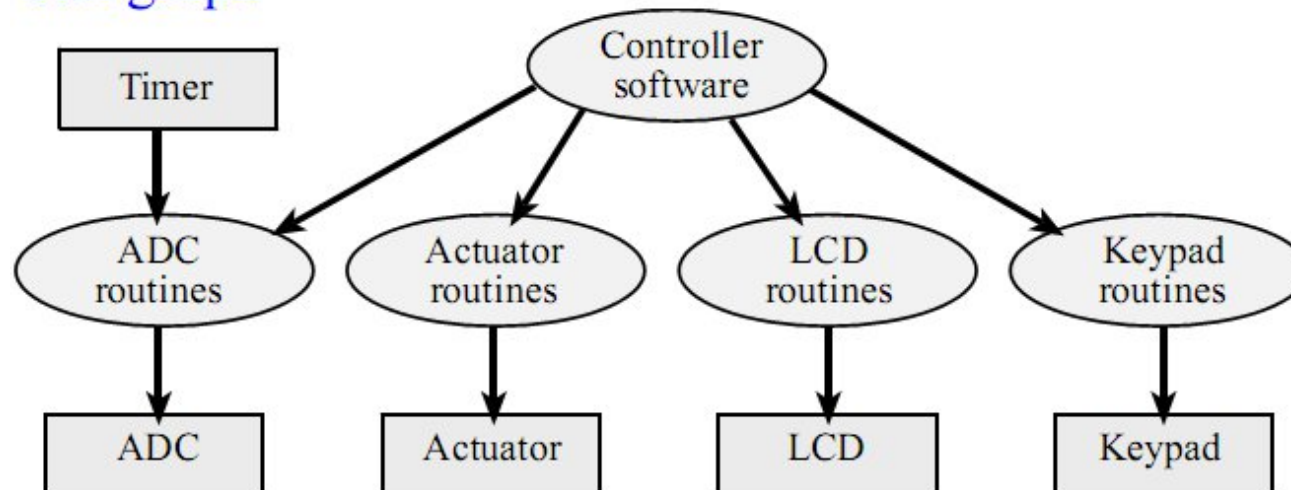
Figure 1.5. A data flow graph showing how the temperature signal passes through a simple thermometer.

Complex Example of Dataflow and Call Graph

Data flow



Call graph



System/SW Design

- System Design
- **SW/Program Design**
- Finite State Machine (FSM)
- Software Architecture
- Interfacing
and I/O Synchronization
- Debugging Techniques



Program Design

- The program design stage lays out the structure and algorithms of the software
- The structure and algorithms may be laid out as:
 - **flowchart**
 - timing diagram
 - description of a protocol
 - memory map; or
 - equation

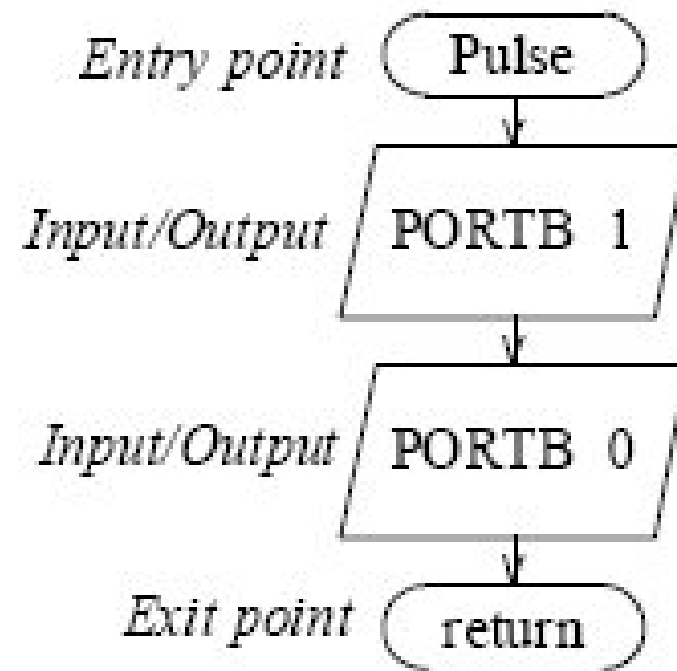
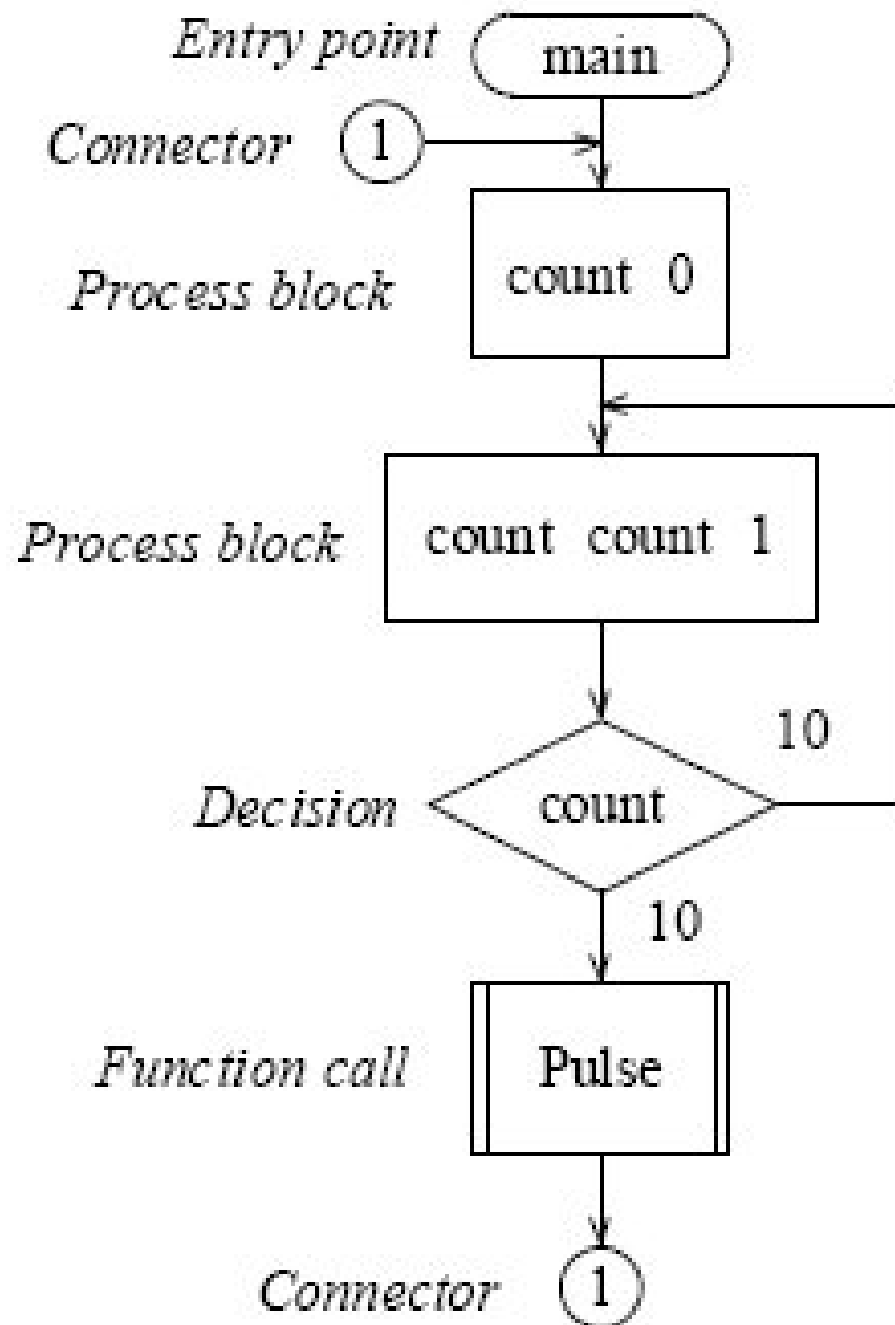
Flowcharts

- Abstraction for describing software algorithms:
 - two dimensional
 - conditional branching
 - function calls
 - interrupts

Flowcharts

- The oval shapes define entry and exit points. The main entry point is the starting point of the software.
- Each function, or subroutine, also has an entry point. The exit point returns the flow of control back to the place from which the function was called.
- We use rectangles to specify *process* blocks. In a high-level flowchart, a process block might involve many operations, but in a low-level flowchart, the exact operation is defined in the rectangle.
- The parallelogram will be used to define an input/output operation. Some flowchart artists use rectangles for both processes and input/output. Since input/output operations are an important part of embedded systems, we will use the parallelogram format, which will make it easier to identify input/output in our flowcharts.
- The diamond-shaped objects define a branch point or decision block.
- The rectangle with double lines on the side specify a call to a predefined function/subroutine/procedure.
 - Functions usually return a result parameter, while procedures usually do not.
 - Functions and procedures are terms used when describing a high-level language
 - Subroutines are often used when describing assembly language.
- Circles are used as connectors.
 - In general, it is bad programming style to develop software that requires a lot of connectors when drawing its flowchart.

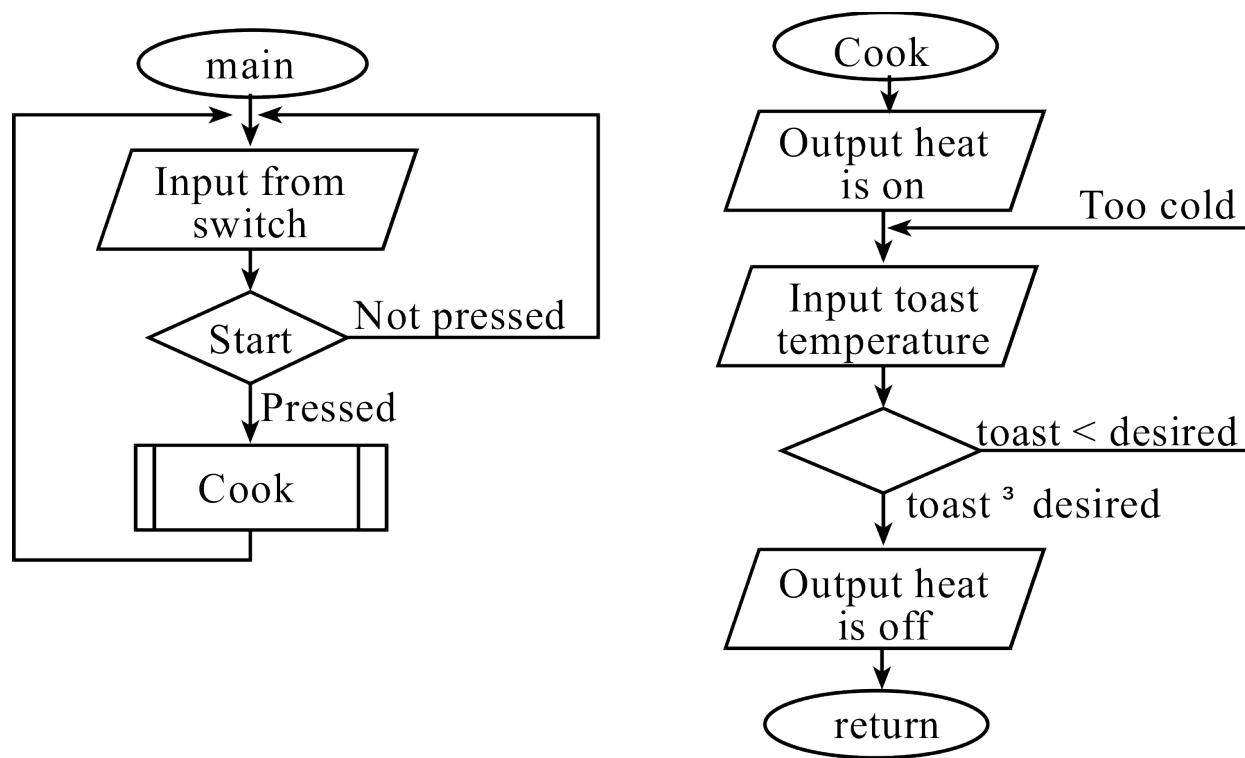
Flowcharts



```
void Pulse(void) {  
    PORTB = 1;  
    PORTB = 0;  
}  
void main(void) { int counts;  
    while (1) {  
        count = 0;  
        do {  
            count ++;  
        }  
        while (count < 10);  
        Pulse();  
    }  
}
```

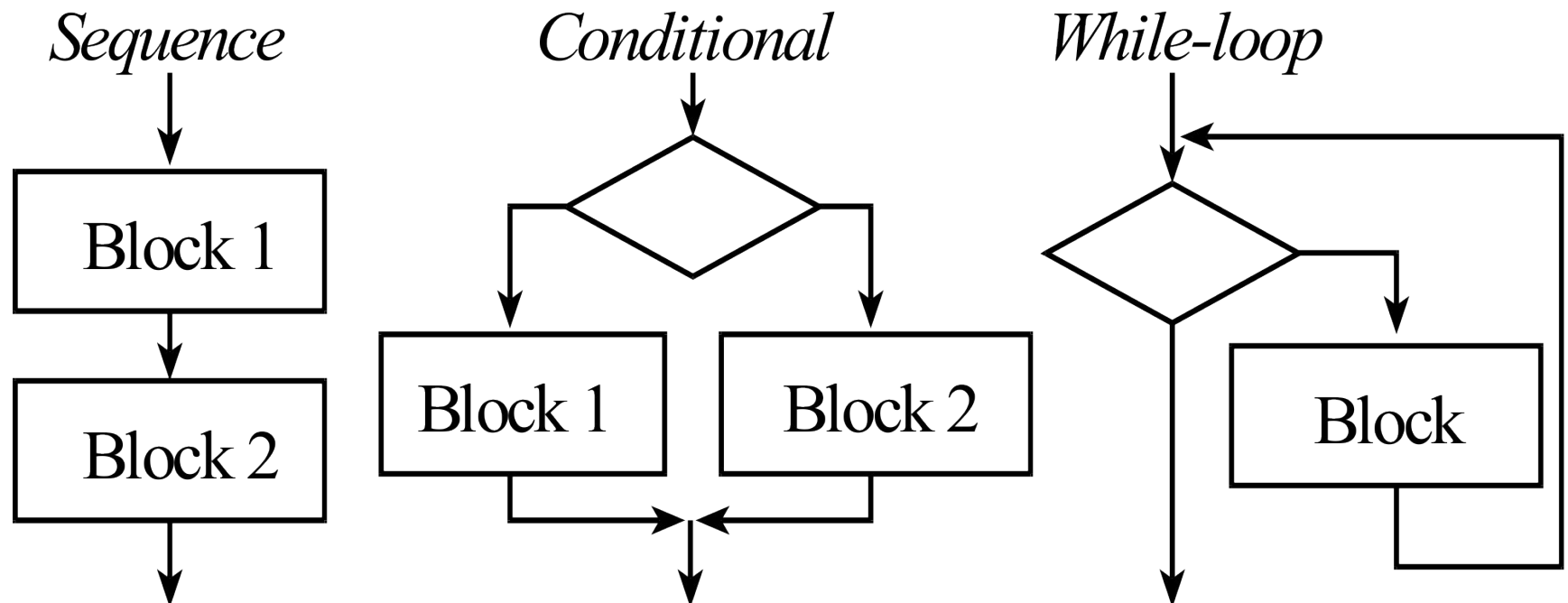
Flowcharts Example

- Design a flowchart for a toaster:
 - Using a flowchart describe the control algorithm that a toaster must use to cook toast. Assume the inputs are toast temperature in F, and desired temperature in F. The output is heat (on/off).



Flowcharts: Structured Programming

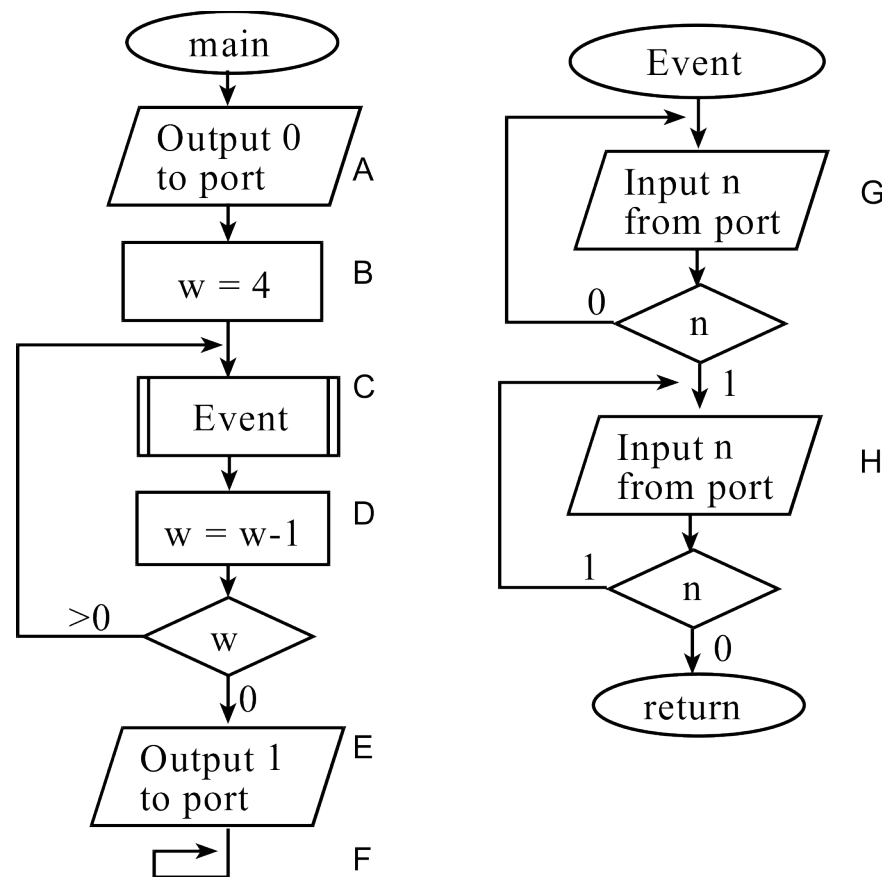
- Structured programs are built from three basic building blocks: **the sequence, the conditional, and the while-loop.**
- At the lowest level, the process block contains simple and well-defined commands. I/O functions are also low-level building blocks.
- Structured programming involves combining existing blocks into more complex structures, as shown in the figure.



Structured programming building blocks

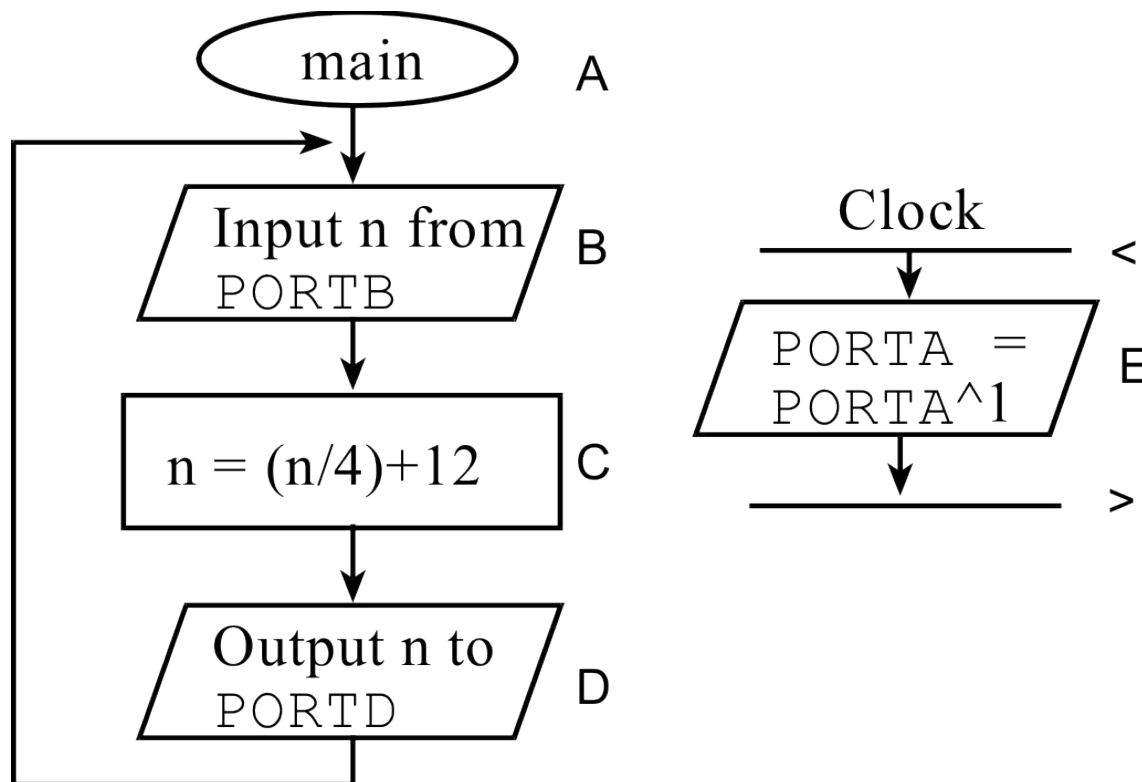
Flowcharts Example

- Turn on after switch pushed 4 times:



Flowchart Example with Interrupt

- **Example 1.3.** Design a flowchart for a system that performs two independent tasks. The first task is to output a 20 kHz square wave on **PORTA** in real time (period is 50 ms). The second task is to read a value from **PORTB**, divide the value by 4, add 12, and output the result on **PORTD**. This second task is repeated over and over.



```
void SysTick_Handler(void) {
    PORTA = PORTA ^ 0x01; ← E
}

void main(void) { ← A
    unsigned long n;
    while(1) {
        n = PORTB; ← B
        n = (n/4) + 12; ← C
        PORTD = n; ← D
    }
}
```

Flowchart Example with Interrupt

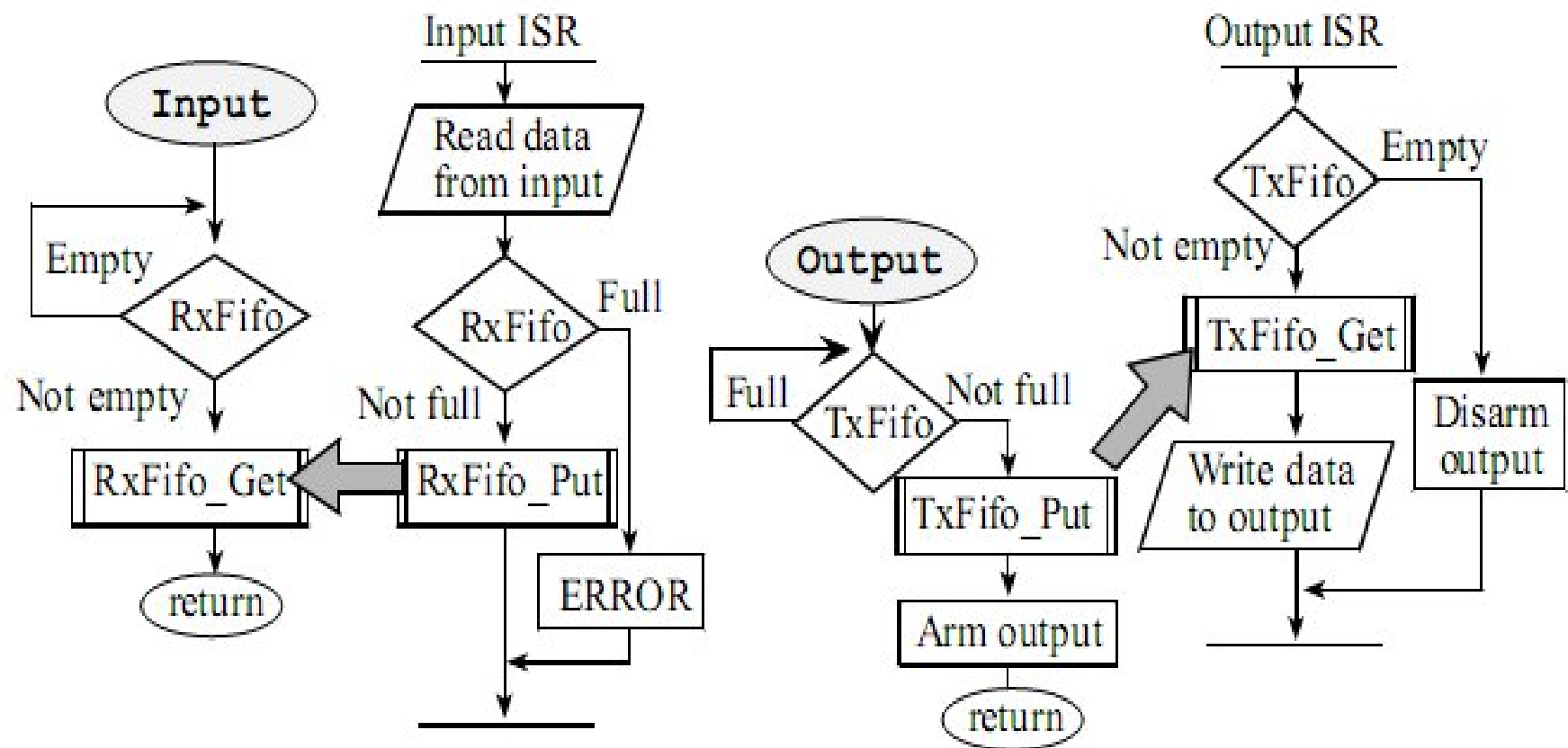
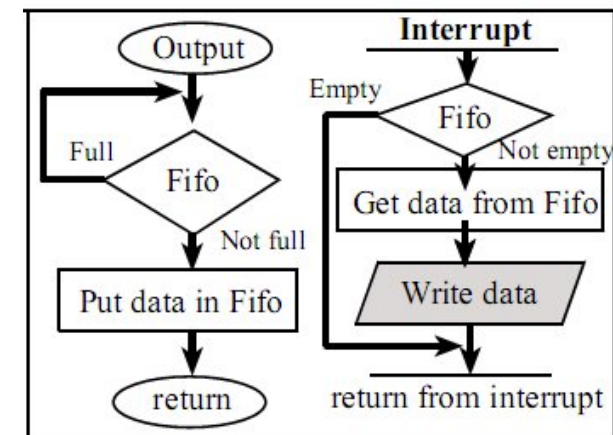
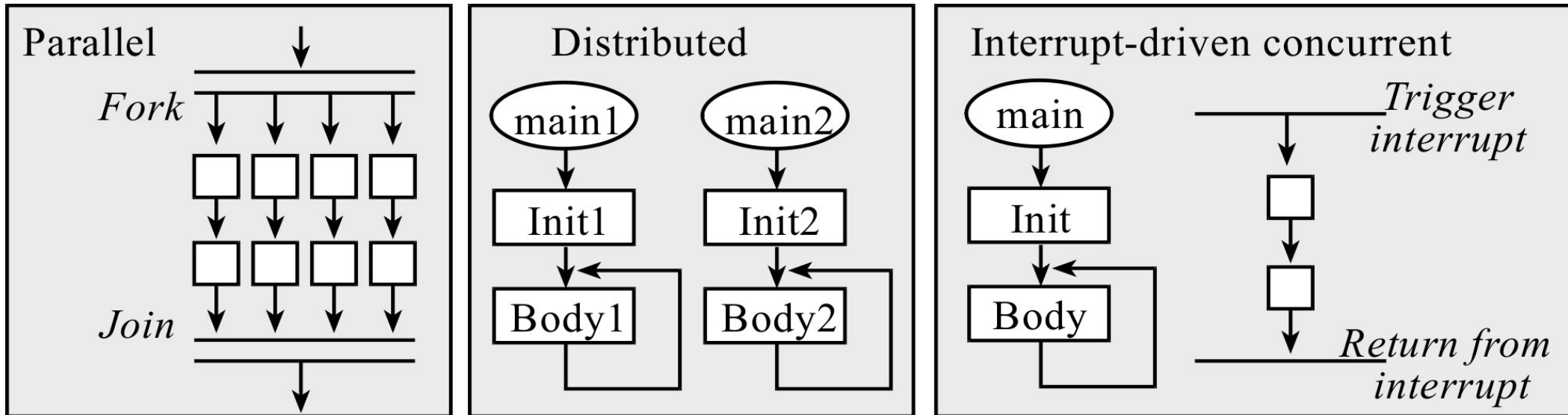
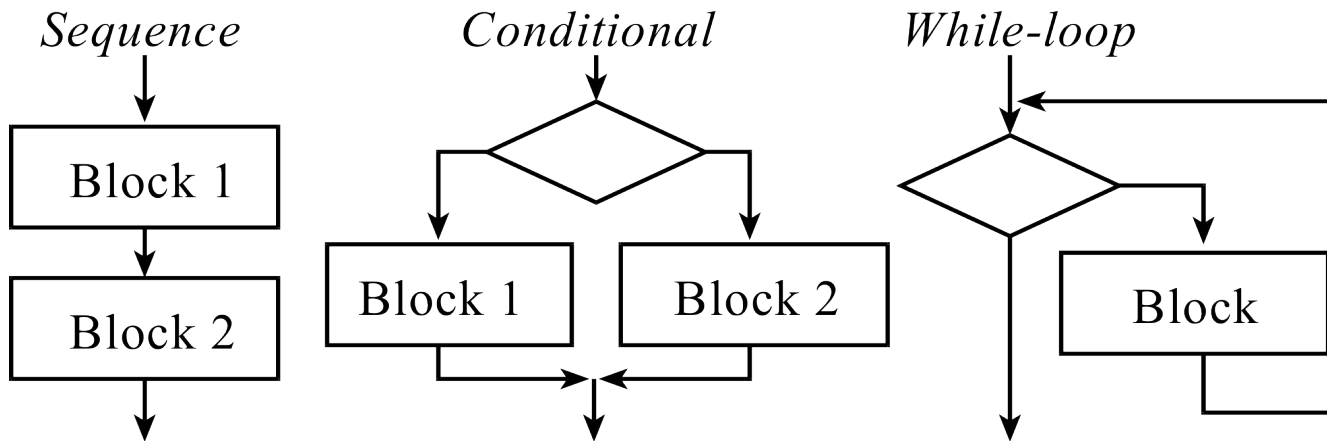


Figure 5.4. In a producer/consumer system, FIFO queues can be used to pass data between threads.

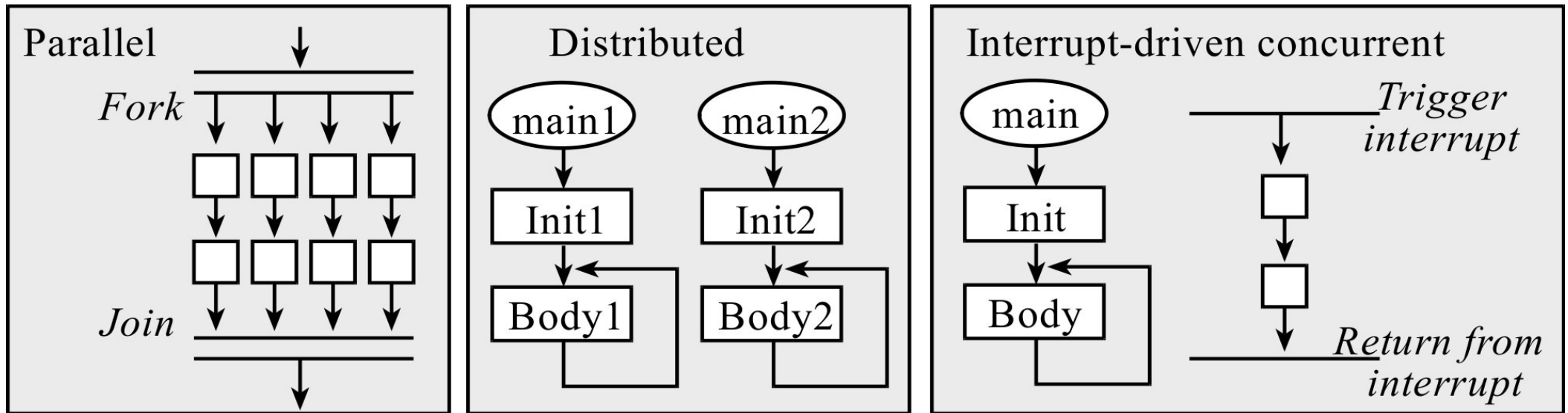


Structured Programming

- Common Constructs (as Flowcharts)



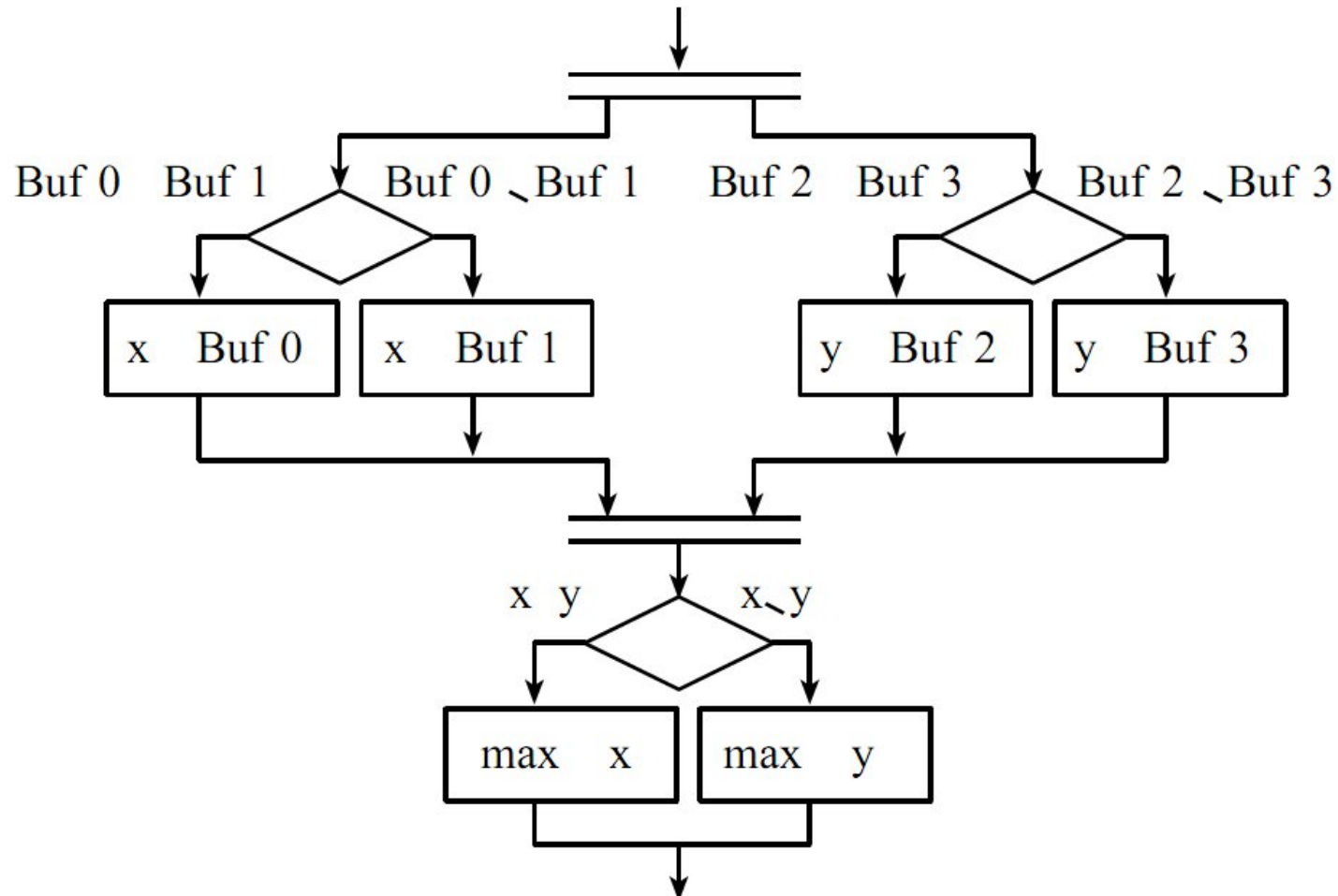
Flowcharts



Structured programming building blocks

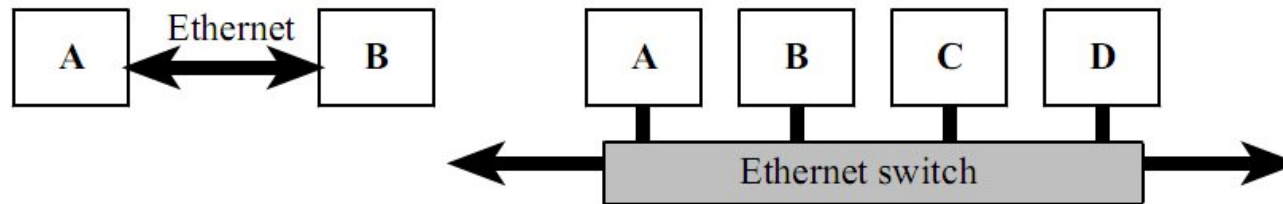
Parallel Processing

- Multiple processors, shared memory
- Simultaneous execution of two or more software tasks
- e.g., multi-core Pentium

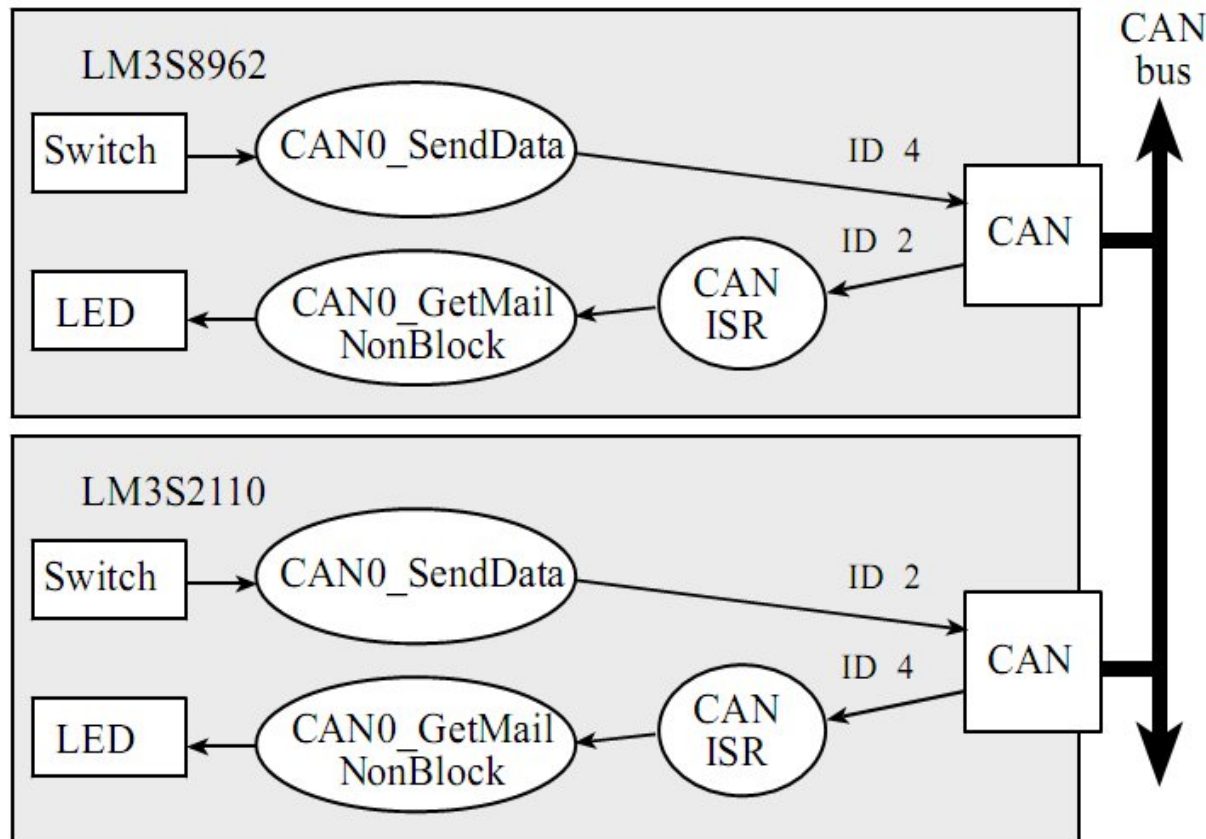


Distributed Processing

- Multiple computers, separate memory, I/O network link
- Simultaneous execution of two or more software tasks



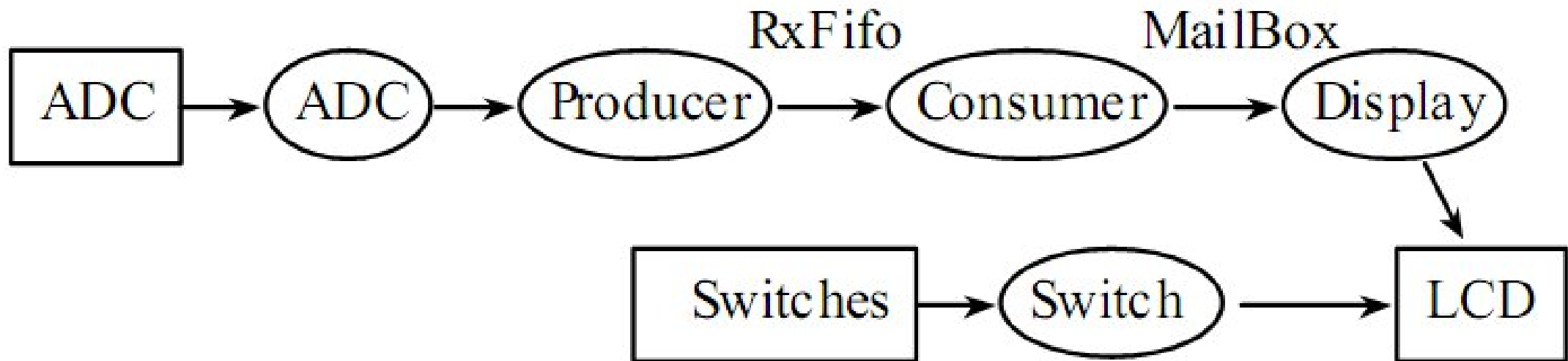
Ethernet has a bus-based topology.



Simple CAN network.

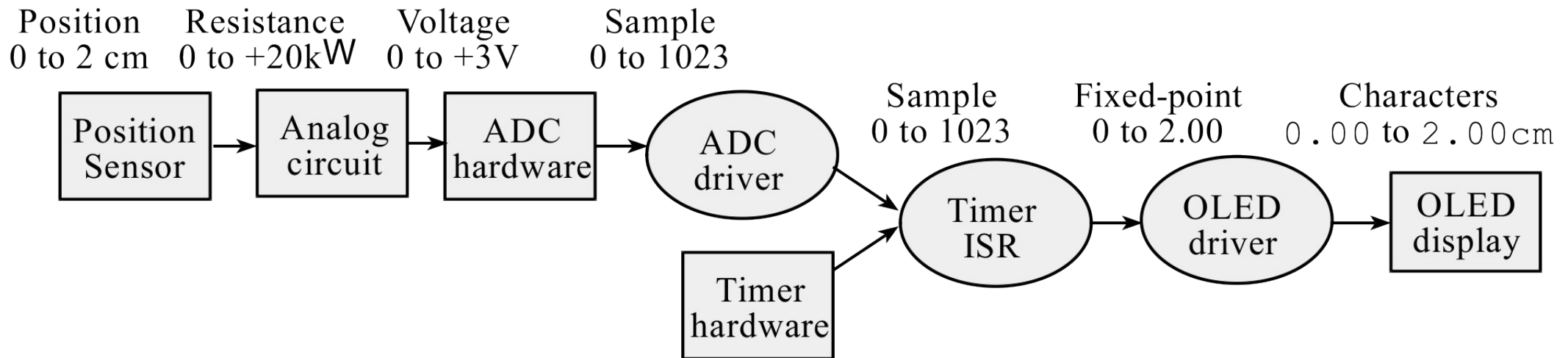
Multithreading (Concurrent)

- One foreground and multiple background threads
- Multiple foreground threads using a thread scheduler



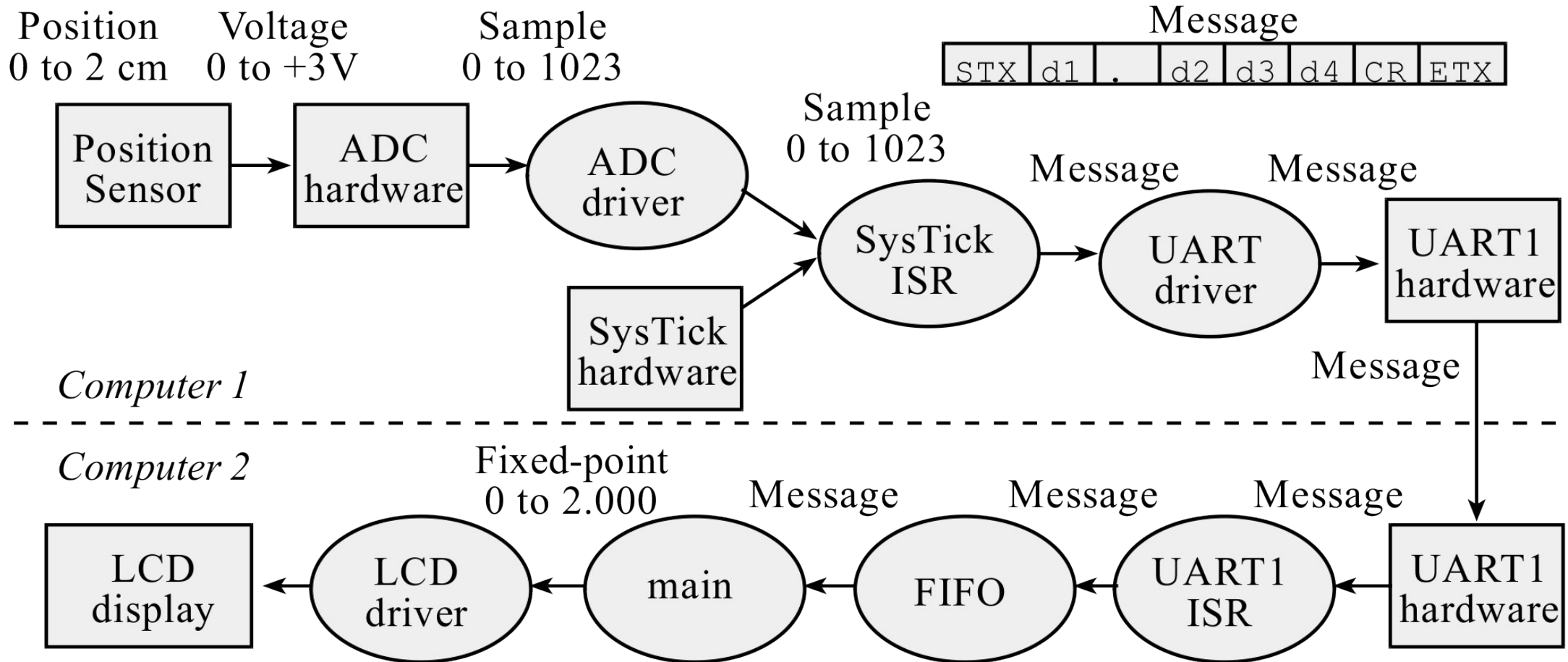
Data Flow Graphs

high level design tool showing flow of information

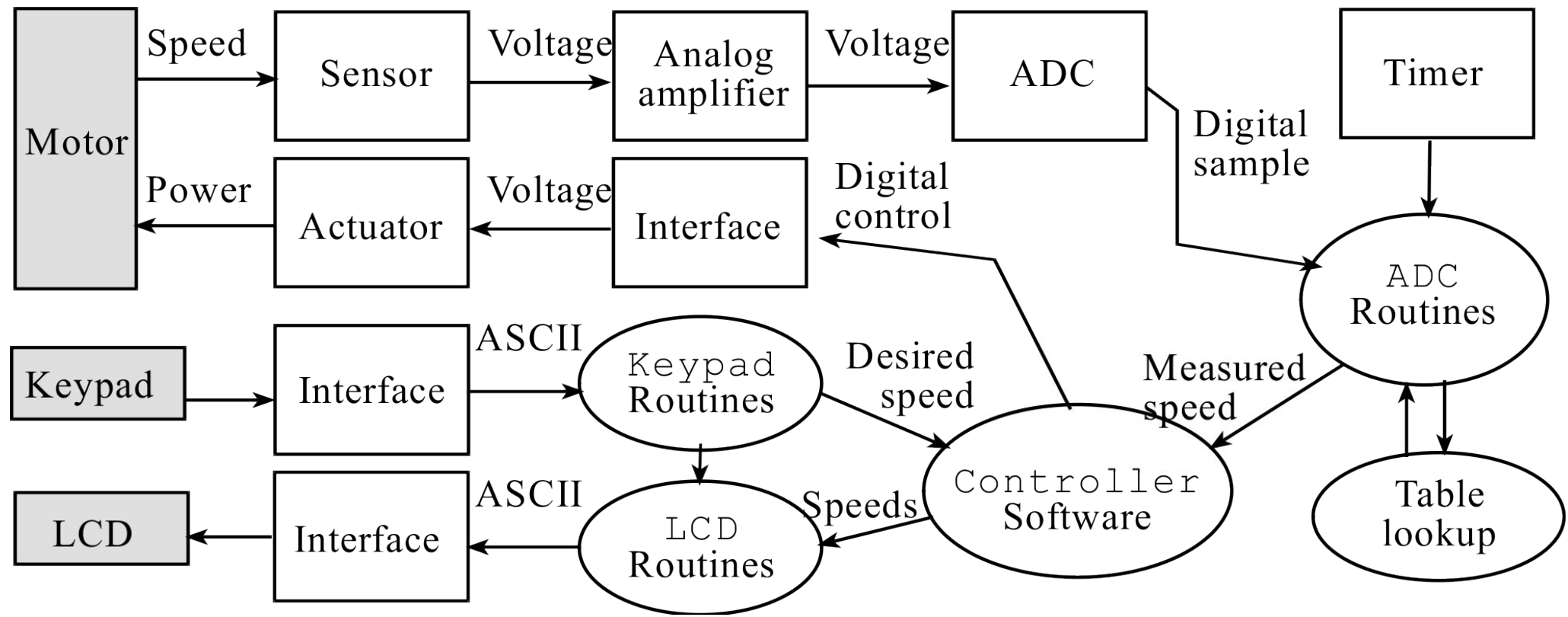


ovals => software; rectangles => hardware

Example: Data Flow Graphs



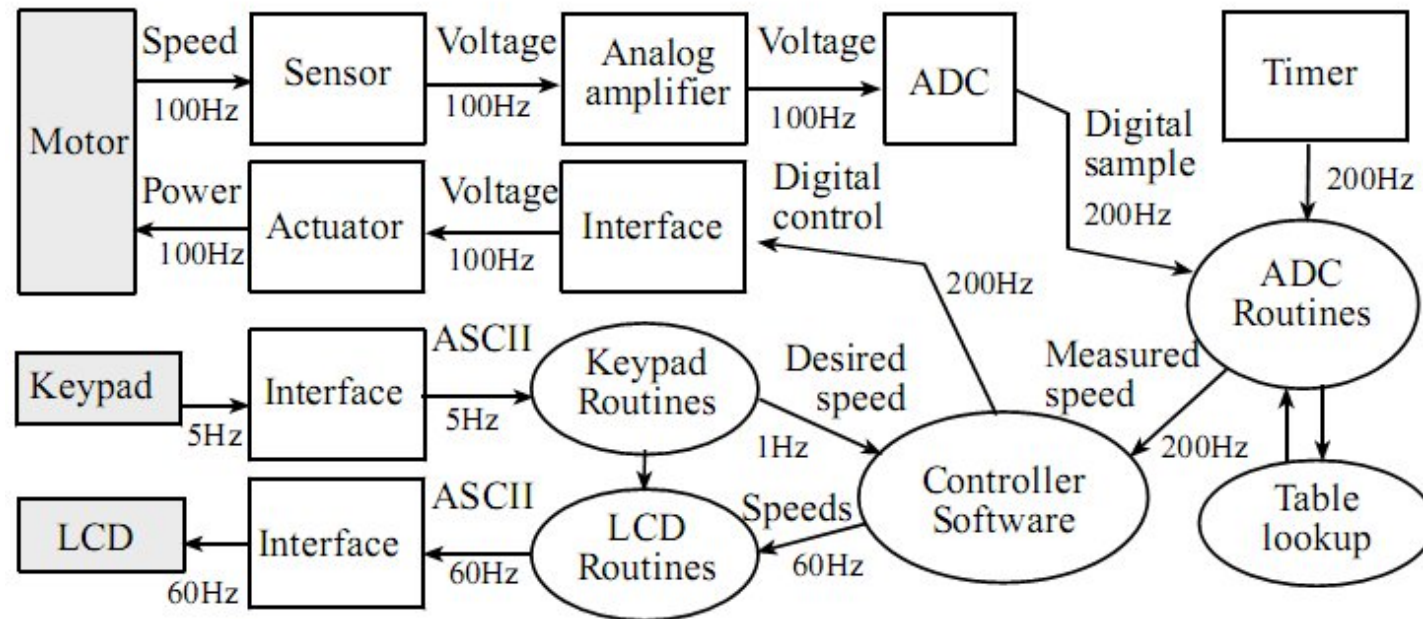
Complex Example of Dataflow and Call Graph



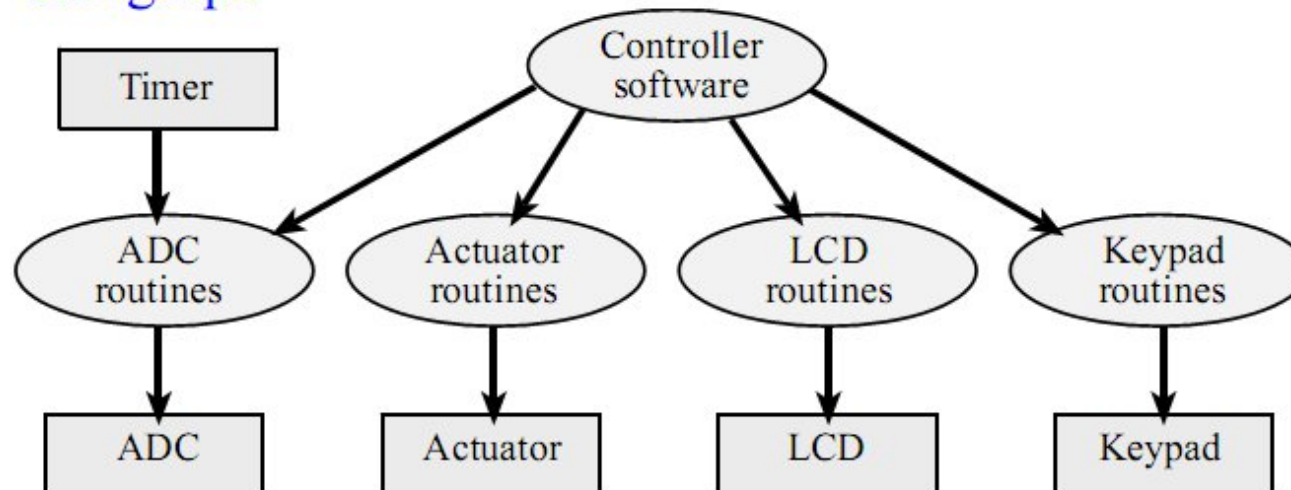
Motor controller system

Complex Example of Dataflow and Call Graph

Data flow

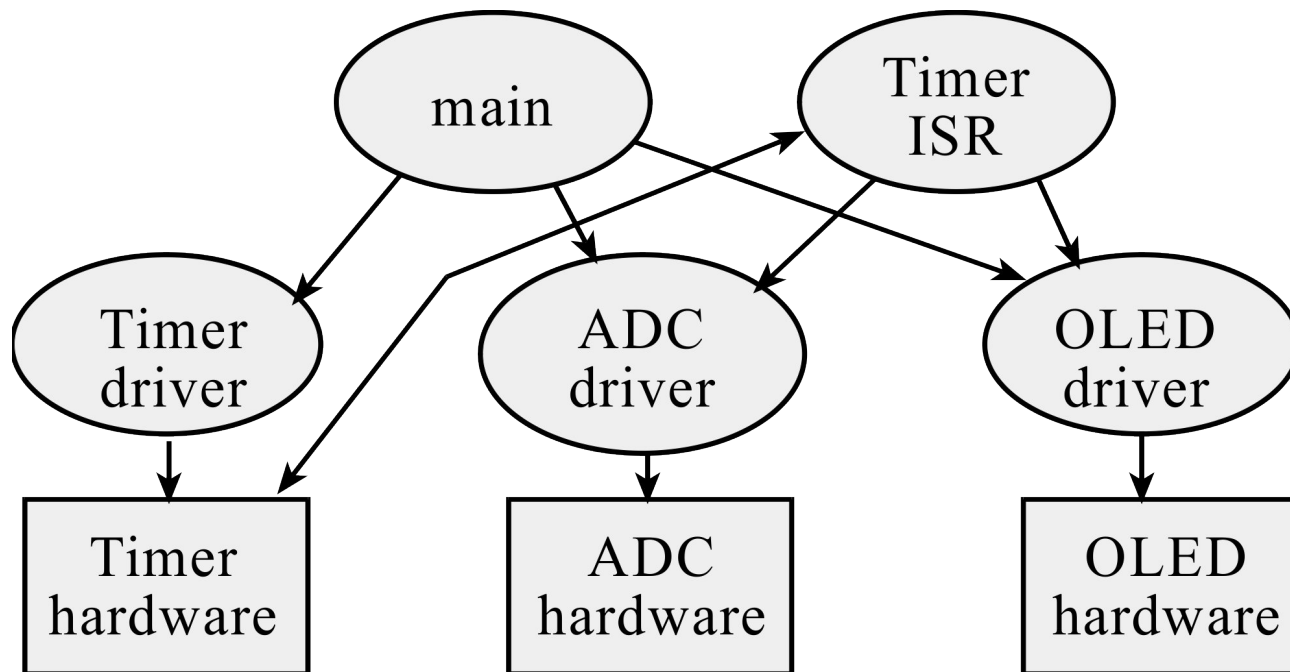


Call graph



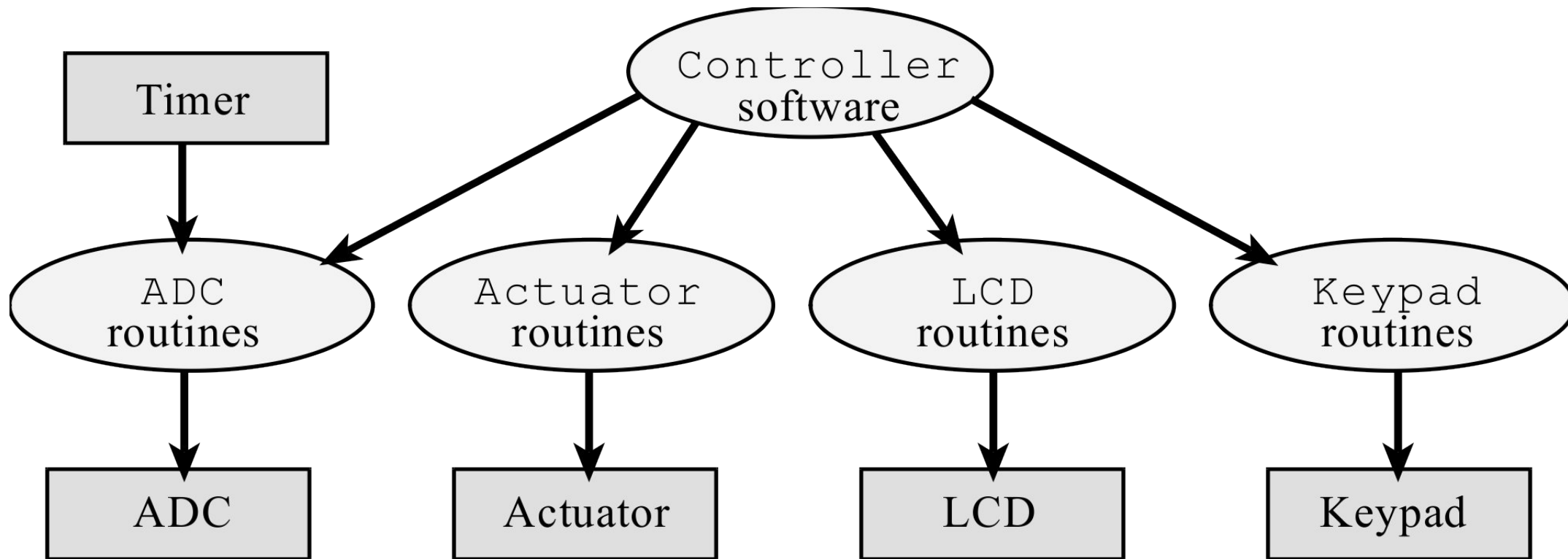
Call Graphs

A graphical abstraction to describe the interaction between software and hardware components



Position measurement system

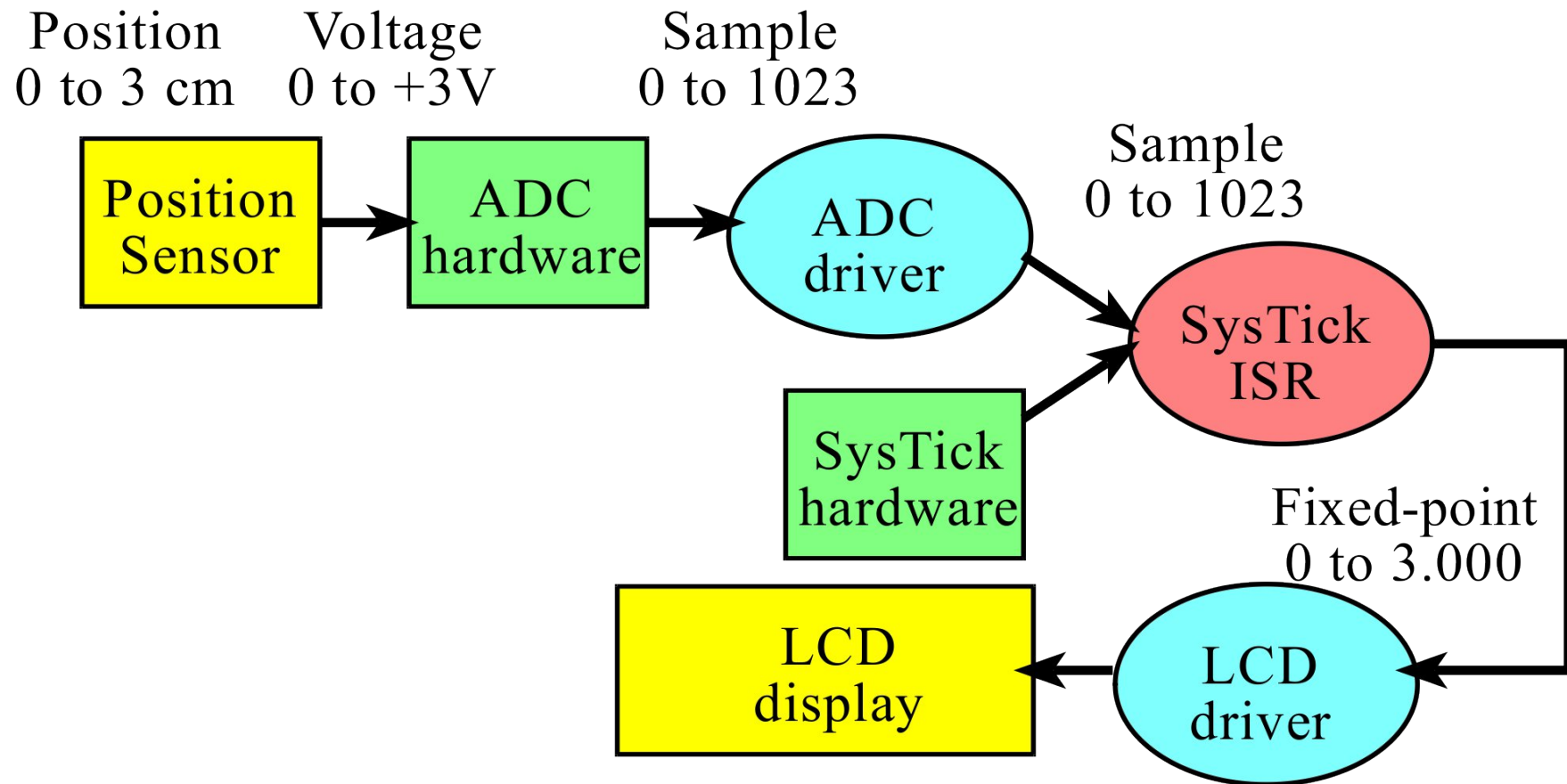
Example: Call Graphs



Motor controller call graph

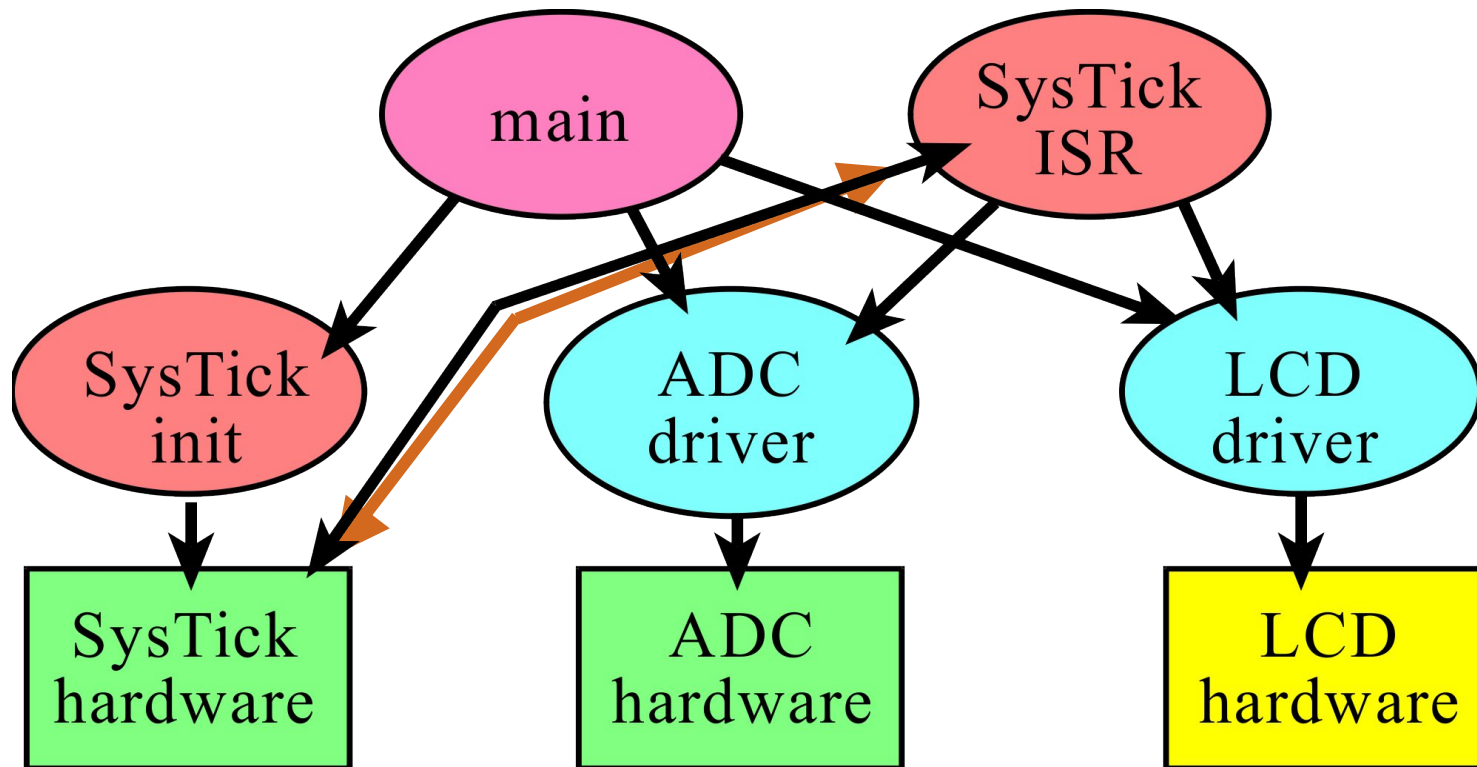
Example: Data Flow Graph for DAS

Position Measurement System



Example: Call Flow Graph for DAS

Position Measurement System



Writing Code

- If the program is well specified and the algorithm design stage has been thorough, the actual code writing stage can become almost mechanical
- By defining the software at the outset, before code is written, a much more defined, integrated, set of code can be produced
- The extra space occupied by comments costs nothing, and if the comments are well laid out there is no possibility that they can detract from understanding the code

SW Style: Naming Convention

type	examples
constants	<code>SAFE_TO_RUN</code> <code>START_OF_RAM</code> <code>PORTA</code>
local variables	<code>maxTemperature</code> <code>lastCharTyped</code> <code>errorCnt</code>
private global variable	<code>MaxTemperature</code> <code>LastCharTyped</code> <code>ErrorCnt</code>
public global variable	<code>DAC_MaxTemperature</code> <code>Key_LastCharTyped</code> <code>Network_ErrorCnt</code>
private function	<code>ClearTime</code> <code>wrapPointer</code> <code>InChar</code>
public function	<code>Timer_ClearTime</code> <code>RxFifo_Put</code> <code>Key_InChar</code>

Examples of names.

C language Style Guidelines: Code Files

- Code File Structure, the *.c file

Opening comments in the code file.

- Intended to be read by co-worker
 - first line should contain the file name.
 - the overall purpose of the software module,
 - the names of the programmers,
 - the creation (optional) and last update dates,
 - the hardware/software configuration required to use the module,
 - any copyright information.
- Including .h files.
 - will help us draw a call-graph
 - avoid having one header file include other header files
 - only those files that are absolutely necessary

C language Style Guidelines: Code Files

- #define statements.
 - they will be private
 - client does not need to use
- struct union enum statements.
 - create the necessary data structures
 - they will be private.
- Global variables and constants.
 - If static then it will be private
 - If no static then it will be public
- The scope of a variable includes all the software in the system that can access it. In general, we wish to minimize the scope of our data.

```
char publicGlob;      // any function
static char privGlob; // this file only
void function(void){
static char veryPrivGlob; // this function only
}
```

C language Style Guidelines: Code Files

- Prototypes of private functions
 - maintains a top-down organization
 - private functions by defining them as static.
 - include the parameter names with the prototypes.
E.g., which of the following prototypes is easier to understand?
 - `static void plot(short, short);`
 - `static void plot(short time, short pressure);`
- Implementations of the functions
 - private functions should be defined as static
 - sequenced in a logical manner
 - highest level to lowest level, showing hierarchy
 - order in which the functions will be used
 - open
 - input
 - output
 - close

C language Style Guidelines: Header Files

- Header File Structure, the *.h file (show sci12.h, heap.h)

There are two types of header files.

- 1) has no corresponding code file (not part of a module)
 - list global constants and helper macros
 - I/O port addresses and calibration coefficients
 - Debugging macros could be grouped together
 - global in nature and do not belong to a module
- 2) has a corresponding code file (part of a module)
 - define the prototypes for public functions
 - contains the policies (behavior or what it does)
 - read by the client
 - often one sells header file with compiled object code

C language Style Guidelines: Header Files

- Opening comments in header file.
 - intended to be read by the client
 - first line should contain the file name.
 - the overall purpose of the software module,
 - the names of the programmers,
 - the creation (optional) and last update dates,
 - the HW/SW configuration required to use module
 - any copyright information.
- Including .h files
 - Nested includes in the header file should be avoided

C language Style Guidelines: Header Files

- `#define` statements.
 - Public constants and macros are next.
- Public or private?
 - begin with everything private, and then shift if necessary
 - relates to how to use the module, then public.
 - relates to how it works or how it is implemented, private.
- `struct union enum` statements.
 - public data structures
- Global variables and constants
 - public global variables should be avoided
- Prototypes of public functions
 - arrange in a meaningful order
 - comments should be directed to the client
 - clarify what the function does
 - explain how the function can be used.

C language Style Guidelines: Formatting

- Formatting
 - easier to understand,
 - easier to debug, and
 - easier to change.
- Make the software easy to read on the screen.
 - no hardcopy printouts during the development phase
 - looks pretty on the computer screen.
 - no horizontal scrolling allowed
 - If we do make printouts then it will be easy to read.
 - functions should understandable without vertical scrolling
- Indentation should be set at 2 spaces.
 - no tabs
 - Local variables
 - on same line as function definition, or
 - in first column on next line.

C language Style Guidelines: Formatting

- Be consistent about where we put spaces.
 - no space before a comma or a semicolon,
 - at least one space or return after comma or semicolon.
 - no space before or after open or close parentheses.
 - Assignment and comparison operations
 - should have a single space before and after the operation.
 - we can line up the operators and values. For example

```
data    = 1;
pressure = 100;
voltage = 5;
```

```
void main(void){ int i, j, k;
    j = 1;
    if(sub0(j)){
        for(i = 0; i < 6; i++){
            sub1(i);
        }
        k = sub2(i, j);
    }
    else{
        k = sub3();
    }
}
```

- Be consistent about where we put braces {}.
 - braces cause both syntax and semantic errors
 - opening brace visual clue that a new code block has started
 - close brace gives visual clue that code is in different block

Bad

```
if(flag)
    n = 0;
```

Good

```
if(flag){
    n = 0;
}
```

Good

```
if(flag)
{
    n = 0;
}
```

C language Style Guidelines: Code Structure

- Make the presentation easy to read
 - the majority of a function fits on a single computer screen.
 - reduce the 2-D area of our functions
 - encapsulating components
 - and defining them as private functions,
 - combining multiple statements on a single line.
 - group multiple statements on a single line
 - if the collection makes sense
 - we can draw a circle around the statements
 - and assign a simple collective explanation
 - ++ and -- should not appear in complex statements.

bad	good
<code>*(--pt) = buffer[n++];</code>	<code>--pt;</code>
	<code>*(pt) = buffer[n];</code>
	<code>n++;</code>

C language Style Guidelines: Code Structure

The first example has a horrific style.

```
void testFilter(long start, long stop, long step){ long x,y;
    initFilter();UART_OutString("x(n) y(n)"); UART_OutChar(CR);
    for(x=start;x<=stop; x=x+step){ y=filter(x); UART_OutUDec(x);
    UART_OutChar(SP); UART_OutUDec(y); UART_OutChar(CR);} }
```

The second example places each statement on a separate line, unnecessarily vertical.

```
void testFilter(long start, long stop, long step){
long x;
long y;
    initFilter();
    UART_OutString("x(n) y(n)");
    UART_OutChar(CR);
    for(x = start; x <= stop; x = x+step){
        y = filter(x);
        UART_OutUDec(x);
        UART_OutChar(SP);
        UART_OutUDec(y);
        UART_OutChar(CR);
    }
}
```

The last example collections are considered as a single object.

```
void testFilter(long start, long stop, long step){ long x, y;
    initFilter();
    UART_OutString("x(n) y(n)"); UART_OutChar(CR);
    for(x = start; x <= stop; x = x+step){
        y = filter(x);
        UART_OutUDec(x); UART_OutChar(SP);
        UART_OutUDec(y); UART_OutChar(CR);
    }
}
```

C language Style Guidelines: Comments

- The beginning of every file (Show UART.c)
- The beginning of every function (Show UART.c)

- line delimiting the start of the function
- purpose
- input parameters
- output parameters, and
- special conditions that apply.
- explain the policies
- intended to be read by the client.

- variable or constant definition

- clarify the usage
- specify the units
- include examples.

```
short V1; // voltage at node 1 in mV,  
// range -5000 mV to +5000 mV  
unsigned short Fs; // sampling rate in Hz
```

```
char FoundFlag; // keyword found yet?  
// 0 if keyword not yet found, 1 if found
```

```
unsigned char RunMode; // system mode  
// 0 means idle  
// 1 means startup  
// 2 means active run  
// 3 means stopped
```

C language Style Guidelines: Comments

- describe complex algorithms
 - intended to be read by our coworkers
 - assist in changing the code in the future
- Examples of bad comments include:
 `time++; // add one to time`
 `mode = 0; // set mode to zero`
- Good comments explain why the operation is performed, and what it means:
 `time++; // elapsed time in msec`
 `mode = 0; // idle mode, no data is avail`

Program Test

- Divide the testing of the design into small, autonomous units
 - It is easier to detect faults before they are compounded by other factors
- Program testing requires getting diagnostic data out of the target, for analysis
 - By emulation tools, or through the hardware itself (for instance a serial port)

System/SW Design

- System Design
- SW/Program Design
- **Finite State Machine (FSM)**
- Software Architecture
- Interfacing
and I/O Synchronization
- Debugging Techniques

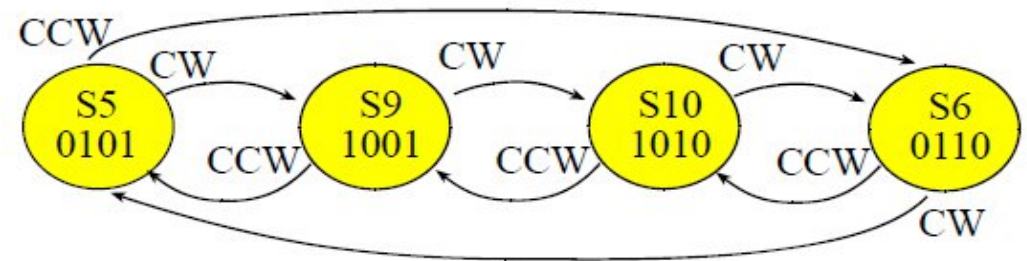


Finite State Machine (FSM)

- What is Finite State Machine (FSM)?
 - inputs, outputs, states, and state transitions
 - state graph defines relationships of inputs and outputs
- FSM is a good example of abstraction. The abstract principles of FSM development :
 - inputs,
 - outputs,
 - states, and
 - state transitions.
- The advantages of this abstraction are
 - faster to develop
 - easier to debug (prove correct) and
 - easier to change
- If we can take a complex problem and map it into a FSM model, then we can solve it with a simple FSM software tools.

Finite State Machine

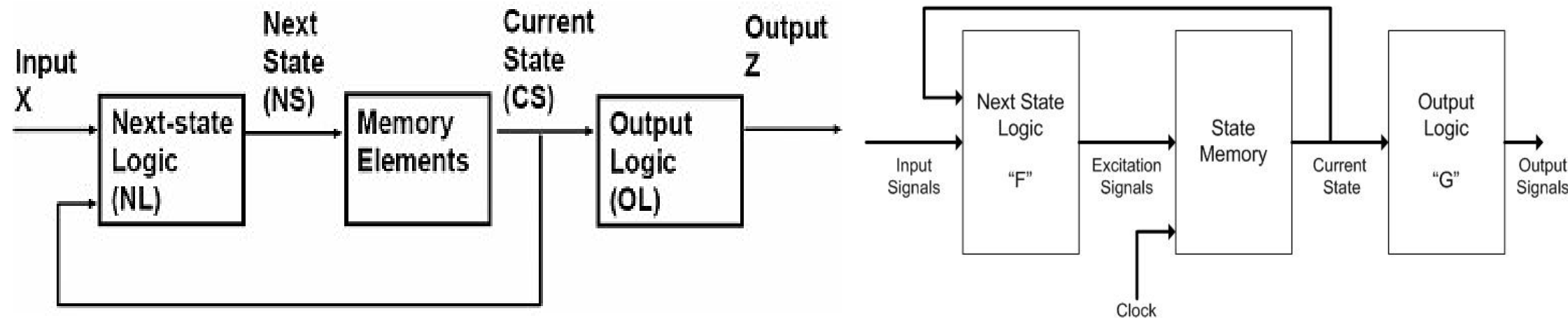
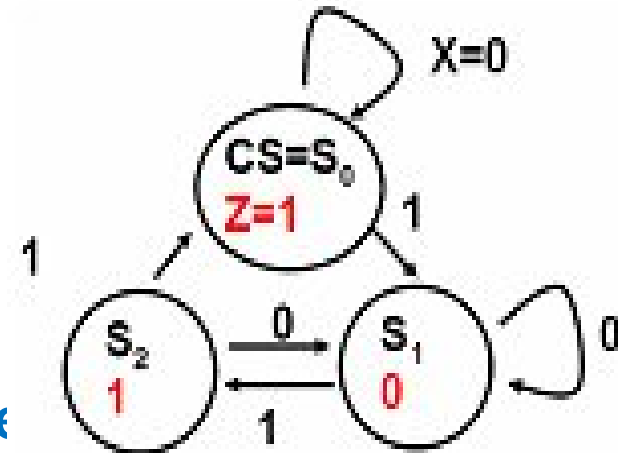
- What are the parts of a finite state machine?
 - Input sensors
 - Output actuators
 - Controller
 - Software that inputs, outputs, changes state
 - Accesses the state graph
 - State graph
 - Graphical interconnection between states
- What is a state (system not changing)?
 - Description of current conditions
 - Actions required to keep the system from changing



Moore FSM

- **Moore FSM**

- output value depends only on the current state,
- inputs affect the state transitions
- significance is being in a state
- Next state depends on input and current state
- **Use Moore if the output means being in that state**
 - Need the output to be in that state, e.g., traffic light.
- Input: when to change state
- Output: definition of being in that state



Moore FSM

- Moore FSM Execution Sequence
 1. Perform output, which depends on the current state
 2. Wait a prescribed amount of time (optional)
 3. Input
 4. Go to next state, which depends on the input and the current state

Moore FSM Code Template

```
//This is HW_Moore.c
```

```
// your global variables go here
```

```
//use these definitions (Port V is not a real Port)
```

```
#define GPIO_PORTV_DATA_R (*((volatile unsigned long *) 0x40007404))
```

```
#define GPIO_PORTV_DIR_R (*((volatile unsigned long *) 0x40007400))
```

```
#define GPIO_PORTV_AFSEL_R (*((volatile unsigned long *) 0x40007420))
```

```
#define GPIO_PORTV_DEN_R (*((volatile unsigned long *) 0x4000751C))
```

```
#define SYSCTL_RCGC2_R (*((volatile unsigned long *) 0x400FE108))
```

```
#define SYSCTL_RCGC2_GPIOV 0x00000000 // port V Clock Gating Control
```

Moore FSM Code Template: FSMInit

```
// ***** YourFSMInit *****  
/* Part a) put your initialization here. Add code to this function so it  
   initializes the direction register of Port V, be friendly */  
// PV6,PV5,PV4 are outputs  
// PV3,PV1 are inputs  
// initialize global variables (state pointer)  
// initial state is Stop  
// Error conditions: none  
void YourFSMInit(void){  
    SYSCTL_RCGC2_R |= SYSCTL_RCGC2_GPIOV; // clock to Port V  
    GPIO_PORTV_AFSEL_R &= ~0x7A; // Alternative function register  
    GPIO_PORTV_DEN_R |= 0x7A;    // data enable register  
// add your code here  
  
}
```

Moore FSM Code Template: FSMOutput

```
//*****YourFSMOutput*****
```

```
// Part b) 3-bit output to Port V, be friendly
```

```
// Input: call by value (0 to 7)
```

```
// Do not change the values of PV7,3,2,1,0
```

```
// There are 8 possibilities
```

```
// Input | PV6 PV5 PV4
```

```
// 0 | 0 0 0
```

```
// 1 | 0 0 1
```

```
// 2 | 0 1 0
```

```
// 3 | 0 1 1
```

```
// 4 | 1 0 0
```

```
// 5 | 1 0 1
```

```
// 6 | 1 1 0
```

```
// 7 | 1 1 1
```

```
// Error conditions: none
```

```
void YourFSMOutput(unsigned long data){
```

```
// put your code here
```

```
}
```

Moore FSM Code Template: FSMInput

```
/*******YourFSMInput*****
```

```
// Part c) 2-bit input from Port V bits PV3 PV1
```

```
// Input: none
```

```
// Output: return by value (0 to 3)
```

```
// PV3 PV1 | Return value
```

```
// 0 0 | 0
```

```
// 0 1 | 1
```

```
// 1 0 | 2
```

```
// 1 1 | 3
```

```
//Error conditions: none
```

```
unsigned long YourFSMInput(void){
```

```
// put your code here
```

```
    return 123; // remove/replace this line
```

```
}
```

Moore FSM Code Template: FSMController

```
void MyGrader(void);
```

```
void YourFSMController(void){
```

```
YourFSMInit(); // your initialization
```

```
while(1){
```

```
    MyGrader(); //do not move or remove this line
```

```
    // Part d) put your output-input-next engine here
```

```
    //the Grader will check outputs and make inputs happen
```

```
    //1. output to PTV bits 6,4,3 (be friendly)
```

```
    //2. input from PTV bits 3,1
```

```
    //3. next
```

```
// put your FSM controller here
```

```
}
```

```
}
```

Moore FSM Example

```

struct State {
    unsigned long Out;
    unsigned long Time; // ms
    unsigned char Next[4];};
typedef const struct State STyp;
#define goN    0
#define waitN  1
#define goE    2
#define waitE  3
STyp FSM[4]={
{0x21,30000,{goN,waitN,goN,waitN}},
{0x22, 5000,{goE,goE,goE,goE}},
{0x0C,30000,{goE,goE,waitE,waitE}},
{0x14, 5000,{goN,goN,goN,goN}}};
int main(void){ unsigned char n;
    unsigned long input;
    // activate port D and port B
    SysTick_Init();
    // make PB1-0 in
    // enable digital I/O on PB1-0
    // make PD5-0 out
    // enable digital I/O on PD5-0
    n = goN;
    while(1){
        GPIO_PORTD_DATA_R = FSM[n].Out;
        SysTick_Wait1ms(FSM[n].Time);
        input = GPIO_PORTB_DATA_R&0x03;
        n = FSM[n].Next[input];
    }
}

```

Program 3.1

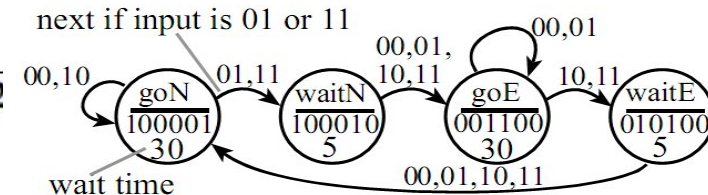
separate **how we are getting it done** from **what we are doing**
 separate **mechanisms** from **policies**

```

struct State {
    unsigned long Out;
    unsigned long Time; //ms
    const struct State *Next[4];};
typedef const struct State STyp;
#define goN    &FSM[0]
#define waitN  &FSM[1]
#define goE    &FSM[2]
#define waitE  &FSM[3]
STyp FSM[4]={
{0x21,30000,{goN,waitN,goN,waitN}},
{0x22, 5000,{goE,goE,goE,goE}},
{0x0C,30000,{goE,goE,waitE,waitE}},
{0x14, 5000,{goN,goN,goN,goN}}};
int main(void){ STyp *Pt;
    unsigned long input;
    // activate port D and port B
    SysTick_Init();
    // make PB1-0 in
    // enable digital I/O on PB1-0
    // make PD5-0 out
    // enable digital I/O on PD5-0
    Pt = goN;
    while(1){
        GPIO_PORTD_DATA_R = Pt->Out;
        SysTick_Wait1ms(Pt->Time);
        Input = GPIO_PORTB_DATA_R&0x03;
        Pt = Pt->Next[Input];
    }
}

```

Program 3.2



Moore FSM Example

Run FSM in background using Output compare (ms time)

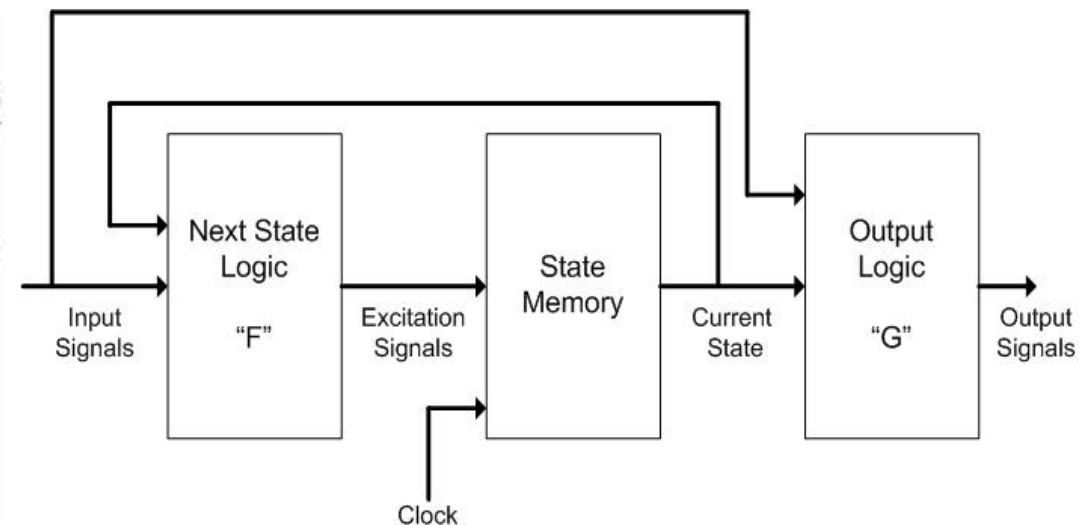
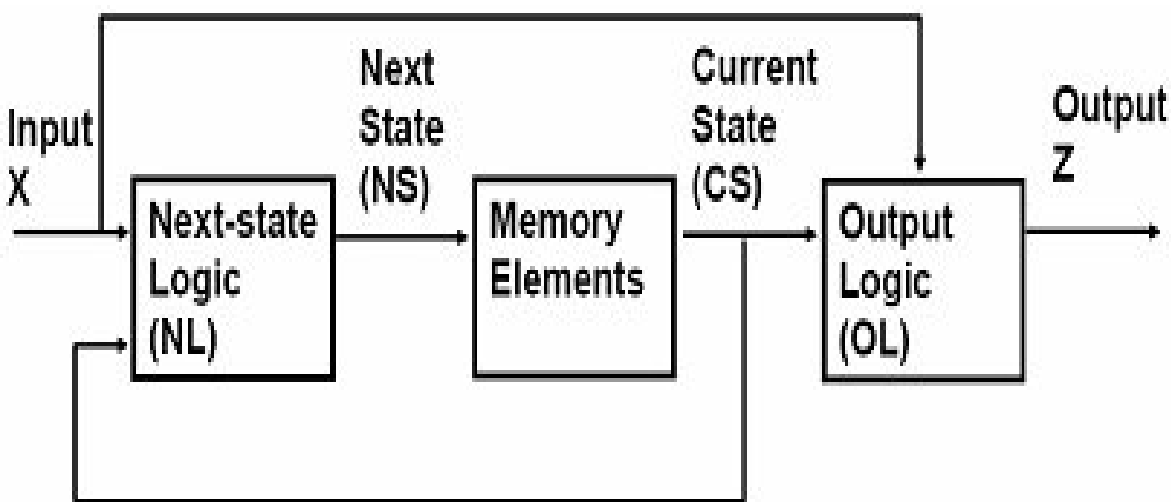
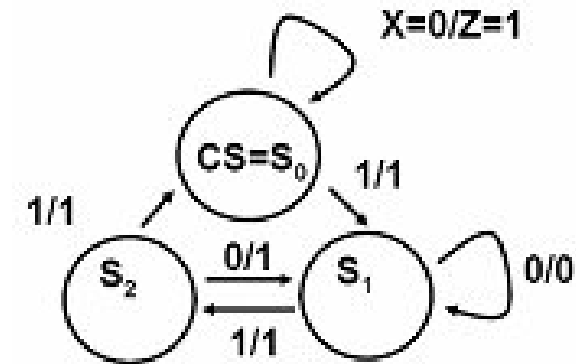
```
STyp *Pt;
int FSM_Init(void){
// activate port D and port B
// make PB1-0 in
// enable digital I/O on PB1-0
// make PD5-0 out
// enable digital I/O on PD5-0
    SYSCCTL_RCGC1_R |= SYSCCTL_RCGC1_TIMER0; // 0) activate
    Pt = goN;
    TIMER0_CTL_R &= ~0x00000001;           // 1) disable timer0A
    TIMER0_CFG_R = 0x00000004;             // 2) configure 16-bit
    TIMER0_TAMR_R = 0x00000002;            // 3) periodic mode
    TIMER0_TAILR_R = 49999;                // 4) reload value
    TIMER0_ICR_R = 0x00000001;             // 5) clear timeout flag
    TIMER0_IMR_R |= 0x00000001;           // 6) arm timeout
    NVIC_PRI4_R = (NVIC_PRI4_R&0x00FFFFFF)|0x40000000;
    NVIC_EN0_R |= NVIC_EN0_INT19;         // 8) enable int 19
    TIMER0_CTL_R |= 0x00000001;           // 9) enable timer0A
    EnableInterrupts(); // interrupts every 1ms
}
void Timer0A_Handler(void){ unsigned char Input;
    TIMER0_ICR_R = TIMER_ICR_TATOCINT; // acknowledge timer0A
    Count--;
    if(Count==0){
        Input = GPIO_PORTB_DATA_R&0x03; // 3) Input
        Pt = Pt->Next[Input];           // 4) Next
        GPIO_PORTD_DATA_R = Pt->Out;     // 1) output
        Count = Pt->Time;                 // 2) Wait
    }
}
```


Mealy FSM

- **Mealy FSM** depend both on the current state and the inputs.

- output value depends on input and current state
- Next state depends on input and current state
- inputs affect the state transitions.
- significance is the state transition
- **use Mealy if the output causes the state to change**
- do not need a specific output to be in that state
- E.g., four-legged robot

- Input: when to change state
- Output: how to change state

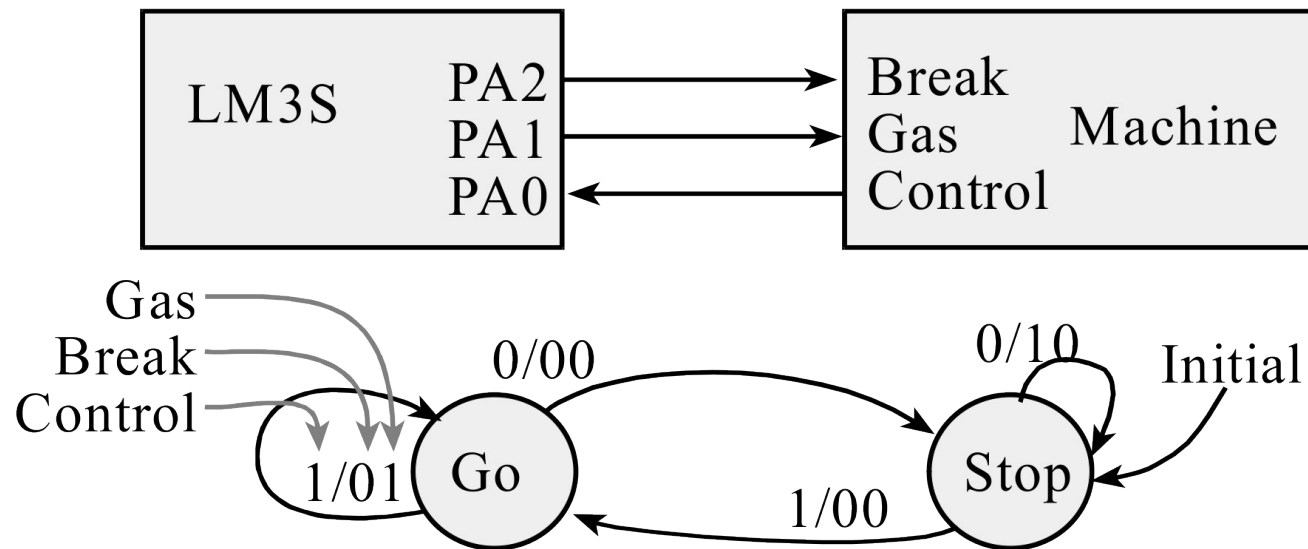


Mealy FSM

- Mealy FSM Execution Sequence
 1. Wait a prescribed amount of time (optional)
 2. Input
 3. Perform output, which depends on the input and the current state
 4. Go to next state, which depends on the input and the current state

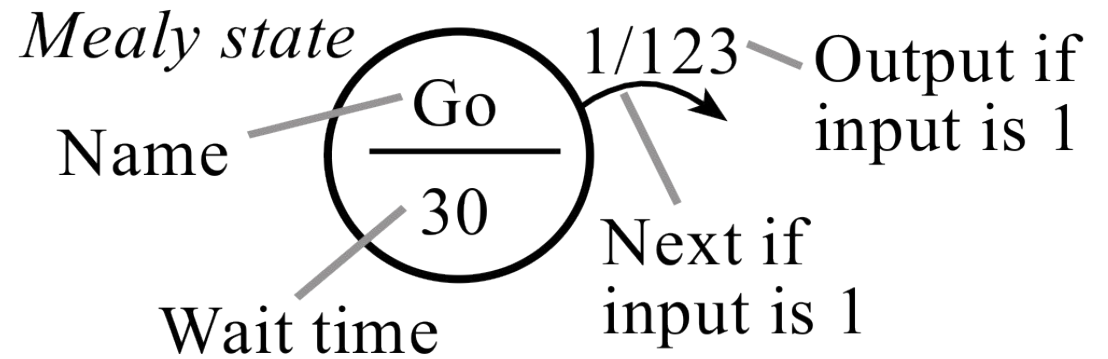
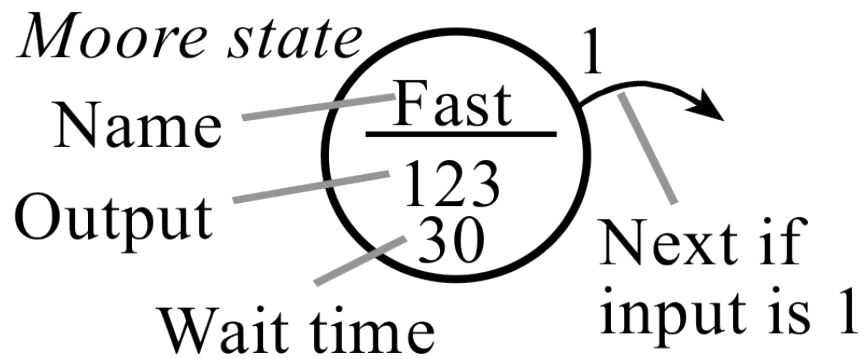
Mealy FSM Example

Inputs: Control



Outputs: Brake, Gas

Finite State Machine (FSM)



Finite State Machine (FSM)

- **Data Structure** embodies the FSM
 - multiple identically-structured nodes
 - statically-allocated fixed-size linked structures
 - one-to-one mapping FSM state graph and linked structure
 - one structure for each state
- **Linked Structure**
 - pointer (or link) to other nodes (define next states)
- **Table structure**
 - indices to other nodes (define next states)

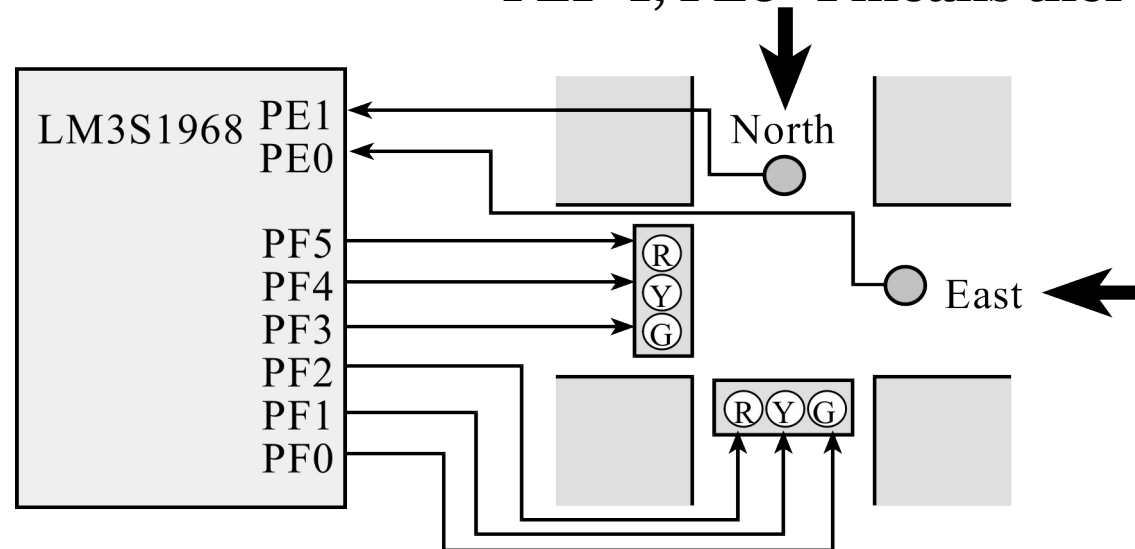
Traffic Light Control

PE1=0, PE0=0 means no cars exist on either road

PE1=0, PE0=1 means there are cars on the East road

PE1=1, PE0=0 means there are cars on the North road

PE1=1, PE0=1 means there are cars on both roads



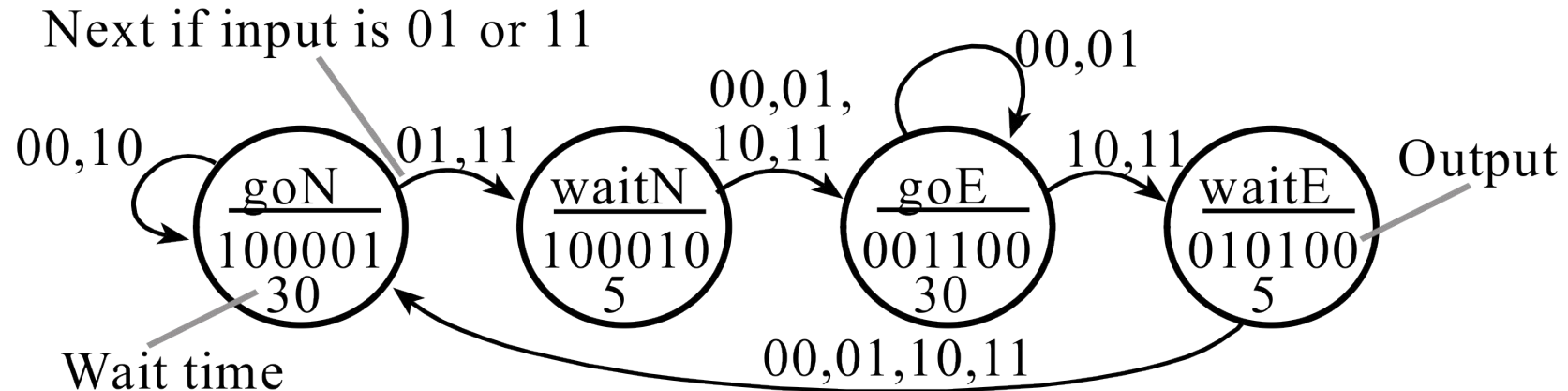
goN, PF5-0 = 100001 makes it green on North and red on East

waitN, PF5-0 = 100010 makes it yellow on North and red on East

goE, PF5-0 = 001100 makes it red on North and green on East

waitE, PF5-0 = 010100 makes it red on North and yellow on East

Traffic Light Control



State \ Input	00	01	10	11
goN (100001,30)	goN	waitN	goN	waitN
waitN (100010,5)	goE	goE	goE	goE
goE (001100,30)	goE	goE	waitE	waitE
waitE (010100,5)	goN	goN	goN	goN

Linked Data Structure

```
OUT    EQU 0      ;offset for output
WAIT   EQU 4      ;offset for time (10ms)
NEXT   EQU 8      ;offset for next
goN    DCD 0x21    ;North green, East red
        DCD 3000   ;30 sec
        DCD goN,waitN,goN,waitN
waitN  DCD 0x22    ;North yellow, East red
        DCD 500    ;5 sec
        DCD goE,goE,goE,goE
goE    DCD 0x0C    ;North red, East green
        DCD 3000   ;30 sec
        DCD goE,goE,waitE,waitE
waitE  DCD 0x14    ;North red, East yellow
        DCD 500    ;5 sec
        DCD goN,goN,goN,goN
```

In ROM

Linked Data Structure

```
const struct State {
    unsigned long Out;
    unsigned long Time;    // 10 ms units
    const struct State *Next[4];};
typedef const struct State STyp;
#define goN    &FSM[0]
#define waitN  &FSM[1]
#define goE    &FSM[2]
#define waitE  &FSM[3]
STyp FSM[4]={
    {0x21,3000,{goN,waitN,goN,waitN}},
    {0x22, 500,{goE,goE,goE,goE}},
    {0x0C,3000,{goE,goE,waitE,waitE}},
    {0x14, 500,{goN,goN,goN,goN}}};
```

PointerTrafficLight

```
// PointerTrafficLight.c
// Runs on LM3S811. Use a pointer implementation //of a Moore finite state
// machine to operate a traffic light.
// north facing car detector connected to PB1 (1=car present)
// east facing car detector connected to PB0 (1=car present)
// east facing red light connected to PD5
// east facing yellow light connected to PD4
// east facing green light connected to PD3
// north facing red light connected to PD2
// north facing yellow light connected to PD1
// north facing green light connected to PD0
#include "SysTick.h"
#define GPIO_PORTB_DATA_R      (*((volatile unsigned long *)0x400053FC))
#define GPIO_PORTB_DIR_R      (*((volatile unsigned long *)0x40005400))
#define GPIO_PORTB_DEN_R      (*((volatile unsigned long *)0x4000551C))
```

PointerTrafficLight

```
#define GPIO_PORTD_DATA_R    (*((volatile unsigned long *)0x400073FC))
#define GPIO_PORTD_DIR_R    (*((volatile unsigned long *)0x40007400))
#define GPIO_PORTD_DEN_R    (*((volatile unsigned long *)0x4000751C))
#define SYSCTL_RCGC2_R      (*((volatile unsigned long *)0x400FE108))
#define SYSCTL_RCGC2_GPIOD  0x00000008 // port D Clock Gating Control
#define SYSCTL_RCGC2_GPIOB  0x00000002 // port B Clock Gating Control
struct State {
    unsigned long Out;        // 6-bit output
    unsigned long Time;       // 10 ms
    const struct State *Next[4]; // depends on 2-bit input
typedef const struct State STyp;
#define goN  &FSM[0]
#define waitN &FSM[1]
#define goE  &FSM[2]
#define waitE &FSM[3]

STyp FSM[4]={ {0x21,3000,{goN,waitN,goN,waitN}}, {0x22, 500,{goE,goE,goE,goE}},
              {0x0C,3000,{goE,goE,waitE,waitE}}, {0x14, 500,{goN,goN,goN,goN}}};
```

PointerTrafficLight

```
int main(void){
    STyp *Pt; // state pointer
    unsigned long Input;
    SYSCTL_RCGC2_R |= SYSCTL_RCGC2_GPIOD+SYSCTL_RCGC2_GPIOB; // activate port D & B
    SysTick_Init();      // initialize SysTick timer
    GPIO_PORTB_DIR_R &= ~0x03; // make PB1-0 in
    GPIO_PORTB_DEN_R |= 0x03; // enable digital I/O on PB1-0
    GPIO_PORTD_DIR_R |= 0x3F; // make PD5-0 out
    GPIO_PORTD_DEN_R |= 0x3F; // enable digital I/O on PD5-0
    Pt = goN;
    while(1){
        GPIO_PORTD_DATA_R = Pt->Out;
        SysTick_Wait10ms(Pt->Time);
        Input = GPIO_PORTB_DATA_R&0x03;
        Pt = Pt->Next[Input];
    }
}
```

PortableTrafficLight

```
// PortableTrafficLight.c. Runs on LM3S811
/* Use a table implementation of a Moore finite state machine to operate a traffic light. Only the "#define"
   section is processor specific, and the rest of the code can easily be adapted to another system.*/
// north facing car detector connected to BSP_InPort1 (1=car present)
// east facing car detector connected to BSP_InPort0 (1=car present)
// east facing red light connected to BSP_OutPort5
// east facing yellow light connected to BSP_OutPort4
// east facing green light connected to BSP_OutPort3
// north facing red light connected to BSP_OutPort2
// north facing yellow light connected to BSP_OutPort1
// north facing green light connected to BSP_OutPort0
#include "SysTick.h"
#define BSP_InPort      (*((volatile unsigned long *)0x400053FC))
#define BSP_InPort_DDR  (*((volatile unsigned long *)0x40005400))
#define BSP_InPort_DEN  (*((volatile unsigned long *)0x4000551C))
#define BSP_OutPort     (*((volatile unsigned long *)0x400073FC))
#define BSP_OutPort_DDR (*((volatile unsigned long *)0x40007400))
#define BSP_OutPort_DEN (*((volatile unsigned long *)0x4000751C))
#define BSP_GPIO_EN     (*((volatile unsigned long *)0x400FE108))
#define BSP_InPort_Mask  0x00000008 // bit mask for InPort
#define BSP_OutPort_Mask 0x00000002 // bit mask for OutPort
```

PortableTrafficLight

```
struct State {
    unsigned long Out;
    unsigned long Time;
    const struct State *Next[4];};
typedef const struct State STyp;
#define goN  &FSM[0]
#define waitN &FSM[1]
#define goE  &FSM[2]
#define waitE &FSM[3]
STyp FSM[4]={
    {0x21,3000,{goN,waitN,goN,waitN}},
    {0x22, 500,{goE,goE,goE,goE}},
    {0x0C,3000,{goE,goE,waitE,waitE}},
    {0x14, 500,{goN,goN,goN,goN}}};
```

PortableTrafficLight

```
int main(void){
    STyp *Pt; // state pointer
    unsigned long Input;
    BSP_GPIO_EN |= BSP_InPort_Mask|BSP_OutPort_Mask; // activate I/O ports
    SysTick_Init(); // initialize SysTick timer
    BSP_InPort_DDR &= ~0x03; // make BSP_InPort1-0 in
    BSP_InPort_DEN |= 0x03; // enable digital I/O on BSP_InPort1-0
    BSP_OutPort_DDR |= 0x3F; // make BSP_OutPort5-0 out
    BSP_OutPort_DEN |= 0x3F; // enable digital I/O on BSP_OutPort5-0
    Pt = goN;
    while(1){
        BSP_OutPort = Pt->Out;
        SysTick_Wait10ms(Pt->Time);
        Input = BSP_InPort&0x03;
        Pt = Pt->Next[Input];
    }
}
```

TableTrafficLight

```
/* TableTrafficLight.c. Runs on LM3S811. Use a table implementation of a Moore finite state machine to
   operate a traffic light.*/
// north facing car detector connected to PB1 (1=car present)
// east facing car detector connected to PB0 (1=car present)
// east facing red light connected to PD5
// east facing yellow light connected to PD4
// east facing green light connected to PD3
// north facing red light connected to PD2
// north facing yellow light connected to PD1
// north facing green light connected to PD0
#include "SysTick.h"
#define GPIO_PORTB_DATA_R    (*((volatile unsigned long *)0x400053FC))
#define GPIO_PORTB_DIR_R    (*((volatile unsigned long *)0x40005400))
#define GPIO_PORTB_DEN_R    (*((volatile unsigned long *)0x4000551C))
#define GPIO_PORTD_DATA_R    (*((volatile unsigned long *)0x400073FC))
#define GPIO_PORTD_DIR_R    (*((volatile unsigned long *)0x40007400))
#define GPIO_PORTD_DEN_R    (*((volatile unsigned long *)0x4000751C))
#define SYSCTL_RCGC2_R      (*((volatile unsigned long *)0x400FE108))
#define SYSCTL_RCGC2_GPIOD  0x00000008 // port D Clock Gating Control
#define SYSCTL_RCGC2_GPIOB  0x00000002 // port B Clock Gating Control
```


TableTrafficLight

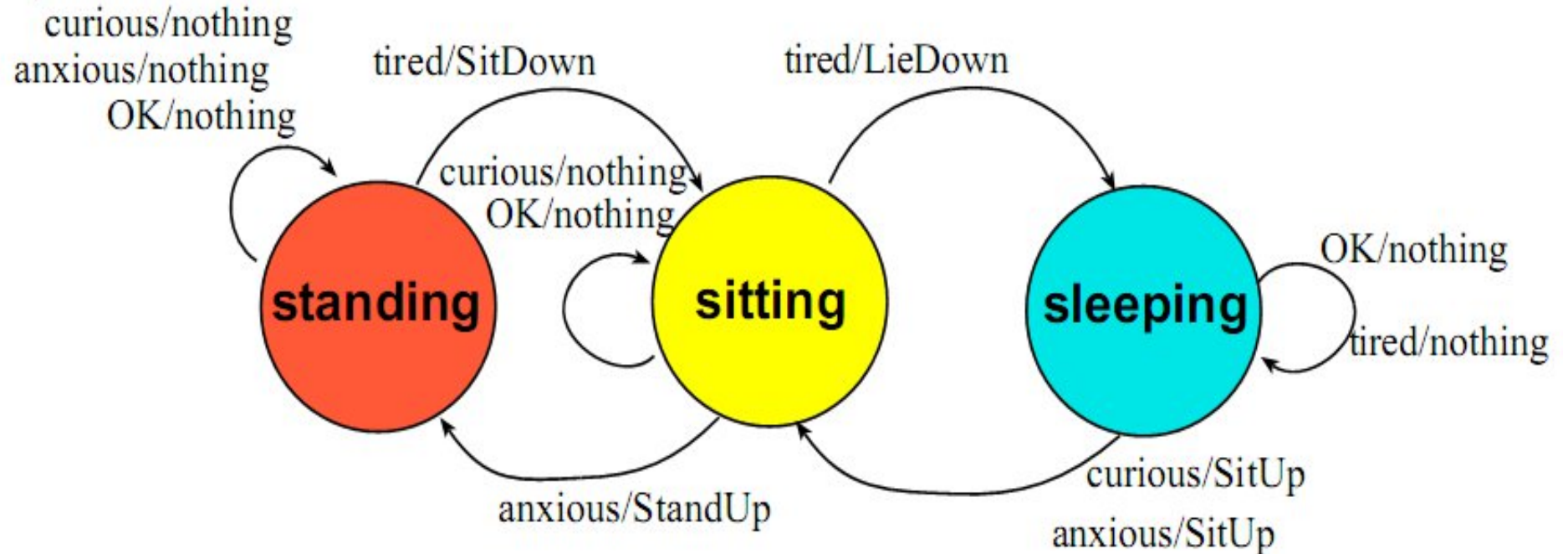
```
struct State {  
    unsigned long Out;    // 6-bit output  
    unsigned long Time;   // 10 ms  
    unsigned char Next[4]; // depends on 2-bit input  
};  
typedef const struct State STyp;  
#define goN 0  
#define waitN 1  
#define goE 2  
#define waitE 3  
STyp FSM[4]={  
    {0x21,3000,{goN,waitN,goN,waitN}},  
    {0x22, 500,{goE,goE,goE,goE}},  
    {0x0C,3000,{goE,goE,waitE,waitE}},  
    {0x14, 500,{goN,goN,goN,goN}}};
```

TableTrafficLight

```
int main(void){
    unsigned char n; // state number
    unsigned long Input;
    // activate port D and port B
    SYSCTL_RCGC2_R |= SYSCTL_RCGC2_GPIOD+SYSCTL_RCGC2_GPIOB;
    SysTick_Init();      // initialize SysTick timer
    GPIO_PORTB_DIR_R &= ~0x03; // make PB1-0 in
    GPIO_PORTB_DEN_R |= 0x03; // enable digital I/O on PB1-0
    GPIO_PORTD_DIR_R |= 0x3F; // make PD5-0 out
    GPIO_PORTD_DEN_R |= 0x3F; // enable digital I/O on PD5-0
    n = goN;
    while(1){
        GPIO_PORTD_DATA_R = FSM[n].Out;
        SysTick_Wait10ms(FSM[n].Time);
        Input = GPIO_PORTB_DATA_R&0x03;
        n = FSM[n].Next[Input];
    }
}
```

Mealy FSM Example

Mealy Finite State Machine

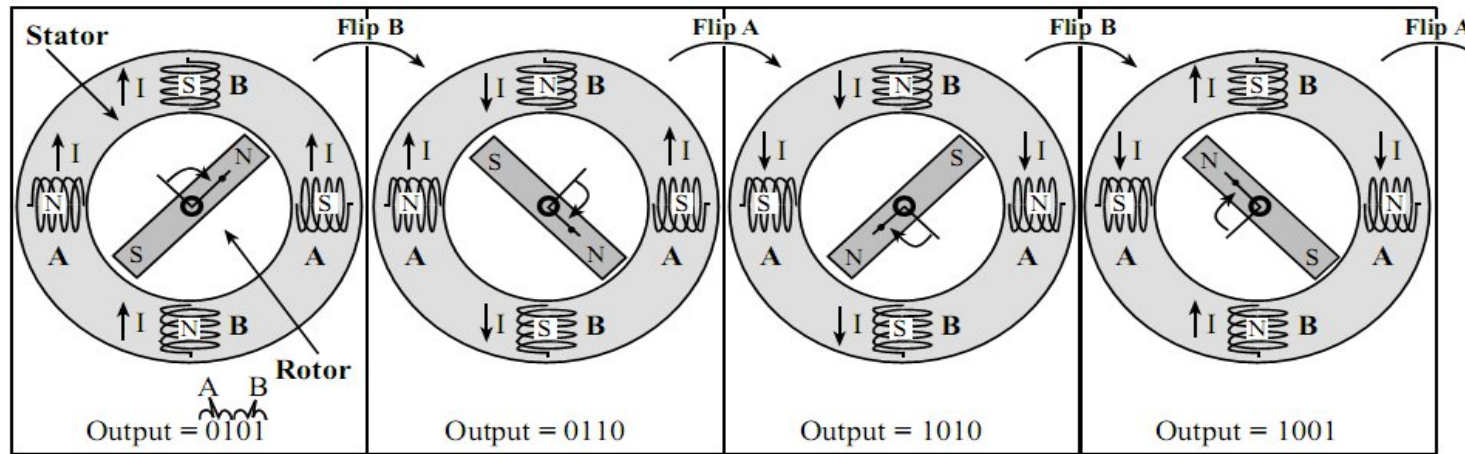


Place a function call for output

```
const struct State{  
    void (*CmdPt)(void);    // function to execute  
    const struct State *Next[4]; // Next if 0,1  
};  
  
(*Pt->CmdPt)(); // 2) execute function
```

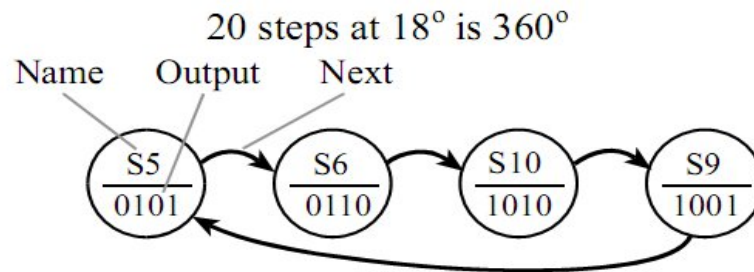
FSM Example: Stepper Motor Control

Initial stable state



The full-step sequence to rotate a stepper motor.

To rotate this stepper by 18°, the interface flips the direction of one of the currents.



full step 0101,

0110,

1010,

1001,...

half-step

0101, 0100, 0110, 0010, 1010, 1000, 1001, 0001, ...

Linked List Data

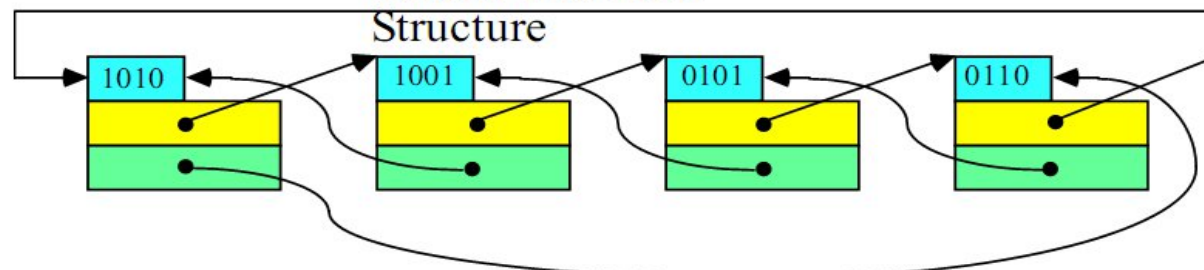


Figure A double circular linked-list used to control the stepper motor.

A double circular linked list used to control the stepper motor.

FSM Example: Stepper Motor Control

```

const struct State{
    unsigned char Out;  // Output for this state
    const struct State *Next[2]; // CW or CCW
};

typedef const struct State StateType;
typedef StateType * StatePtr;
unsigned short POS;
/* between 0 and 199 representing shaft angle */
#define clockwise 0      /* Next index*/
#define counterclockwise 1 /* Next index*/

full step    0101,    0110,    1010,    1001,...
half step    0101,0100,0110,0010,1010,1000,1001,0001,...

StateType fsm[4]={
    { 5,{&fsm[1],&fsm[3]}}},
    { 6,{&fsm[2],&fsm[0]}}},
    {10,{&fsm[3],&fsm[1]}}},
    { 9,{&fsm[0],&fsm[2]}}
};

StatePtr Pt; /* Current State */
#define STEPPER (*(volatile unsigned long *)0x4000703C)

void CW(void){
    Pt = Pt->Next[clockwise]; // circulates
    STEPPER = Pt->Out;         // step motor
    POS++;
    if(POS==200){
        POS=0; // shaft angle
    }
}

void CCW(void){
    Pt = Pt->Next[counterclockwise]; // circulates
    STEPPER = Pt->Out;               // step motor
    if(POS==0){
        POS = 199;
    }
    else{
        POS--;
    }
    // shaft angle
}

half-step    0101,0100,0110,0010,1010,1000,1001,0001,...
StateType fsm[8]={
    { 5,{&fsm[1],&fsm[7]}}},
    { 4,{&fsm[2],&fsm[0]}}},
    { 6,{&fsm[3],&fsm[1]}}},
    { 2,{&fsm[4],&fsm[2]}}},
    {10,{&fsm[5],&fsm[3]}}},
    { 8,{&fsm[6],&fsm[4]}}},
    { 9,{&fsm[7],&fsm[5]}}},
    { 1,{&fsm[0],&fsm[6]}}
};

StatePtr Pt; /* Current State */
void CW(void){
    Pt = Pt->Next[clockwise]; // circulates
    STEPPER = Pt->Out;         // step motor
    POS++;
    if(POS==400){
        POS=0; // shaft angle
    }
}

void CCW(void){
    Pt = Pt->Next[counterclockwise]; // circulates
    STEPPER = Pt->Out;               // step motor
    if(POS==0){
        POS = 399;
    }
    else{
        POS--;
    }
    // shaft angle
}

```

FSM: Stepper Machine Control

```
/* StepperTestMain.c. Runs on LM3S811. Test the functions provided by Stepper.c. */  
// PD3 connected to driver for stepper motor coil A  
// PD2 connected to driver for stepper motor coil A'  
// PD1 connected to driver for stepper motor coil B  
// PD0 connected to driver for stepper motor coil B'  
#include "stepper.h"  
#define T1ms 6000  
int main(void){  
    Stepper_Init();  
    Stepper_CW(T1ms); // Pos=1; GPIO_PORTD_DATA_R=9  
    Stepper_CW(T1ms); // Pos=2; GPIO_PORTD_DATA_R=5  
    Stepper_CW(T1ms); // Pos=3; GPIO_PORTD_DATA_R=6  
    Stepper_CW(T1ms); // Pos=4; GPIO_PORTD_DATA_R=10  
    Stepper_CW(T1ms); // Pos=5; GPIO_PORTD_DATA_R=9  
    Stepper_CW(T1ms); // Pos=6; GPIO_PORTD_DATA_R=5  
    Stepper_CW(T1ms); // Pos=7; GPIO_PORTD_DATA_R=6  
    Stepper_CW(T1ms); // Pos=8; GPIO_PORTD_DATA_R=10  
    Stepper_CW(T1ms); // Pos=9; GPIO_PORTD_DATA_R=9
```

FSM: Stepper Machine Control

```
Stepper_CCW(T1ms); // Pos=8; GPIO_PORTD_DATA_R=10
Stepper_CCW(T1ms); // Pos=7; GPIO_PORTD_DATA_R=6
Stepper_CCW(T1ms); // Pos=6; GPIO_PORTD_DATA_R=5
Stepper_CCW(T1ms); // Pos=5; GPIO_PORTD_DATA_R=9
Stepper_CCW(T1ms); // Pos=4; GPIO_PORTD_DATA_R=10
Stepper_Seek(8,T1ms); // Pos=8; GPIO_PORTD_DATA_R=10
Stepper_Seek(0,T1ms); // Pos=0; GPIO_PORTD_DATA_R=10
while(1){
    Stepper_CW(10*T1ms); // output every 10ms
}
```

=====

```
/* Stepper.c. Runs on LM3S811. Provide functions that step the motor once clockwise, step once
   counterclockwise, and initialize the stepper motor interface.*/
// PD3 connected to driver for stepper motor coil A
// PD2 connected to driver for stepper motor coil A'
// PD1 connected to driver for stepper motor coil B
// PD0 connected to driver for stepper motor coil B'
```

FSM: Stepper Machine Control

```
#include "lm3s811.h"
#include "systick.h"
struct State{
    unsigned char Out;        // Output
    const struct State *Next[2]; // CW/CCW
};
typedef const struct State StateType;
typedef StateType *StatePtr;
#define clockwise 0          // Next index
#define counterclockwise 1 // Next index
StateType fsm[4]={
    {10,{&fsm[1],&fsm[3]}},
    { 9,{&fsm[2],&fsm[0]}},
    { 5,{&fsm[3],&fsm[1]}},
    { 6,{&fsm[0],&fsm[2]}}
};
```


FSM: Stepper Machine Control

```
unsigned char Pos;    // between 0 and 199
const struct State *Pt;// Current State
#define STEPPER (*(volatile unsigned long *)0x4000703C)
// Move 1.8 degrees clockwise, delay is the time to wait after each step
void Stepper_CW(unsigned long delay){
    Pt = Pt->Next[clockwise];    // circular
    STEPPER = Pt->Out; // step motor
    if(Pos==199){    // shaft angle
        Pos = 0;    // reset
    }
    else{
        Pos++; // CW
    }
    SysTick_Wait(delay);
}
```

FSM: Stepper Machine Control

```
// Move 1.8 degrees counterclockwise, delay is wait after each step
void Stepper_CCW(unsigned long delay){
    Pt = Pt->Next[counterclockwise]; // circular
    STEPPER = Pt->Out; // step motor
    if(Pos==0){ // shaft angle
        Pos = 199; // reset
    }
    else{
        Pos--; // CCW
    }
    SysTick_Wait(delay);} // blind-cycle wait
// Initialize Stepper interface
```

FSM: Stepper Machine Control

```
void Stepper_Init(void){  
    SYSCTL_RCGC2_R |= SYSCTL_RCGC2_GPIOD; //activate port D  
    SysTick_Init();  
    Pos = 0;  
    Pt = &fsm[0];  
    GPIO_PORTD_DIR_R |= 0x0F; // make PD3-0 out  
    GPIO_PORTD_DEN_R |= 0x0F; // enable digital I/O on PD3-0  
    GPIO_PORTD_DR8R_R |= 0x0F;} // enable 8 mA drive
```

FSM: Stepper Machine Control

```
// Turn stepper motor to desired position (0 <= desired <= 199)
// time is the number of bus cycles to wait after each step
void Stepper_Seek(unsigned char desired, unsigned long time){
short CWsteps;
  if((CWsteps = (desired-Pos))<0){
    CWsteps+=200;
  } // CW steps is 0 to 199
  if(CWsteps > 100){
    while(desired != Pos){
      Stepper_CCW(time);
    }
  }
  else{
    while(desired != Pos){
      Stepper_CW(time);
    }
  }
}
```

Another FSM Example: Stepper Motor Control

- We will control a stepper motor using a FSM.
- The FSM must be implemented as a linked data structure. Switches will allow the operator to control the motor.
 - Perform state machine input/output in the background.
- A background periodic interrupt using Periodic timer will perform inputs from the switches and outputs to the stepper motor coils. The periodic timer module must be written at a low level.
- There will be at least three hardware/software modules:
 - Switch, input, motor output, and FSM

Another FSM Example: Stepper Motor Control

- System Description:
 - If all buttons are released, then the motor should stop.
 - If switch 1 is pressed, spin slowly in one direction as long as switch 1 continues to be pressed (note to students: you should extend this sentence to specify the speed).
 - If switch 2 is pressed, spin slowly in the other direction as long as switch 2 continues to be pressed (note to students: you should extend this sentence to specify the speed).
 - If switch 1 and 2 are both pressed, the motor should continuously oscillate as fast as possible back and forth. The amplitude of this oscillation will be 8 steps. In other words, output the sequence {CW, CW, CW, CW, CW, CW, CW, CW, CCW, CCW, CCW, CCW, CCW, CCW} over and over as long as both switch 1 and 2 continue to be pressed (note to students: you should extend this sentence to specify the oscillation rate).
 - If switch 3 is pressed then released, then step once in one direction (note to students: you should edit this sentence to define which direction it steps).
 - Any other combination (e.g., 1+3, 2+3, 1+2+3) should be ignored. You may assume the operator will release the switches when it reaches the end of the worm gear. (Note to students: if you use the internal switches you could rename the switches Up Down Left Right or Select to match the switches you use)

Another FSM Example: Stepper Motor Control

- There will be a 1-1 mapping from the FSM graph and the C data structure. There is a delay for each state, which is saved in the FSM structure.
- The motor output is always one of these values 5, 6, 10, or 9. The motor moves such that the output never skips from 5 to 10, 10 to 5, 6 to 9, or 9 to 6.
- There are about 20 to 30 states (note: you can place your FSM graphs here).
- The FSM runs in the background using periodic interrupts.
- The foreground (main) initializes the FSM, then executes `for(;;){}` do nothing loop.
- The maximum time to execute one instance of the ISR is xxxx (note: replace the xxxx with performance measure of your solution).

Another FSM Example: Stepper Motor Control

- FSM for Stepper Motor (See lab 4):

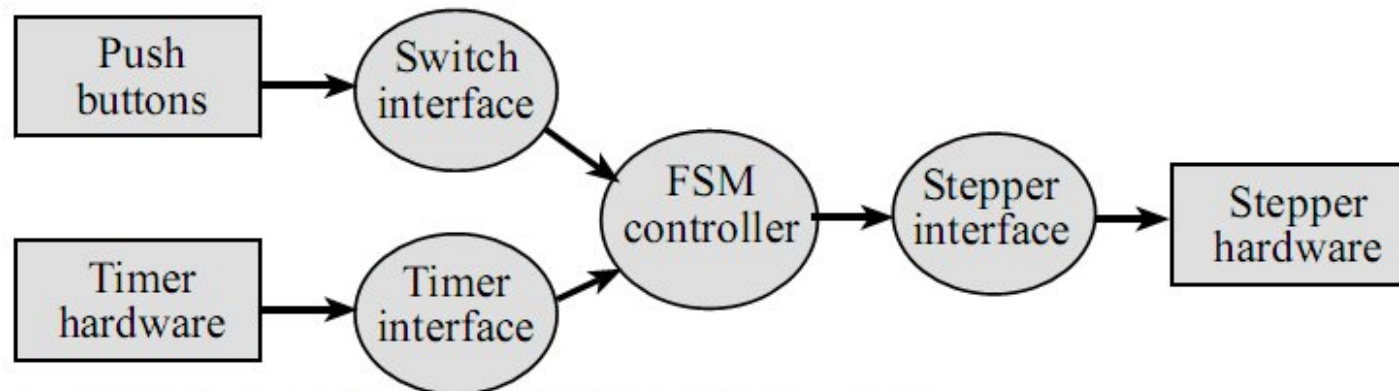


Figure 4.3. Data flows from the timer and the switches to the stepper motor.

Figure 4.4 shows a possible call graph of the system. Dividing the system into modules allows for concurrent development and eases the reuse of code.

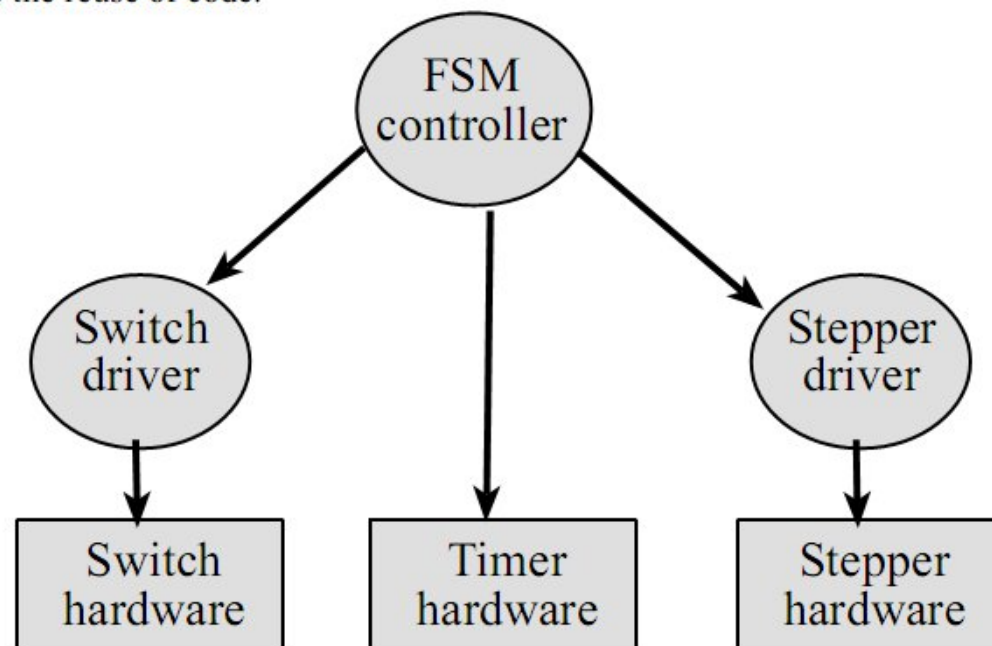


Figure 4.4. A call graph showing the three modules used by the stepper controller.

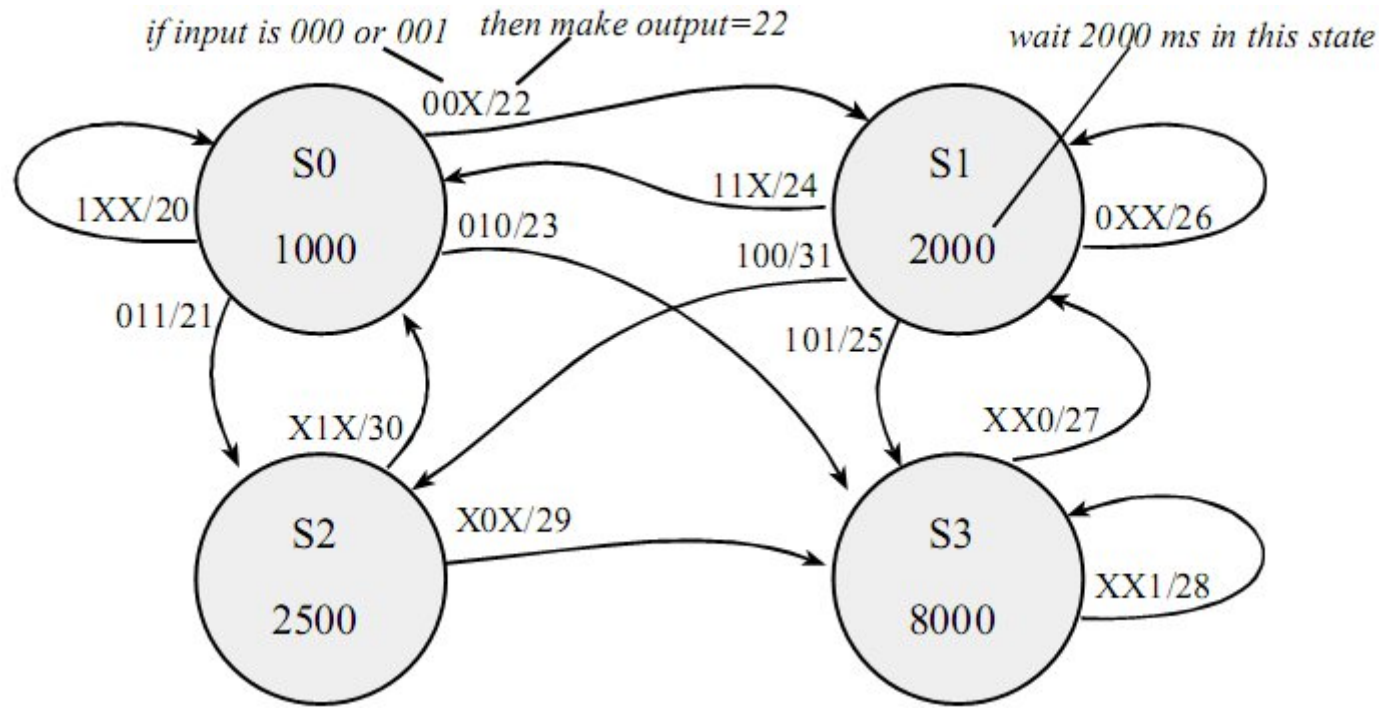
FSM Example 1

- You will use my linked data structure and periodic output compare interrupts to implement this Mealy finite state machine.
- Each state has a time to wait in milliseconds, eight 5-bit output values, and eight next state pointers. In a Mealy machine both the outputs and the next states depend on the input. For each state your software will :
 - 1) wait the specified amount of time;
 - 2) read the 3-bit input;
 - 3) perform the 5-bit output that depends on the current state and the input;
 - 4) jumps to the next state.

Which state to go to next also depends on the current state and the input.

FSM Example 1

- The hardware uses all 8 bits of PORTH, in particular your software will configure PORTH bits 4,3,2,1,0 to be outputs and bits 7,6,5 to be input.
- I am giving you the specification of the linked data structure, and you must write the output compare initialization and the output compare interrupt service routine. You may assume the main program executes your ritual, then performs other unrelated functions. Other than initialization, YOU MAY NOT PERFORM ANY FUNCTIONS IN THE FOREGROUND. S0 is the initial state.



FSM Example 1

```
const struct State {
    unsigned short wait;          // time in ms to wait
    unsigned char out[8];         // 5-bit outputs
    const struct Node *next[8];}; // Next if 3-bit input is 0-7
typedef const struct State StateType;
typedef StateType *StatePtr;
#define S0 &fsm[0]
#define S1 &fsm[1]
#define S2 &fsm[2]
#define S3 &fsm[3]
StateType fsm[4]={
    {1000,{22,22,23,21,20,20,20,20},{S1,S1,S3,S2,S0,S0,S0,S0}},
    {2000,{26,26,26,26,31,25,23,23},{S1,S1,S1,S1,S2,S3,S0,S0}},
    {2500,{29,29,30,30,29,29,30,30},{S3,S3,S0,S0,S3,S3,S0,S0}},
    {8000,{27,28,27,28,27,28,27,28},{S1,S3,S1,S3,S1,S3,S1,S3}},
};
```

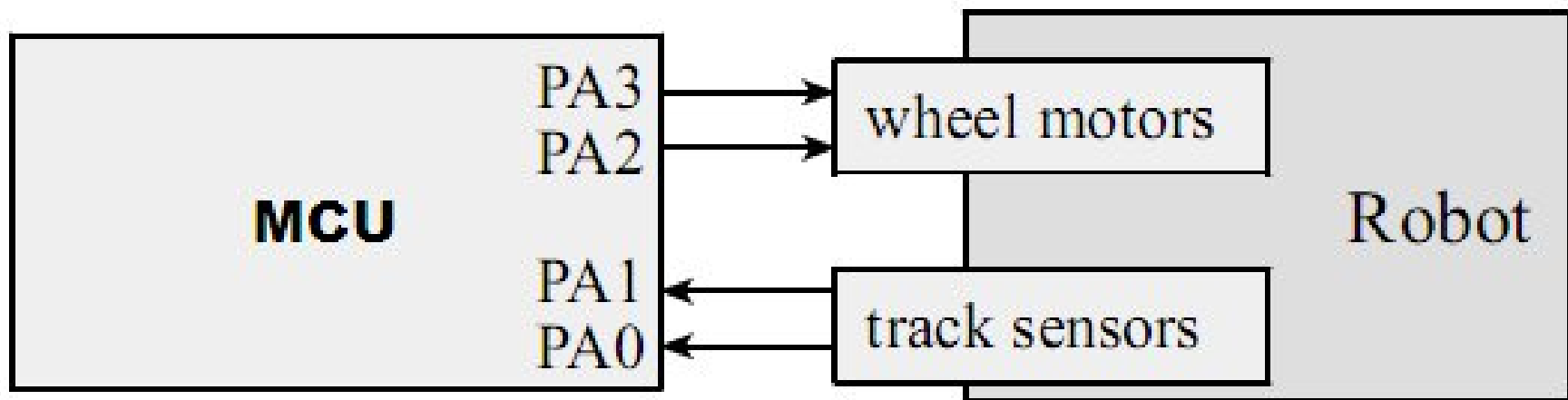
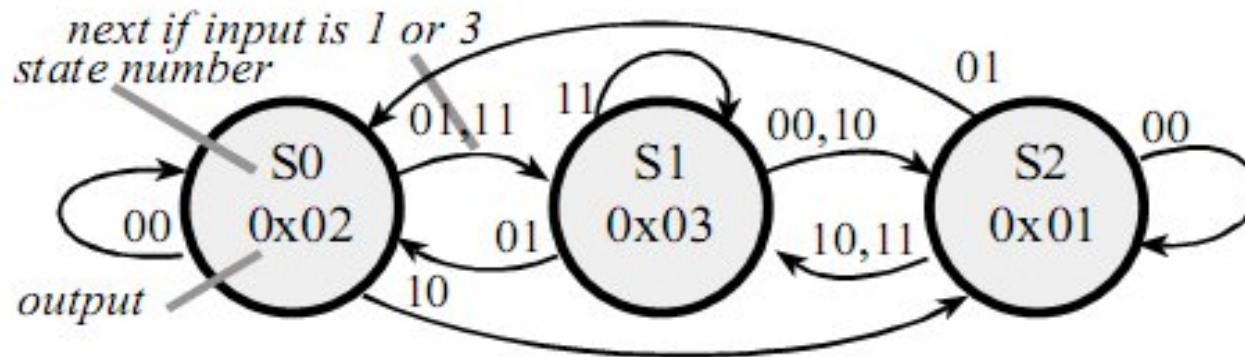
FSM Example 1: Solution

Linked data structure and output compare interrupts to implement this Mealy finite state machine.

```
StatePtr = pt;          // current state
unsigned short count;    // 1 ms counter used to create time delays
void Initialization(void){
    asm(" sei");         // make atomic
    TIOS |= 0x80;        // PT7 is output compare
    TSCR = 0x80;         // enable TCNT
    TMSK2= 0x33;         // 1us clock
    pt = S0;             // initial state
    DDRH = 0x3F;         // PH4,3,2,1,0 outputs, PH7,6,5 inputs
    count = pt->wait;     // time to wait in initial state
    TFLG1 = 0x80;        // Clear C7F
    TMSK1 |= 0x80;       // Arm output compare 7
    asm(" cli");
}
#pragma interrupt_handler TC7handler()
void TC7handler(void){ unsigned char input;
    TC7 = TC7+1000;      // interrupts every 1ms
    if(--count == 0){
        input = PORTH>>5;           // 0 through 7
        PORTH = pt->out[input];     // output depends on input and state
        pt = pt->next[input];       // next depends on input and state
        count = pt->wait;           // time to wait in this new state
    }
    TFLG1=0x80; // ack by clearing C7F
}
```

FSM Example 2: Line-Tracking Robot.

- You will use my linked data structure to implement this line-tracking robot. Each state has a 2-bit output value, and four next state pointers.
- The hardware uses 4 bits of PORTA, so your software will configure PORTA bits 3,2 to be outputs and bits 1,0 to be input. The other four bits of PORTA should not be modified.



FSM Example 2: Line-Tracking Robot.

- I am giving you the specification of the linked data structure, and you must write the main program (no interrupts and no time delays) that executes the Moore finite state machine. S1 is the initial state.

```
const struct State {
    unsigned char out;          // 2-bit output
    const struct Node *next[4]; // Next if 2-bit input is 0-3
} typedef const struct State StateType;
typedef StateType *StatePtr;
#define S0 &fsm[0]
#define S1 &fsm[1]
#define S2 &fsm[2]
StateType fsm[3]={
    {0x02,{S0,S1,S2,S1}},
    {0x03,{S2,S0,S2,S1}},
    {0x01,{S2,S0,S1,S1}}
};
```

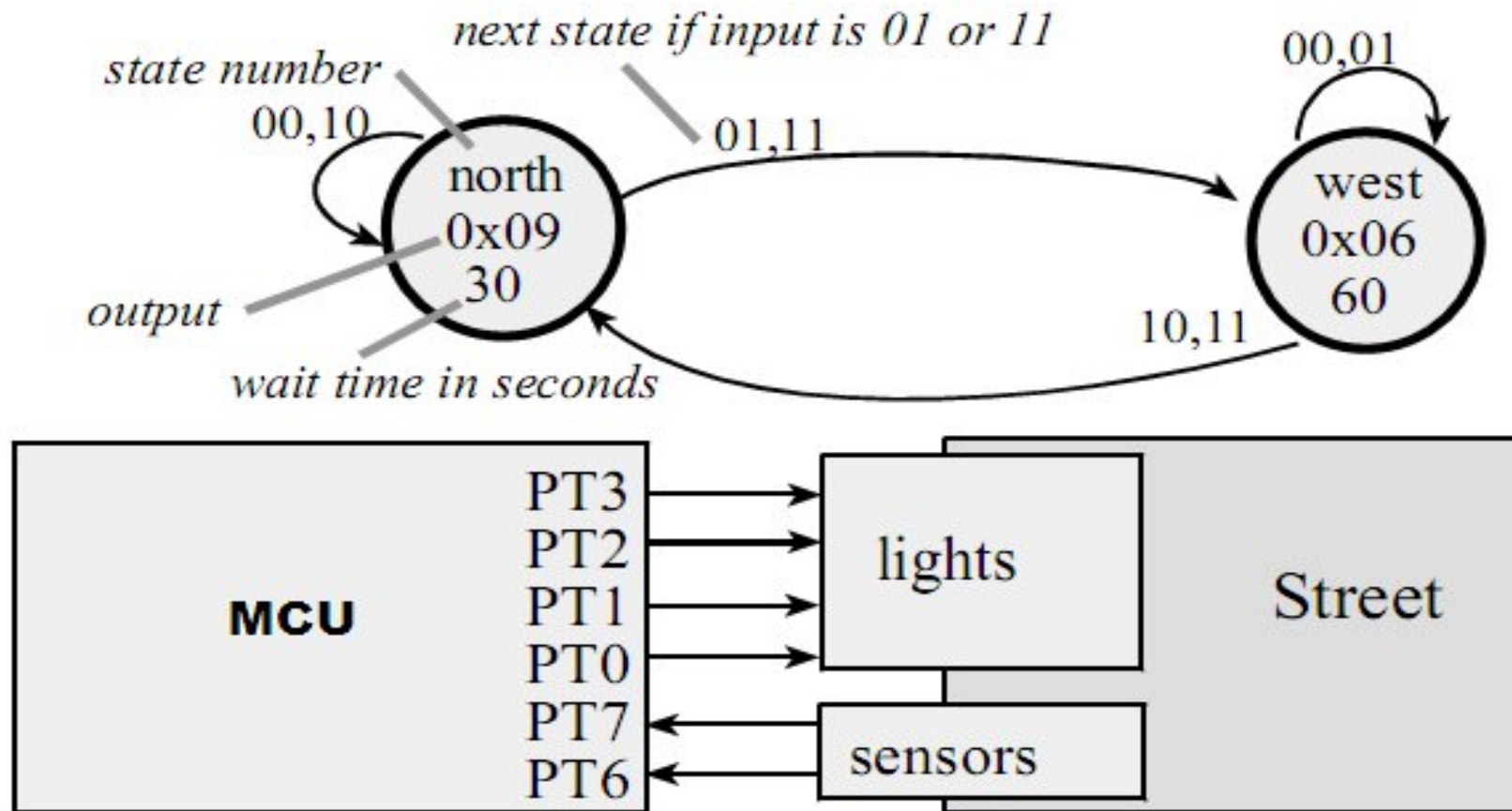
Sol. to FSM Example 2: Line-Tracking Robot.

- **The code to execute the FSM.**

```
void main(void){
StatePtr Pt;          // Current State
int Input;
Pt = S1;              // Initial State
DDRA |= 0x0C;         // PA3,2 are outputs
DDRA &= ~0x03;        // PA1,0 are inputs
while(1){
    PORTA = (PORTA&0xF0)+(Pt->Out)<<2; // Output depends on the current
state
    Input = PORTA&0x03;                // Input=0,1,2,or 3
    Pt = Pt->Next[Input];              // Next state depends on the input
}
}
```


FSM Example 3: Traffic Lights

- You will implement this traffic light finite state machine. Each state has a 4-bit output value, and four next state pointers. The sequence is output, wait, input, next.
- The hardware uses 6 bits of PORTT, so your software will configure PORTT bits 7,6 to be inputs and bits 3-0 to be output.



FSM Example 3: Traffic Lights

- You will write two regular C functions, `ritual()` and `machine()`. My main program calls your `ritual()` once before I enable interrupts. My RTI handler will call your `machine()` every 1 second. You are not allowed to change my code.

```
unsigned short Count;
void interrupt 7 handler(){ // interrupts at 244Hz
    CRGFLG = 0x80; // acknowledge
    if((--Count)==0){ // every 1 second
        machine(); // your program
        Count = 244;
    }
}
void RTI_Init(void){
    CRGINT = 0x80; // arm
    RTICTL = 0x33; // period=250ns*4096*4=4096us or 244.14Hz
    Count = 244; // interrupt counter
    asm cli
}
```

FSM Example 3: Traffic Lights

```
void main(void){  
    ritual()    // your program  
    RTI_Init();  
    for(;;){  
    }  
}
```

- Part a) I am giving you the specification of the linked data structure. You can not change this specification. Show the C code that defines this finite state machine

```
const struct State {  
    unsigned char out;        // 4-bit output  
    unsigned short wait;      // time in seconds to wait  
    const struct Node *next[4]; // Next if 2-bit input is 0-3  
};  
typedef const struct State StateType;
```

- Part b) You may add global variables as needed.

FSM Example 3: Traffic Lights

- Part c) Write the `ritual()` function. Among other things the ritual should initialize the PORTT direction register, DDRT and set the initial state to north.
- Part d) Write the `machine()` function that implements the FSM. In particular, you will perform input and output using PORTT. Since this is executed in the background, no backwards jumps (do while, while, or for) are allowed.
- *Note: If you combine my C code that I put on this page with your code that you place in parts a,b,c,d, it will constitute the entire software system to run this FSM.*

Sol. To FSM Example 3: Traffic Lights

- **a) Show the C code that defines the finite state machine.**

```
#define north &fsm[0]
#define west &fsm[1]
StateType fsm[2]={
    {0x09,30,{north,west,north,west},
    {0x06,60,{north,west,north,west}}};
```

- **b) You may add global variables as needed.**

```
unsigned short Time;
StateType *Pt;
```

Sol. To FSM Example 3: Traffic Lights

- **c) Show the ritual() function that initializes the finite state machine.**

```
void ritual(void){  
    DDRT = 0x0F;    // PT7-6 inputs, PT3-0 outputs  
    Pt = north;    // initial state  
    PTT = Pt->out; // initial output  
    Time = Pt->wait; // time to wait in initial state  
}
```

- **d) Show the machine() function that executes the finite state machine.**

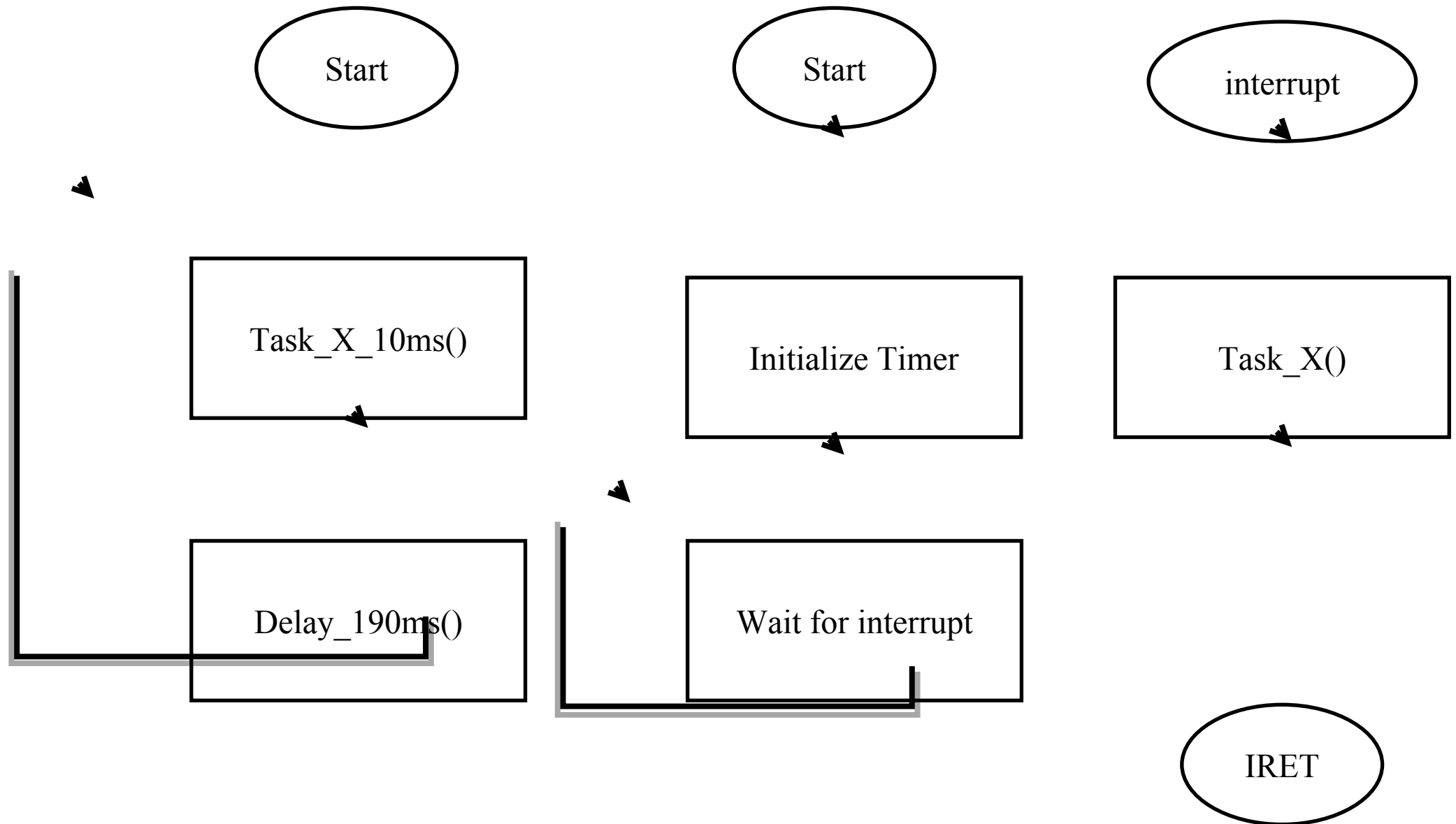
```
void machine(void){unsigned char input;  
    if((--Time)==0){  
        input = (0xC0&PTT)>>6; // 0,1,2,3  
        Pt = Pt->next[input]; // next state  
        PTT = Pt->out;        // output  
        Time = Pt->wait;      // time to wait in next state  
    }  
}
```

System/SW Design

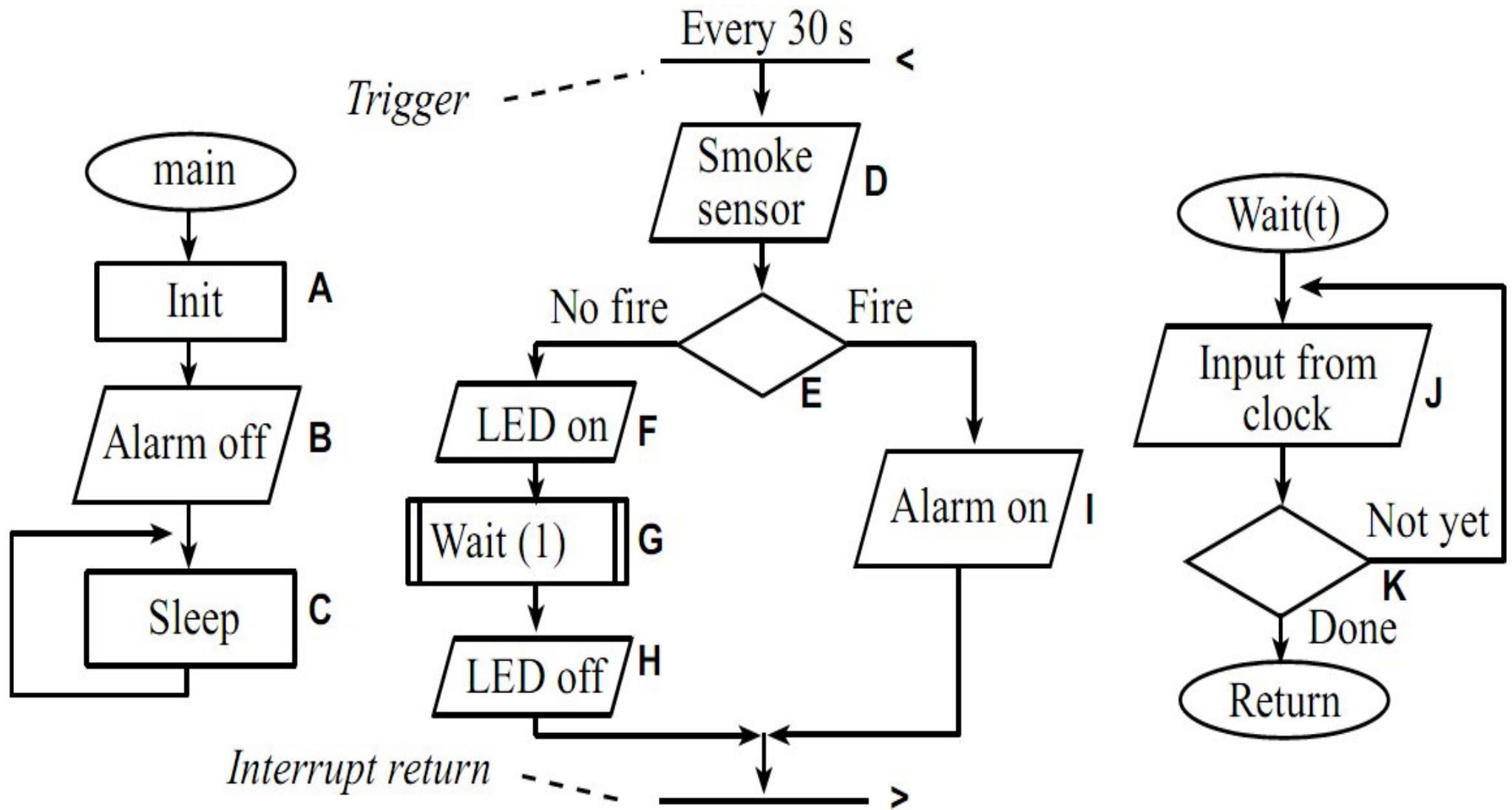
- System Design
- SW/Program Design
- Finite State Machine (FSM)
- **Software Architecture**
- Interfacing
and I/O Synchronization
- Debugging Techniques



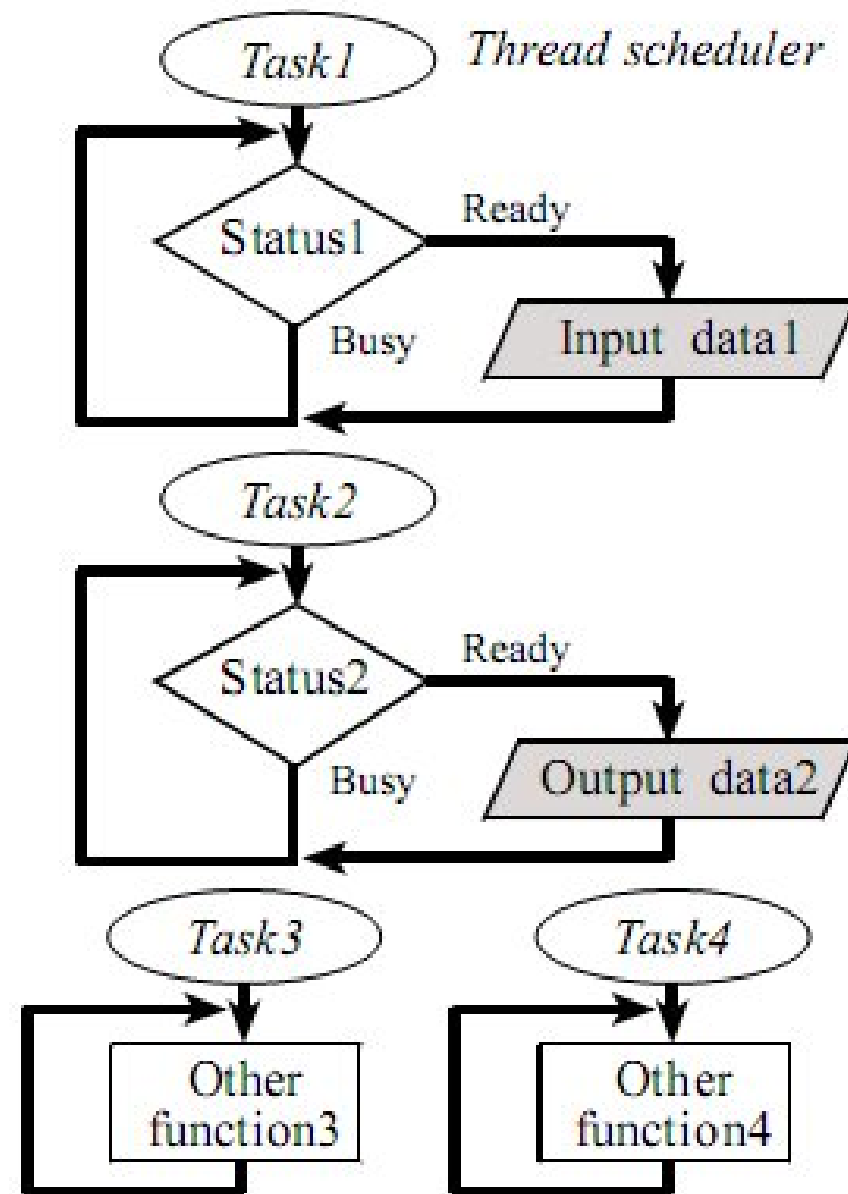
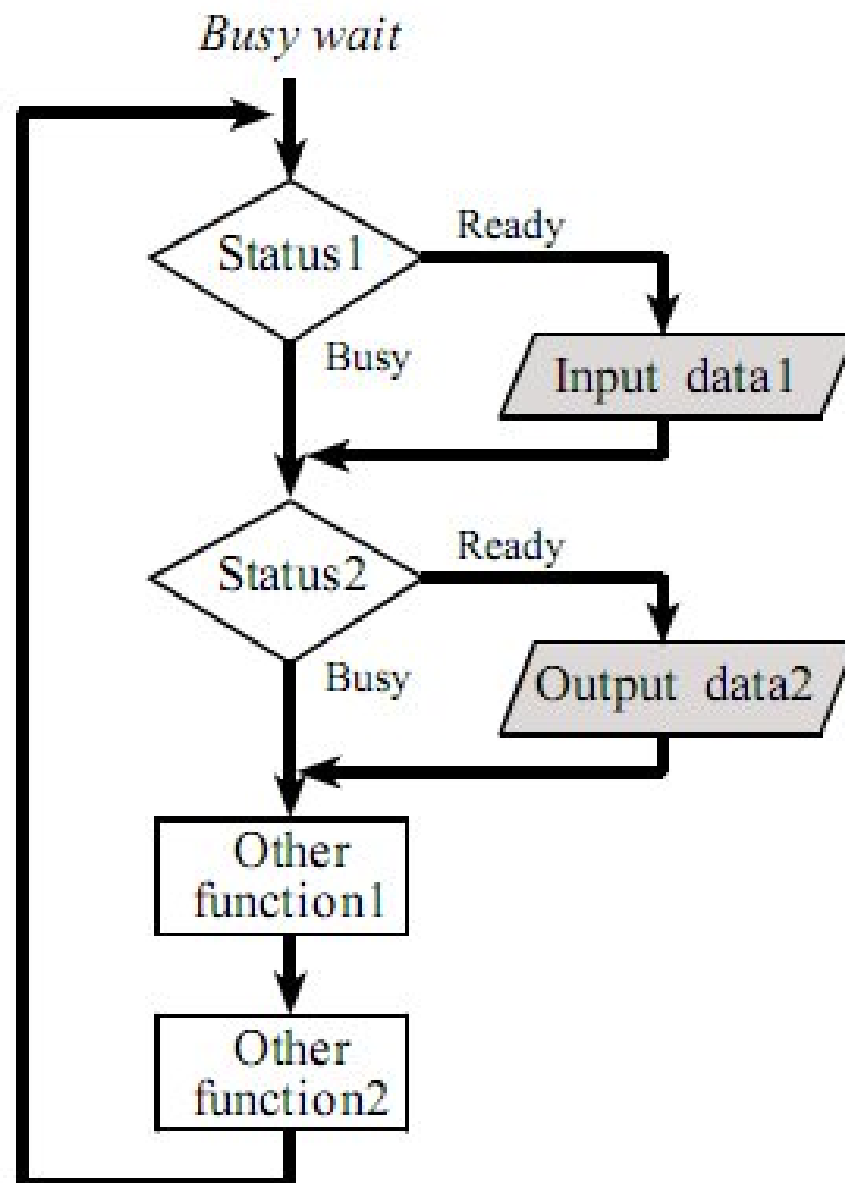
SW Architectures in Embedded Systems



Low Power Design



Single Thread vs Multi-Thread



SW Architectures in RTOS

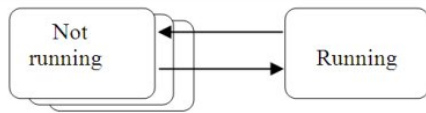


Fig. 1. Simplified life cycle of a task

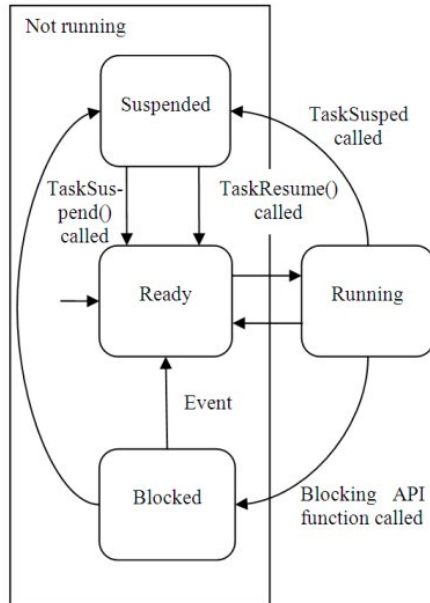


Fig. 2. Life cycle of a task

- Tracking input to Controller (Task1)
- ZigBee Communication (Task2)
- Error handling (Task3)
- Monitor process value (Task4)

In this study four tasks are created, namely Task1 (uctsk_setpoint), Task2 (uctsk_zigbee), Task3 (uctsk_error) and Task4 (uctsk_display). The system initialization flow chart is shown in Fig. 6. The function of Task1 is getting the setpoint values from server and writes these data into controller. Task1 flowchart is given in Fig. 7.

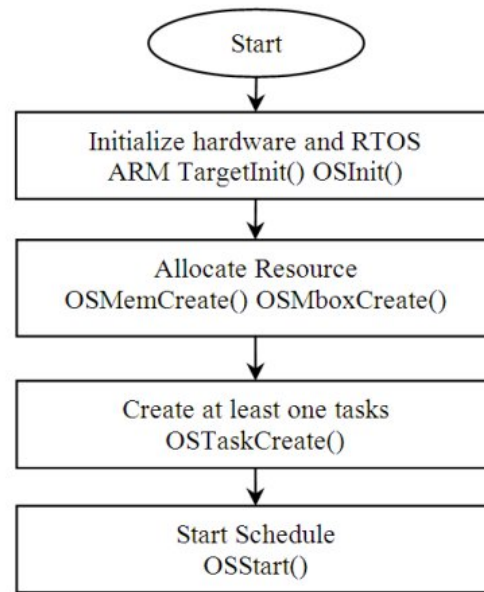


Fig. 6. Flowchart for initialization

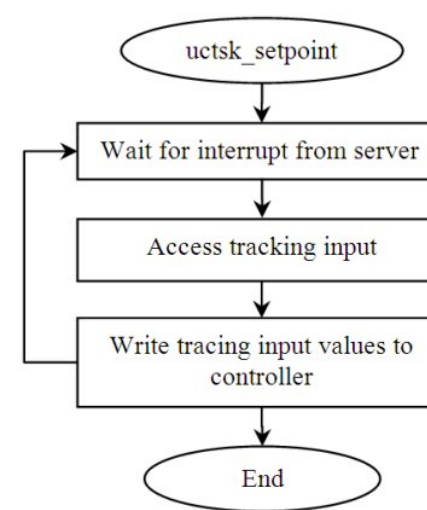


Fig. 7. Flowchart for set point tracking

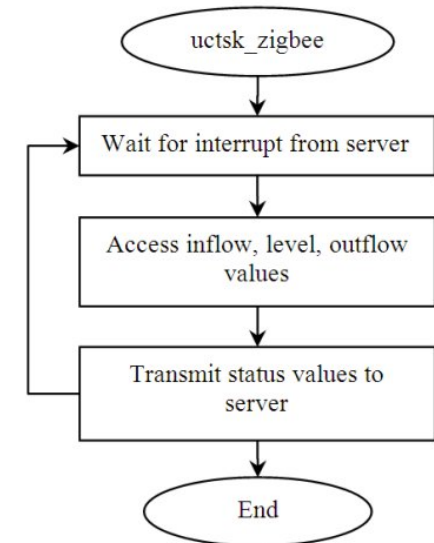


Fig. 8. Flowchart for Zigbee communication in Task2

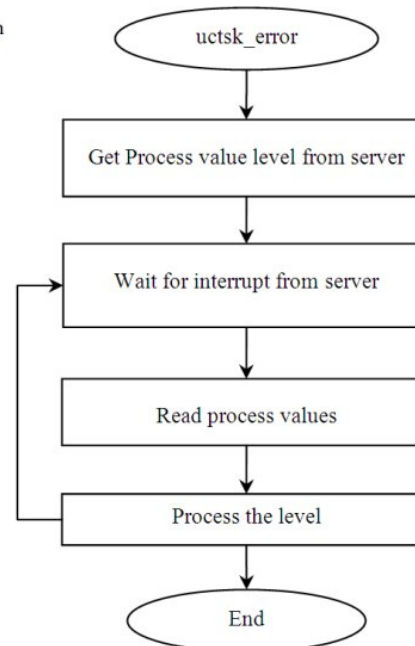


Fig. 9. Flowchart for error tracking

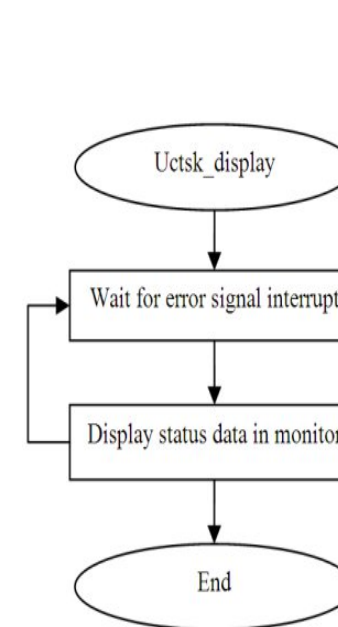


Fig. 10. Flowchart for display parameters without critical section

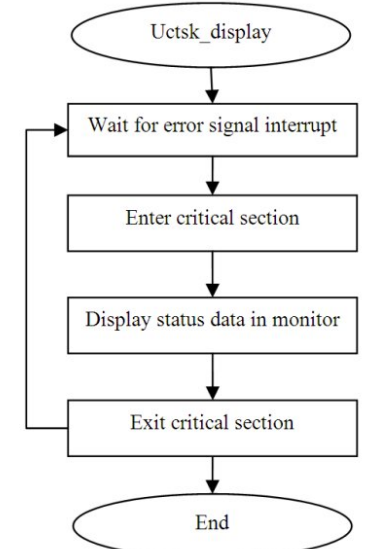
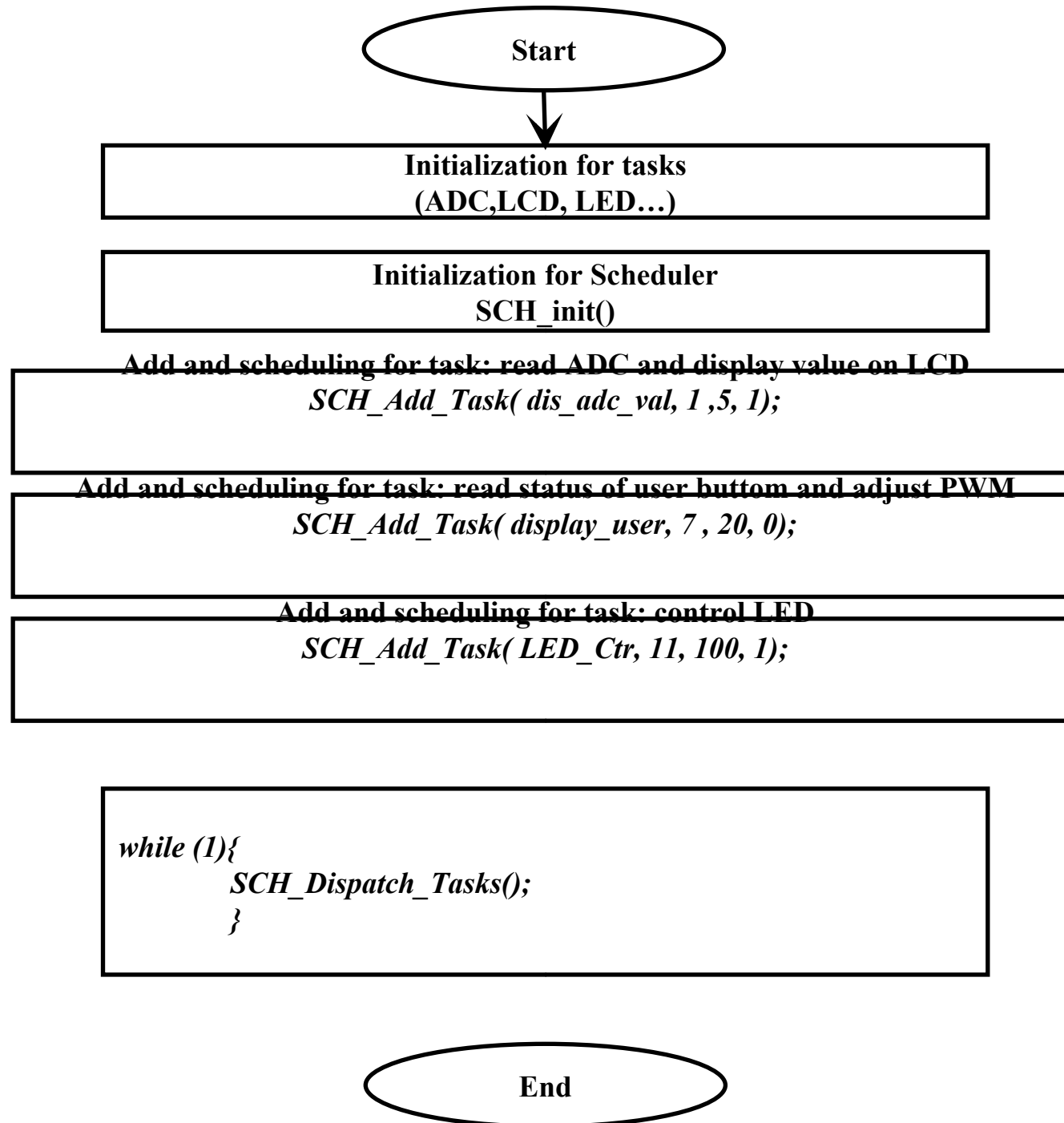
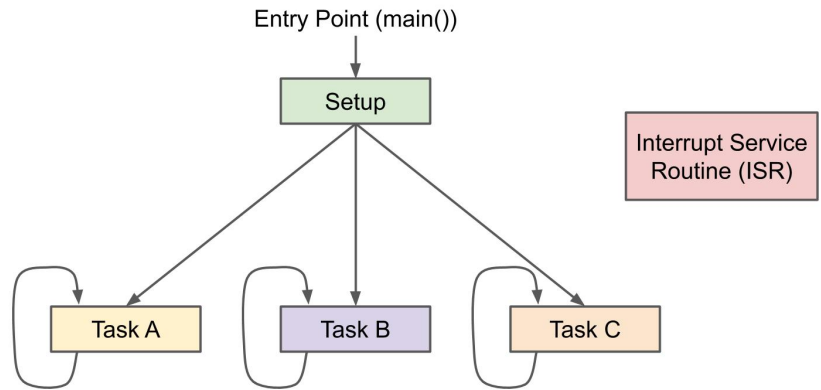


Fig. 11. Flowchart for display parameters with critical section

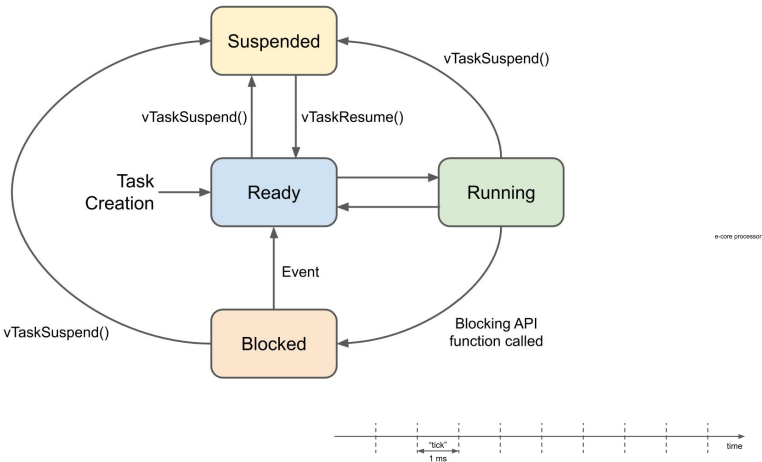
Main program



What our code looks like

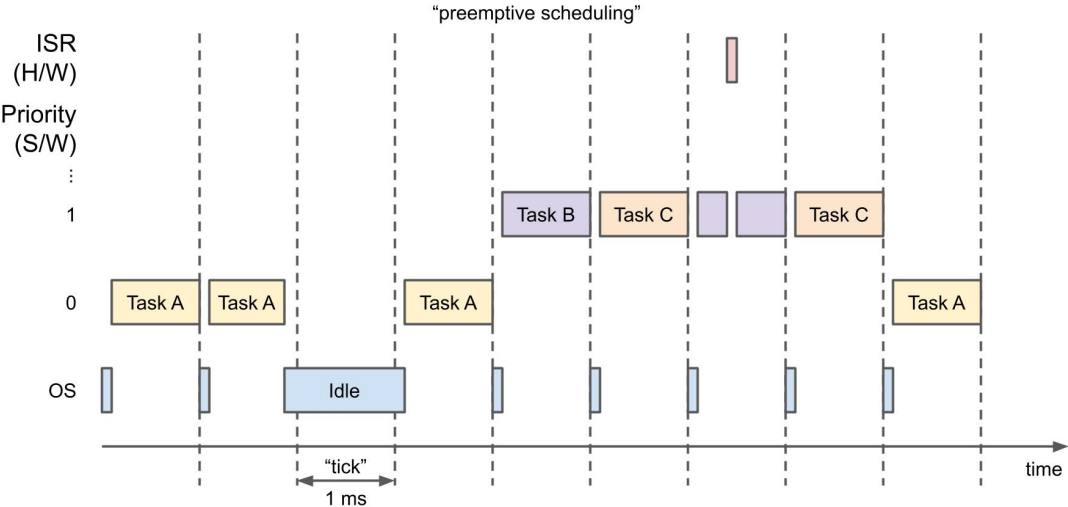


Task States

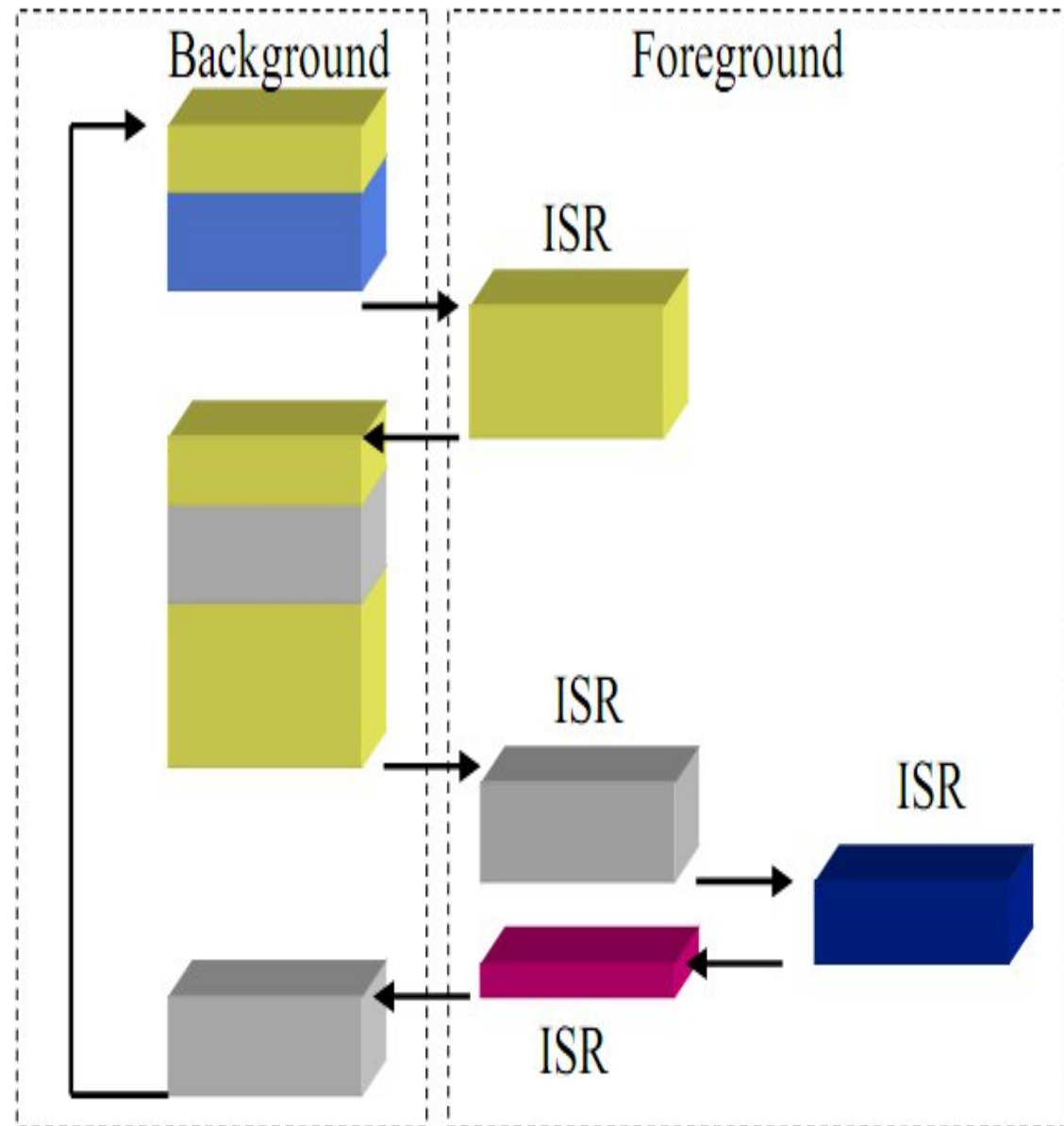


What actually happens*

*assuming single-core processor



Super-Loop Execution



```
void main (void) {      // background
  initialization;
  FOREVER {
    read analog inputs;
    read discrete inputs;
    perform monitoring functions;
    perform control functions;
    update analog outputs;
    scan keyboard;
    handle communication requests;
    update display;
  }
}

ISR (void) {             // foreground
  handle asynchronous events;
}
```

Simple Super-Loop Architecture

- A particular limitation with this architecture is that it is very difficult to execute function $X()$ at precise intervals of time (No control over execution timing).
 - *Note: Task/Function $X()$ can consist of several processes (e.g., $p1()$ and $p2()$) => No control over timing of individual processes $p1()$ and $p2()$.*

```
void main(void)
{
    /* Prepare run function X */
    X_Init();

    while(1) /* 'for ever' (Super Loop) */
    {
        X(); /* Run function X */
    }
}
```

```
void main(void)
{
    Init_System();

    while(1) /* 'for ever' (Super Loop) */
    {
        X(); /* Call the function (10 ms duration) */
        Delay_50ms(); /* Delay for 50 ms */
    }
}
```

This will be fine, if:

1. We know the precise duration of function $x()$, and,
2. This duration never varies.

Polling

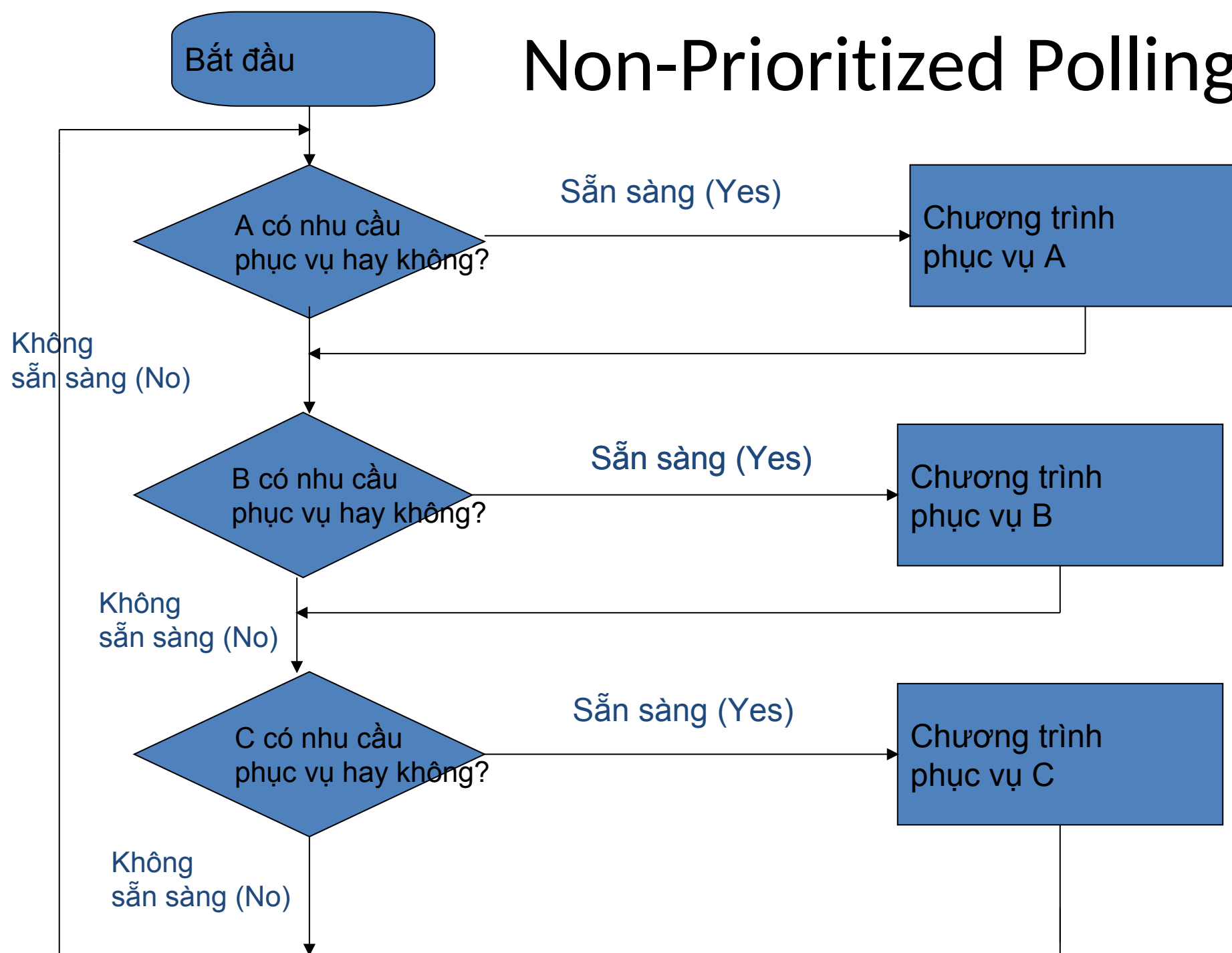
Đặc điểm:

- . Điều khiển vào /ra đơn giản và bằng phần mềm.
- . Thời gian của CPU bị chiếm nhiều để thực hiện chương trình lặp thăm dò.
- . Chỉ nên dùng cho một số thiết bị.
- . Các thiết bị ngoại vi phải đưa ra các bit cờ trạng thái sẵn sàng của thiết bị (trong thanh ghi trạng thái).
- . Phần mềm kiểm tra các bit trạng thái sẵn sàng của thiết bị có trong hệ thống vi xử lý bằng các vòng lặp. Thiết bị nào có trạng thái sẵn sàng thì được phục vụ.
- . Có thể thăm dò tùy ý nhưng phải thăm dò hết các thiết bị có trong có hệ thống và có thể thăm dò theo ưu tiên phục vụ.
- . Dễ dàng thực hiện các sơ đồ ưu tiên(priority schemes).

Non-Prioritized Polling

- Đọc nội dung của thanh ghi trạng thái để kiểm tra.
- Chỉ thăm dò hết các t/b thì vòng thăm dò mới quay trở lại để bắt đầu vòng thăm dò tiếp theo.
- Phụ thuộc vào đặc điểm của t/b ngoại vi mà số vòng nhiều hay ít.
- Phù hợp các hệ thống vi xử lý nhỏ chỉ có một ít t/b ngoại vi và cung thương tương tự nhau về đặc điểm.

Non-Prioritized Polling



Điều khiển vào/ra bằng thăm dò không ưu tiên

Non-Prioritized Polling Code Example

```
while (1) {  
    if (TA_Ready)  
        {TaskA(); }  
  
    if (TB_Ready)  
        {TaskB(); }  
  
    if (TC_Ready)  
        {TaskC(); }  
}
```

Prioritized Polling

- Task readiness checked in order of priority
- Task runs to completion

$$T_{response} = \sum_{i < n} N_i T_{Ti} + \max[T_n, T_{n-1}, \dots] + T_{TDn} + T_{cir} + \sum T_{int, srv}$$

higher priority tasks; N_i = #times T_i readv	time to finish a lower priority task	context switch & OS overhead	circuit delays	service interrupts
---	--	---------------------------------------	-------------------	-----------------------

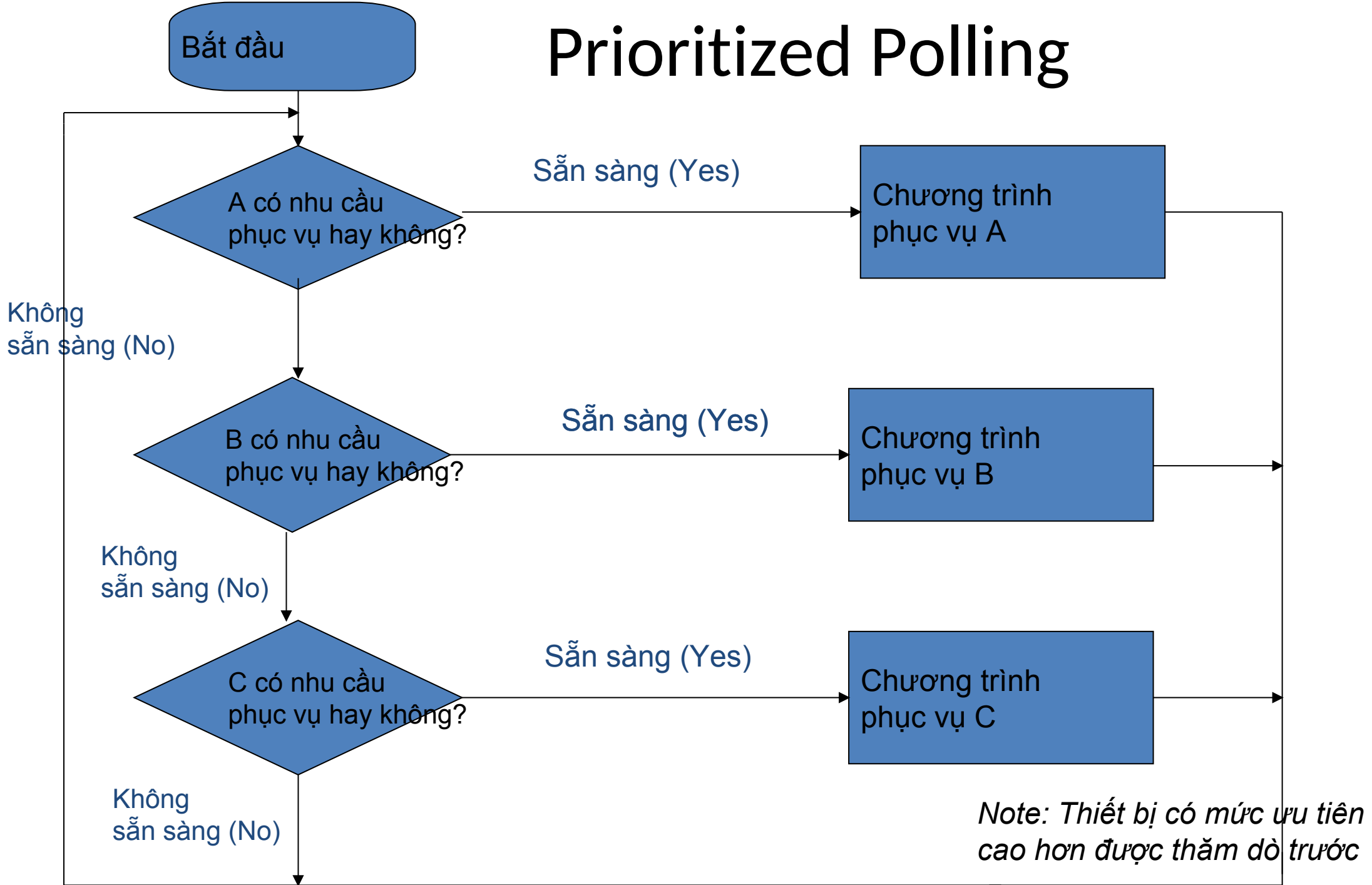
Determining Task Priorities

-  In real-time system assume normal state for all tasks is blocked!
 - ❖ Allows other tasks to execute
 - ❖ A task that runs continuously will prevent lower-priority tasks from executing

-  When a task executes it will delay execution of all lower-priority tasks for as long as it is running.
 - ❖ Determine task priority according to:
 - 🕒 a) For how long (max) they execute before blocking. Long execution time => low priority
 - 🕒 b) How much delay can they tolerate between being ready-to-run and running. Long delay Ok => lower priority

-  Often task priority does not matter very much. When it does we will learn how to quantify delays later.

Prioritized Polling



Điều khiển vào/ra bằng thăm dò ưu tiên

Prioritized Polling Code Example

```
while (1) {  
    if (TA_Ready)  
        {TaskA(); }  
    else if (TB_Ready)  
        {TaskB(); }  
    else if (TC_Ready)  
        {TaskC(); }  
}
```

Time-slice Scheduling

```
while (1) {  
    wait_for_timer();  
    if (TA_Ready)  
        {TaskA(); }  
    else if (TB_Ready)  
        {TaskB(); }  
    else if (TC_Ready)  
        {TaskC(); }  
}
```



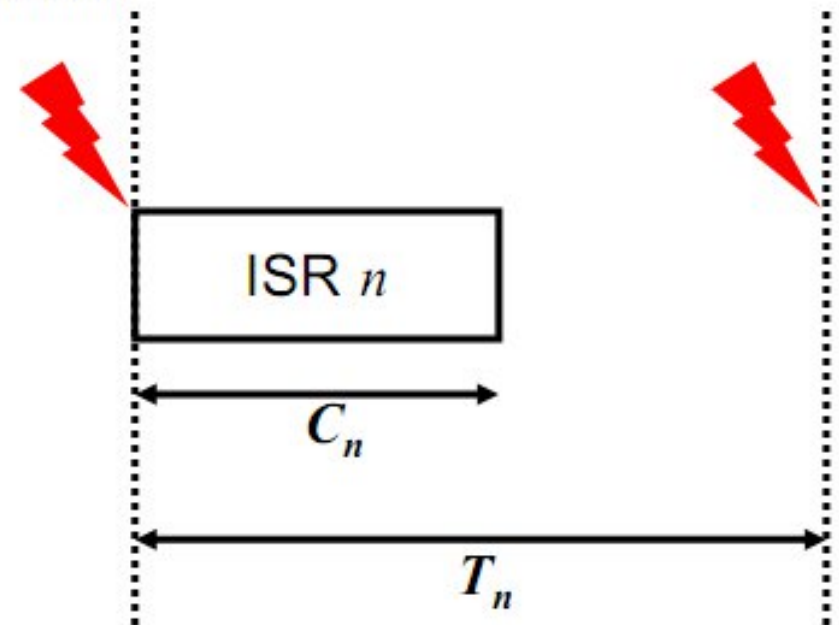
EOS using Interrupts

- Interrupts are the fundamental device which allows sequential computer programs to respond to real-time events
 - Provide a very fast response to a hardware event: Timer timeout, I/O request
- Interrupts require hardware support
 - ALL CPUs provide some way of implementing interrupts
 - Boundary between hardware & software support is variable
 - As a minimum save/switch of PC must be implemented in hardware
- Interrupt processing:
 - Stop code currently executing
 - Branch to Interrupt Service Routine (ISR)
 - Execute ISR
 - Return transparently to previous executing code
- Note that ISR is very different from a task – no state is saved from one invocation to the next
 - Can't implement infinite loop
 - Can't block

Rate Monotonic Analysis for Interrupts

- ◆ Prioritised interrupts provide response to deadlines very similar to prioritised tasks implementing jobs
 - ❖ RMA analysis can be used to determine whether all ISRs will terminate before next ISR invocation

Interrupt priority n
Hardware event initiates computation



Interrupt	Interrupt n occurs	ISR n runs	ISR n returns
Task	Task n becomes ready	Task n runs	Task n blocks

Foreground/Background System Design

- Real-time systems can be implemented without an RTOS using foreground/background design
 - Time-critical computation implemented via interrupt-driven ISR **(foreground)**
 - Non-critical computation implemented in **background** loop
 - Note only one background loop is possible
 - Non-critical hardware can be serviced from background loop using polling
 - Loop length is variable depending on which devices need to be serviced
 - Worst-case loop length determines latency for polled devices
- Advantages: Simple and more efficient than RTOS
- Disadvantages
 - More computation must be implemented in ISR – coding more complex & difficult to debug
 - Can't use tasks to simplify & modularise design

Polling from Timer interrupts

- ◆ **Interrupt-driven paradigm locates computation in specific ISRs executed in response to hardware events**
 - ❖ Interrupt-driven computation in ISR triggered by hardware event
 - ❖ Response time is limited by CPU-determined interrupt time + execution time of any higher or equal priority interrupts
- ◆ **Timer-interrupt ISR polls multiple devices at regular intervals**
 - ❖ provides guaranteed response with less hardware overhead than device-specific hardware interrupts
 - ❖ Simpler to code than multiple separate ISRs
- ◆ **Cf time-interrupt polling & background loop polling**
 - ❖ Background loop also implements polled computation – but time between successive polls cannot easily be determined.

```
BackgroundLoop()
{
    for (;;) {
        if (device A is ready) [ Service Device A ];
        if (device B is ready) [ Service Device B ];
    }
}

Timer1_ISR() // device C or D
{
    if (device C is ready) [ Service Device C ];
    if (device D is ready) [ Service Device D ];
    [ update system clock ]
}

DeviceE_ISR()
{
    [ Service Device E ]; /*NB polling not needed*/
}
```

Response-time:

Timer ISR polling vs background loop

- Can use one or more timer ISRs running at different speeds
 - e.g. 1ms and 10ms – to implement half-way house between polled & interrupt-driven paradigms
 - 1ms interrupt runs higher priority than 10ms interrupt
 - 1ms interrupt provides response-time (latency) of 1ms worst case, but total length of ISR must be $\ll 1\text{ms}$
 - 10ms interrupt provides response-time of 10ms worst case. Total length of ISR must be $\ll 10\text{ms}$. Note that 10ms ISR execution time is effectively slowed down by CPU utilisation due to 1ms interrupt (and other high priority interrupts)
- Background loop can be used to implement polled computation which is not time-critical.
 - Polling time for background loop is variable and depends on overall loop period.

Simple EOS using Timer-based Interrupts

- Timer-based interrupts as the core of an embedded OS

```
#include "Main.H"
#include "Port.H"
#include "Simple_EOS.H"

#include "X.H"

/* ----- */

void main(void)
{
    /* Prepare for dummy task */
    X_Init();

    /* Set up simple EOS (60 ms tick interval) */
    sEOS_Init_Timer2(60);

    while(1) /* Super Loop */
    {
        /* Enter idle mode to save power */
        sEOS_Go_To_Sleep();
    }
}
```

Simple EOS using Timer-based Interrupts

```
/*-----*/
Simple EOS.C (v1.00)
/*-----*/
#include "Main.H"
#include "Simple_EOS.H"

/* Header for dummy task */
#include "X.H"

/*-----*/

sEOS_ISR()

    Invoked periodically by Timer 2 overflow:
    see sEOS_Init_Timer2() for timing details.

/*-----*/
sEOS_ISR() interrupt INTERRUPT_Timer_2_Overflow
{
    /* Must manually reset the T2 flag */
    TF2 = 0;

    /*===== USER CODE - Begin ===== */

    /* Call dummy task here */
    X();

    /*===== USER CODE - End ===== */
}

/*-----*/
sEOS_Go_To_Sleep()

    This operating system enters 'idle mode' between clock ticks
    to save power. The next clock tick will return processor
    to the normal operating state.

/*-----*/
void sEOS_Go_To_Sleep(void)
{
    PCON |= 0x01;    /* Enter idle mode (generic 8051 version) */
}
```

```
/*-----*/
sEOS_Init_Timer2()

/*-----*/
void sEOS_Init_Timer2(const tByte TICK_MS)
{
    tLong Inc;
    tWord Reload_16;
    tByte Reload_08H, Reload_08L;

    /* Timer 2 is configured as a 16-bit timer,
       which is automatically reloaded when it overflows */
    T2CON = 0x04;    /* Load T2 control register */

    /* Number of timer increments required (max 65536) */
    Inc = ((tLong)TICK_MS * (OSC_FREQ/1000)) /
          (tLong)OSC_PER_INST;

    /* 16-bit reload value */
    Reload_16 = (tWord) (65536UL - Inc);

    /* 8-bit reload values (High & Low) */
    Reload_08H = (tByte) (Reload_16 / 256);
    Reload_08L = (tByte) (Reload_16 % 256);

    /* Used for manually checking timing (in simulator) */
    /*P2 = Reload_08H; */
    /*P3 = Reload_08L; */

    TH2 = Reload_08H;    /* Load T2 high byte */
    RCAP2H = Reload_08H; /* Load T2 reload capt. reg h byte */
    TL2 = Reload_08L;    /* Load T2 low byte */
    RCAP2L = Reload_08L; /* Load T2 reload capt. reg l byte */

    /* Timer 2 interrupt is enabled, and ISR will be called
       whenever the timer overflows. */
    ET2 = 1;

    /* Start Timer 2 running */
    TR2 = 1;

    EA = 1;    /* Globally enable interrupts */
}
```

Simple EOS using Timer-based Interrupts

```
/*-----*  
  X.C (v1.00)  
  Dummy task to introduce sEOS.  
-----*/  
  
#include "X.H"  
  
void X_Init(void)  
{  
  /* Dummy task init... */  
}  
  
/*-----*  
  X()  Dummy task called from sEOS ISR.  
-----*/  
  
void X(void)  
{  
  /* Dummy task... */  
}  
  
/*-----*  
  ---- END OF FILE -----
```

Comments on Simple EOS (sEOS)

- Advantages:
 - This sEOS imposes a very low processor load and is rather flexible and useful for simple applications
- Disadvantages:
 - *Task/Function X() can consist of several processes (e.g., p1() and p2()) => No control over timing of individual processes => use multiple-timer based interrupt or RTOS*
 - Unlike a traditional RTOS, this sEOS becomes part of the application itself, rather than forming a separate code layer
=> make the code less abstract and clarified
 - Hard to apply to systems of complexity with many timing deadlines and constraints, priorities.

Simple EOS using Multiple-Timer Based Interrupts

- Each task has its own function and own set of processes
 - E.g., Task/Function X1() has processes p1() and p2().
- Each task has its own timer.
 - E.g., can allocate timer 1 for task X1 & timer 2 for task X2
 - May not have enough timers to implement all the rates.
 - Complexity and unreliability due to the presence of multiple interrupts

```
void X1(){  
/* rate T1 for task X1()  
*/  
    p1();  
    p3();  
}  
  
void X2(){  
/* rate T2 for task X2()  
*/  
    p2();  
    p4();  
    p5();  
}
```

Simple EOS using Virtual Multiple-Timer-Based Interrupts

- Instead of using 'Real' multiple timers, use a software count to divide the timer (i.e., create virtual multiple timers)
- Only works for clean multiples of the timer period.
- Incur overhead and inaccuracy due to the operation for creating virtual multiple timers (e.g., checking instructions or updating the variable countx).

```
int count2, count3 = 0;
void X(){
    /* rate T1 for p1() */
    p1();
    /* rate T2 for p2() */
    if (count2 >= 2) {
        p2();
        count2 = 0;
    } else count2++;
    /* rate T3 for p3() */
    if (count3 >= 5) {
        p3();
        count3 = 0;
    } else count3++;
}
```

Multi-State System

- **Multi-State (Timed)**

In a multi-state (timed) system, the transition between states will depend only on the passage of time.

For example, the system might begin in State A, repeatedly executing `FunctionA()`, for ten seconds. It might then move into State B and remain there for 5 seconds, repeatedly executing `FunctionB()`. It might then move back into State A, ad infinitum.

A basic traffic-light control system might follow this pattern.

- **Multi-State (Input / Timed)**

This is a more common form of system, in which the transition between states (and behaviour in each state) will depend both on the passage of time and on system inputs.

For example, the system might only move between State A and State B if a particular input is received within X seconds of a system output being generated.

The autopilot system discussed at the start of this seminar might follow this pattern, as might a control system for a washing machine, or an intruder alarm system.

Please also note that the system states will usually be represented by means of a `switch` statement in the operating system ISR.

Multi-State (Timed) Example: Traffic light sequencing

In the code, we will represent these states as follows:

```
/* Possible system states */
typedef enum {RED, RED_AND_AMBER, GREEN, AMBER} eLight_;
```

We will store the time to be spent in each state as follows:

```
/* (Times are in seconds) */
#define RED_DURATION 20
#define RED_AND_AMBER_DURATION 5
#define GREEN_DURATION 30
#define AMBER_DURATION 5

void main(void)
{
    /* Prepare to run traffic sequence */
    TRAFFIC_LIGHTS_Init(RED);

    /* Set up simple EOS (50 ms ticks) */
    sEOS_Init_Timer2(50);

    while(1) /* Super Loop
    */ {
        /* Enter idle mode to save power */
        sEOS_Go_To_Sleep();
    }
}
```

```
sEOS_ISR() interrupt INTERRUPT_Timer_2_Overflow
{
    TF2 = 0;
    TRAFFIC_LIGHTS_Update();
}

void TRAFFIC_LIGHTS_Update(void)
{
    switch (Light_state_G) {

        case RED: {

            Red_light = ON;
            Amber_light = OFF;
            Green_light = OFF;

            if (++Time_in_state == RED_DURATION) {

                Light_state_G = RED_AND_AMBER;
                Time_in_state = 0;
            }

            break; }

        ...

        case AMBER: {

            Red_light = OFF;
            Amber_light = ON;
            Green_light = OFF;

            if (++Time_in_state == AMBER_DURATION) {

                Light_state_G = RED;
                Time_in_state = 0;
            }

            break; }

    }
}
```

Multi-State (Input/Timed) Example: Washing machine

```

/* -----
void WASHER_Update(void)
{
    /* Call once per second */
    switch
    (System_state_G) {
        case
            INIT:
            { /* For demo purposes only */
                Debug_port = (tByte) System_state_G;

                /* Set up initial state */
                /* Motor is off */
                WASHER_Control_Motor(OFF);

                /* Pump is off */
                WASHER_Control_Pump(OFF);

                /* Heater is off */
                WASHER_Control_Water_Heater(OFF);

                /* Valve is closed */
                WASHER_Control_Water_Valve(OFF);

                /* Wait (indefinitely) until START is p
            if (WASHER_Read_Start_Switch() !=
                1) {
                    return
                ; }

            /* Start switch pressed ->
               read the selector dial */
            Program_G = WASHER_Read_Selector_Dial()

            /* Change state */
            System_state_G =
                START; break;
        }
        .....
    }

case START: {

    /* For demo purposes only */
    Debug_port = (tByte) System_state_G;

    /* Lock the door */
    WASHER_Control_Door_Lock(ON);

    /* Start filling the drum */
    WASHER_Control_Water_Valve(ON);

    /* Release the detergent (if any) */
    if (Detergent_G[Program_G] == 1) {

        WASHER_Control_Detergent_Hatch(ON);

    }

    /* Ready to go to next state */
    System_state_G = FILL_DRUM;
    Time_in_state_G = 0;

    break;
}
    .....

case HEAT_WATER:
{
    /* For demo purposes only */
    Debug_port = (tByte) System_state_G;

    /* Remain in this state until water is hot NOTE: Timeout
       facility included here */
    if (++Time_in_state_G >= MAX_WATER_HEAT_DURATION){

        /* Should have warmed the water by now... */
        System_state_G = ERROR;
    }

    /* Check the water temperature */
    if (WASHER_Read_Water_Temperature() == 1) {

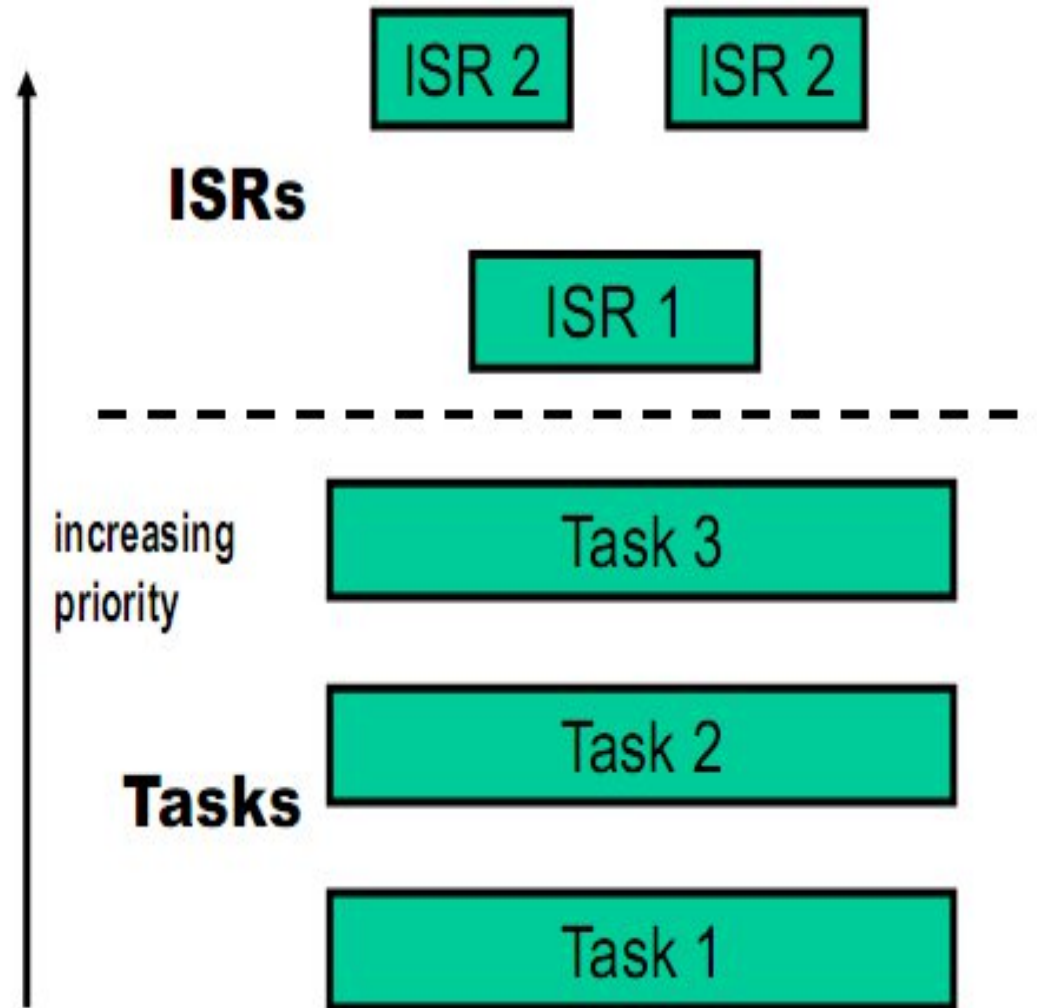
```

Comments on Multi-State System

- The use of the multi-state (timed and input/timed) systems in conjunction with an OS (like the aforementioned sEOS) is widespread in embedded applications thanks to the very flexibility.

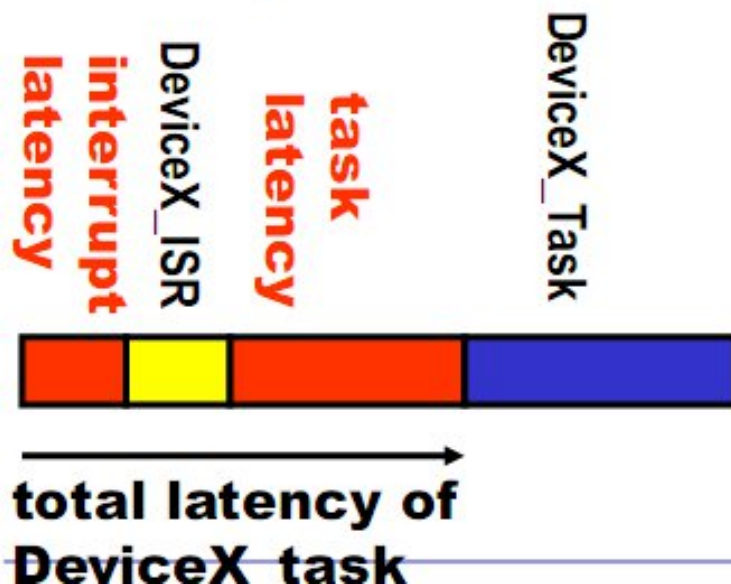
Interrupts and Tasks

- In an RTOS devices can still be handled by interrupts
 - All interrupts run at a higher priority than all tasks
 - ISRs must be written to inform RTOS that they are executing (RTOS-dependent mechanism)
- NOTE – all interrupts are higher in priority than all tasks
- ISRs can handle devices directly
- ISRs can signal tasks which then handle device (next slide)



Tasks Synchronised to Interrupts

- ◆ ISR for device X can signal a semaphore to unblock a waiting device-handler task
 - ❖ Make ISR as short as possible
 - ❖ time-critical code has interrupt latency
 - ❖ Rest of code also has task latency



```
DeviceX_ISR()  
{  
    [ time-critical handler code ]  
    SemaGiveFromISR(SemaX);  
}
```

```
DeviceX_Task()  
{  
    for (;;) {  
        SemaTake(SemaX); /*wait for device*/  
        [ rest of handler code ]  
    }  
}
```

Task allows most of handler code to execute in task context making full use of RTOS system objects

Summary: Interrupts in RTOS

- Simplest real-time systems are implemented using the foreground/background model using interrupts and background loop.
- Interrupts can be used to provide guaranteed latency & response-time
 - Interrupts have priority which determines precedence
 - Interrupt priorities are managed by device-dependent hardware or software.
 - All CPUs allow individual interrupt sources to be switched on or off which in principle allows software control of priorities
 - In practice for speed prioritised interrupts are managed by special hardware in big systems
- I/O operations can be performed:
 - From hardware-specific interrupt
 - From timer interrupt (polled)
 - From background loop (polled)
- RMA can be used to determine whether prioritised interrupt deadlines are met
 - More precise calculation must add interrupt entry and return times to ISR time when calculating RMA compute time C_i
 - Deadline is met if interrupt response-time is less than time between interrupts

Software Engineering for Embedded Systems

- Design for Composability
 - Configurable
 - Adaptable
 - Software reuse
 - Porting of Software
 - The ported software needs extensive checking, modification and re-validation
- Systematic validation
 - Co-design
 - Simulation

Software Engineering for Embedded Systems

- **Software Issues**

- Software maintenance is extremely important.
 - verification of proper operation,
 - updates,
 - fixing bugs,
 - adding features,
 - extending to new applications,
 - change user configurations.
- Good comment
 - how it works
 - why it is done this way
 - how to change
- Bad comment
 - what it is doing

Software types – Application and System

- Software/Program:
 - Software is an umbrella term that refers to anything [data](#) stored in hardware, in general practice, it usually refers to programs that you run on a device like a [computer](#) or PDA
 - Usage of the two words together is redundant, but very common since they are of the same meaning.
 - A program/software often refers to a file/a package of files to execute a function(s)/task(s)
- Application software/program
 - [Software](#) designed to help the user to perform specific tasks (e.g., process data for the user).
 - Examples: Word processors, email clients, media players, ...

Reference

- This document is used for educational purposes only
- This document contains material from other sources (incl. textbook, lecture notes, application notes, ...)
- Our thanks go to the contributors, references